
Conceptual Attention: Linear-Time Attention over Concept Memory

Anonymous Author(s)

Affiliation

Address

email

Abstract

We study online prediction when runtime memory can either repair payloads at fixed information or refine what future predictions can distinguish. Modeling memory as a finite partition Π , we prove an algorithm-independent obstruction: every adaptive repair-only learner whose predictions remain Π -measurable has expected regret at least $T(R_L(\Pi) - R_L(\Pi^*))$ against any refined oracle Π^* , up to refinement cost. Thus missing distinctions cause linear regret independently of the optimizer. We characterize valuable refinements by Bayes-risk decrease. The minimal binary update is the join $\Pi \vee U$; zero value is equivalent to the refined Bayes action being old-measurable. Under log loss the value is $I(Y; U \mid \Pi)$, and under squared loss it is conditional mean-separation energy. We give a sample-split guard accepting refinement only when penalized empirical gain beats the best repair gain by a concentration margin. Guided by the theory, we propose Conceptual Attention (CA): a fixed-bank sequence layer that replaces token-indexed attention memory with trainable concept identities and a bounded concept-value state, updated by retained forward writes and evaluated by an exact chunked affine scan. The first-principles extension is Concept Memory (CoM), whose test-time update is a gradient-free sufficient-statistic repair, such as responsibility-weighted counts, means, or least-squares statistics, rather than inner backpropagation through a neural state.

1 Introduction

We study a basic obstruction in online prediction with mutable memory. A runtime state is not merely a vector; it is an information structure determining which histories remain distinguishable. Once two cases induce the same state, every predictor that factors through that state must give them the same action. More computation, better local statistics, or a more elaborate optimizer can repair the action assigned to the state, but cannot use a distinction that the state has erased.

The paper formalizes this point independently of architecture. A runtime representation induces a partition Π of the case space. A *repair* update changes the payload assigned to atoms of Π while preserving the partition. A *refinement* update joins Π with a prediction-time observable distinction U , producing the finer partition $\Pi \vee U$. The question is therefore not whether a model updates at test time, but whether the update changes the events on which future predictions may condition.

The first result is an algorithm-independent obstruction. For any predictor $f \in \text{Pred}(\Pi)$ and any refined comparator Π^* ,

$$R_L(f) - R_L(\Pi^*) = \underbrace{R_L(f) - R_L(\Pi)}_{\text{repair gap}} + \underbrace{R_L(\Pi) - R_L(\Pi^*)}_{\text{refinement gap}}.$$

The first term can be attacked by estimation, optimization, caching, or local adaptation at fixed information. The second cannot. If every prediction remains Π -measurable, expected regret against a

refined oracle is at least $T(R_L(\Pi) - R_L(\Pi^*))$, up to a one-time refinement cost. Missing distinctions create linear regret not because the optimizer is weak, but because the admissible decision class is too coarse.

The second result identifies the unit of structural growth. A candidate bit is not valuable because it is balanced, geometrically salient, or frequent. It is valuable exactly to the extent that it changes the Bayes action. The minimal refinement is the join operation, and its value is the Bayes-risk decrease

$$V_L(U \mid \Pi) = R_L(\Pi) - R_L(\Pi \vee U).$$

For log loss this is $I(Y; U \mid \Pi)$, and for squared loss it is $\mathbb{E} \|\mathbb{E}[Y \mid \Pi, U] - \mathbb{E}[Y \mid \Pi]\|_2^2$. Under uniqueness, a refinement has zero value iff the refined Bayes action already has an old-measurable version. This rules out accepting distinctions merely because they separate inputs rather than consequences.

The third result is algorithmic. Population value is not directly observable, so refinement must be guarded. We give a sample-split rule: score repair and refinement candidates on a held-out buffer, charge structural cost, and accept refinement only when its penalized empirical gain beats the best repair gain by a uniform concentration margin. This is not a global training theorem; it is the statistical contract needed for a single adaptive structural update to mean what the theory says it means.

The theory leads us to propose Conceptual Attention (CA). A concept is an operational object: a persistent address for a prediction-relevant information cell, not necessarily a human-semantic category. The general DGM picture allows online concept and edge growth, but language-model scale requires a controlled architectural test. We therefore evaluate fixed-bank CA: each layer preallocates a bounded set of trainable concept identities and maintains only a mutable concept-value memory at runtime. Tokens read with attention over concept identities and write by retained continuous updates with trainable retention. The first-principles target is stronger than a value cache: a Concept Memory should update closed-form consequence statistics, such as counts, means, or least-squares predictors, so test-time learning is a forward sufficient-statistic update rather than inner backpropagation. In the current language-model probes, the strongest dense CoM variant is not sparse and does not use a write gate; it uses a parameter-matched local backbone plus a global dense Concept Memory whose key capacity knob is the consequence feature dimension d_u and its learned decoder. For parallel training and efficient CUDA execution, the implementation evaluates the exact read-before-write token recurrence by chunked affine scan. Chunking is a tiling and parallelization device: the code computes exact within-chunk causal reads after scanning chunk summaries, so the mathematical layer matches the streaming state update used at autoregressive inference.

This separation gives the paper’s contribution structure. Sections 2–4 are the theory: the repair-only obstruction, the value characterization, and the guarded refinement test. Section 5 presents the fixed-bank CA layer actually used for the main experiments, including its recurrent state, exact chunk-scan evaluation, and bounded-state accounting. Section 6 evaluates this proposal on state-equalized MQAR and matched language-model pretraining. The appendix contains the general graph-growing CA extension with online edge refinement, finite-class oracle inequalities, data-structure bounds, projection-memory bridges, kernel/scaling notes, and full proofs.

2 Online Prediction under Information Partitions

Let Ω be a finite case space, $\omega \sim P$, $Y : \Omega \rightarrow \mathcal{Y}$ a consequence variable, $\mathcal{A}$ an action space, and $L : \mathcal{Y} \times \mathcal{A} \rightarrow \mathbb{R}$ a loss with finite expectations. An information structure is a partition $\Pi = \{B_1, \dots, B_m\}$. We write $\Pi \preceq \Pi'$ when Π' refines Π . A predictor is Π -measurable when it is constant inside each atom:

$$\text{Pred}(\Pi) = \{f : \Omega \rightarrow \mathcal{A} : \omega, \omega' \in B \in \Pi \Rightarrow f(\omega) = f(\omega')\}.$$

The Bayes risk under Π is

$$R_L(\Pi) = \inf_{f \in \text{Pred}(\Pi)} \mathbb{E}[L(Y, f(\omega))] = \sum_{B \in \Pi} P(B) \inf_{a \in \mathcal{A}} \mathbb{E}[L(Y, a) \mid \omega \in B].$$

A repair update changes the payload $a(B)$ assigned to existing atoms; a refinement changes the equivalence relation itself. If a runtime representation is an information map $I : \Omega \rightarrow \mathcal{Z}$, then

81 it induces $\Pi(I) = \{I^{-1}(z) : I^{-1}(z) \neq \emptyset\}$, and every predictor of the form $a(I(\omega))$ is $\Pi(I)$ -
 82 measurable. This is the bridge to neural or memory implementations: the theorem constrains the
 83 quotient induced by the runtime state, not the syntax of the optimizer used to update that state. All
 84 statements extend to sub-sigma-algebras; the full measure-theoretic version and base-information
 85 convention are in Appendix D.

86 A binary distinction is a prediction-time observable map $U : \Omega \rightarrow \{0, 1\}$. We assume admissible
 87 distinctions lie in a pre-outcome observable class $\mathcal{U}$, so U_t does not depend on Y_t after observation.
 88 The join refinement is

$$\Pi \vee U = \{B \cap U^{-1}(0), B \cap U^{-1}(1) : B \in \Pi\} \setminus \{\emptyset\}.$$

89 **Definition 2.1** (Repair and refinement). *At round t , the learner predicts with $\hat{f}_t \in \text{Pred}(\Pi_t)$, observes*
 90 *Y_t , and then chooses one of two update types. A repair update has $\Pi_{t+1} = \Pi_t$ and may change only*
 91 *atom payloads, local memories, sufficient statistics, or downstream parameters. A refinement update*
 92 *has $\Pi_{t+1} = \Pi_t \vee U_t$ for some admissible distinction $U_t \in \mathcal{U}$. The optional cost model charges*
 93 *$\lambda c(U_t)$ only for refinement.*

94 **Theorem 2.2** (Risk decomposition, informal version of Theorem F.2). *For any $f_t \in \text{Pred}(\Pi_t)$ and*
 95 *any comparator Π^* with $\Pi_t \preceq \Pi^*$,*

$$R_L(f_t) - R_L(\Pi^*) = \underbrace{R_L(f_t) - R_L(\Pi_t)}_{\Delta_{\text{repair}}} + \underbrace{R_L(\Pi_t) - R_L(\Pi^*)}_{\Delta_{\text{refine}}}.$$

96 *Both terms are nonnegative.*

Proof sketch.

$$R_L(f_t) - R_L(\Pi^*) = R_L(f_t) - R_L(\Pi_t) + R_L(\Pi_t) - R_L(\Pi^*).$$

97 Since $f_t \in \text{Pred}(\Pi_t)$,

$$R_L(f_t) \geq R_L(\Pi_t).$$

98 Since Π^* refines Π_t ,

$$\text{Pred}(\Pi_t) \subseteq \text{Pred}(\Pi^*), \quad R_L(\Pi_t) \geq R_L(\Pi^*).$$

99 □

100 The first term is residual risk at fixed information; the second is the population value of changing
 101 information. Thus repair versus refinement is a choice between admissible prediction classes. The
 102 theorem below turns this distinction into a sequential lower bound.

103 **Theorem 2.3** (Repair-only linear regret, informal version of Theorem F.3). *Let Π be fixed and let*
 104 *$\Pi \preceq \Pi^*$. Consider any adaptive learner such that, for every t , $\hat{f}_t \in \text{Pred}(\Pi)$, possibly chosen using*
 105 *all previous observations. If $\omega_t \stackrel{\text{iid}}{\sim} P$, then*

$$\mathbb{E} \left[\sum_{t=1}^T L(Y_t, \hat{f}_t(\omega_t)) \right] \geq T R_L(\Pi).$$

106 *If Bayes actions exist under Π^* , then regret against the refined oracle satisfies*

$$\mathbb{E}[\text{Reg}_T(\Pi^*)] \geq T (R_L(\Pi) - R_L(\Pi^*)).$$

107 *If the infimum under Π^* is not attained, then for every $\epsilon > 0$ the same statement holds against an*
 108 *ϵ -optimal refined oracle with right side $T(R_L(\Pi) - R_L(\Pi^*) - \epsilon)$. If refinement cost is charged once*
 109 *as $\lambda C(\Pi^*)$, subtract $\lambda C(\Pi^*)$ from the bound.*

110 *Proof sketch.* Condition on $\mathcal{H}_{t-1}$. Since $\hat{f}_t \in \text{Pred}(\Pi)$,

$$\mathbb{E}[L(Y_t, \hat{f}_t(\omega_t)) \mid \mathcal{H}_{t-1}] \geq R_L(\Pi).$$

111 Therefore

$$\mathbb{E} \sum_{t=1}^T L(Y_t, \hat{f}_t(\omega_t)) \geq TR_L(\Pi).$$

112 If the refined oracle attains $R_L(\Pi^*)$ per round, subtraction gives

$$\mathbb{E}[\text{Reg}_T(\Pi^*)] \geq T(R_L(\Pi) - R_L(\Pi^*)).$$

113 For unattained infima, replace the oracle by an ϵ -optimal Π^* -measurable predictor; a one-time
114 refinement charge subtracts from the comparator advantage. $\square$

115 Once the quotient is fixed, no optimizer can exploit distinctions outside it. The refined oracle is
116 intentionally generous: it receives Π^* for free except for the optional one-time cost. The result
117 therefore does not claim that a refinement algorithm will find Π^* quickly; it says that refusing to
118 change the quotient pays the information gap every round.

119 3 Bayes-Risk Value of Refinement

120 The obstruction above says that a missing distinction can matter; it does not say which distinctions
121 should be added. The answer is loss value. For $D = U^{-1}(1)$, the minimal way to add a binary
122 observable U is

$$\Pi \vee U = \{B \cap D, B \cap D^c : B \in \Pi\} \setminus \{\emptyset\}.$$

123 **Proposition 3.1** (Minimal join, informal version of Theorem O.1). $\Pi \vee U$ is the unique coarsest
124 refinement of Π that makes U measurable.

125 *Proof sketch.* The nonempty sets $B \cap D$ and $B \cap D^c$ partition each old atom B , hence form a
126 refinement of Π . Since

$$D = \bigcup_{B \in \Pi} (B \cap D),$$

127 U is measurable under $\Pi \vee U$. If $\mathcal{Q}$ refines Π and makes D measurable, then every $Q \in \mathcal{Q}$ is
128 contained in either $B \cap D$ or $B \cap D^c$ for some $B \in \Pi$. Thus $\mathcal{Q}$ refines $\Pi \vee U$. $\square$

129 The population value of the distinction is

$$V_L(U \mid \Pi) = R_L(\Pi) - R_L(\Pi \vee U).$$

130 Risk monotonicity gives $V_L(U \mid \Pi) \geq 0$ at the population optimum, but finite implementations can
131 overfit a split; Section 4 therefore scores refinements on held-out data before committing them.

132 **Theorem 3.2** (Loss-relevant refinement, informal version of Theorem P.1). Let Π' refine Π . Assume
133 Bayes actions exist and the Π' -Bayes action is unique on positive-mass atoms. Then

$$R_L(\Pi) = R_L(\Pi')$$

134 if and only if the Π' -Bayes action has a Π -measurable version. Equivalently, zero value means the
135 refined Bayes action is constant across refined atoms inside each old atom.

136 *Proof sketch.* If $a_{\Pi'}^*$ is Π -measurable, it is feasible under Π , so

$$R_L(\Pi) \leq R_L(\Pi').$$

137 Monotonicity gives the reverse inequality. Conversely, if equality holds and a_{Π}^* is a Π -Bayes action,
138 then a_{Π}^* is also optimal under Π' . Uniqueness gives

$$a_{\Pi}^* = a_{\Pi'}^*$$

139 on every positive-mass refined atom. $\square$

140 For finite Y under log loss, $R_{\log}(\Pi) = H(Y \mid \Pi)$ and

$$\begin{aligned} V_{\log}(U \mid \Pi) &= I(Y; U \mid \Pi) \\ &= \sum_{B \in \Pi} P(B) \sum_{u \in \{0,1\}} P(U = u \mid B) D_{\text{KL}}(P(Y \mid B, U = u) \parallel P(Y \mid B)). \end{aligned}$$

141 For $Y \in L^2(\Omega; \mathbb{R}^d)$ and squared loss,

$$R_2(\Pi) - R_2(\Pi \vee U) = \mathbb{E} \left[\|\mathbb{E}[Y \mid \Pi, U] - \mathbb{E}[Y \mid \Pi]\|_2^2 \right].$$

142 Thus the value of a distinction is consequence value, not geometric novelty or marginal frequency.
 143 A clean cluster split can be worthless if both children have the same Bayes action, while a low-
 144 level bit can be valuable if it changes the conditional consequence. The appendix gives the full
 145 value-of-information identities and the coarsest loss-sufficient refinement.

146 4 Guarded Empirical Refinement

147 Population value must be estimated before a runtime learner commits a new distinction. For an atom
 148 B and candidate split $g : B \rightarrow \{0, 1\}$, the empirical log-loss improvement is

$$\hat{V}_{\log}(B, g) = H(\hat{p}_B) - \sum_{b \in \{0,1\}} \frac{|B_b|}{|B|} H(\hat{p}_{B_b}), \quad B_b = \{i \in B : g(x_i) = b\}.$$

149 The analogous squared-loss score is $\text{SSE}(B) - \text{SSE}(B_0) - \text{SSE}(B_1)$.

150 For a state S and held-out scoring buffer $\mathcal{B}_t^{\text{score}}$ of size m_t , define

$$\hat{R}(S; \mathcal{B}_t^{\text{score}}) = \frac{1}{m_t} \sum_{(\omega_i, Y_i) \in \mathcal{B}_t^{\text{score}}} L(Y_i, f_S(\omega_i)), \quad \hat{G}_t(S') = \hat{R}(S_t; \mathcal{B}_t^{\text{score}}) - \hat{R}(S'; \mathcal{B}_t^{\text{score}}).$$

151 Let $\mathcal{C}_t^{\text{rep}}$ be repair candidates and $\mathcal{C}_t^{\text{ref}}$ refinement candidates. Proposal may use a disjoint buffer, but
 152 the scoring buffer must be held out from proposal and previous state-selection decisions. The identity
 153 state is included among repair candidates, so a harmful local update can be declined.

154 **Theorem 4.1** (Guarded empirical refinement, informal version of Theorem Q.6). *Assume losses lie in*
 155 *$[0, 1]$. At time t , let $\mathcal{C}_t = \mathcal{C}_t^{\text{rep}} \cup \mathcal{C}_t^{\text{ref}}$ be predictable from the past and proposal buffer, and let $\mathcal{B}_t^{\text{score}}$*
 156 *contain m_t fresh held-out samples independent of candidate selection. Define*

$$\epsilon_t = \sqrt{\frac{\log(4t^2(|\mathcal{C}_t| + 1)/\delta)}{2m_t}}.$$

157 Then with probability at least $1 - \delta$, uniformly over all t and $S' \in \mathcal{C}_t$,

$$\left| \hat{G}_t(S') - G_t(S') \right| \leq 2\epsilon_t.$$

158 Consequently, if a refinement S_f is accepted because

$$\hat{G}_t(S_f) - \lambda c(S_f) > \hat{G}_t(S_r) + \tau + 4\epsilon_t,$$

159 where S_r is the best empirical repair candidate, then

$$G_t(S_f) - \lambda c(S_f) > G_t(S_r) + \tau.$$

160 *Proof sketch.* Condition on the past filtration. The old state and all candidate states are fixed.
 161 Hoeffding and a union bound give, uniformly over the old state and $S' \in \mathcal{C}_t$,

$$\left| \hat{R}(S') - R(S') \right| \leq \epsilon_t.$$

162 Since a gain is a difference of two risks,

$$\left| \hat{G}_t(S') - G_t(S') \right| \leq 2\epsilon_t.$$

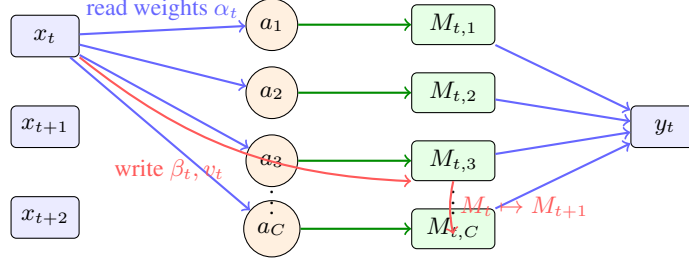


Figure 1: Conceptual Attention replaces token-indexed KV memory with a fixed learned concept bank and a mutable concept-value table. Tokens attend to concept identities for reads, then update the concept values by a retained forward write.

163 On this event,

$$\begin{aligned} G_t(S_f) - \lambda c(S_f) &\geq \hat{G}_t(S_f) - \lambda c(S_f) - 2\epsilon_t \\ &> \hat{G}_t(S_r) + \tau + 2\epsilon_t \\ &\geq G_t(S_r) + \tau. \end{aligned}$$

164 The t^{-2} schedule makes the bound simultaneous over all times. $\square$

165 The theorem is a finite-sample guard, not a global training theorem. The $4\epsilon_t$ term accounts for
 166 sampling uncertainty, while τ is a modeling margin for unmodeled costs or a desired minimum
 167 effect size. If the same validation buffer is reused adaptively, one needs reusable-holdout or adaptive-
 168 validation control. Appendix H gives broader greedy-recovery and oracle-inequality statements.

169 5 Conceptual Attention

170 Motivated by the preceding refine-or-repair theory, we propose Conceptual Attention (CA) as a
 171 fixed-capacity attention replacement. The layer used in our main experiments allocates a bounded
 172 bank of concept identities, keeps all slots active, disables graph edges, and updates only a concept-
 173 value memory during the forward pass. A concept is an operational address for a prediction-relevant
 174 information cell, not necessarily a human-semantic category. The graph-growing and edge-gated
 175 CA variant in the appendix is the dynamic refinement implementation; the main MQAR and CoM
 176 experiments use the fixed-bank, edge-free path.

177 Operationally, CA is linear attention over a learned concept bank. Standard causal attention stores
 178 one key-value pair per prefix token. CA instead stores a fixed number C of concept addresses and a
 179 recurrent value table over those addresses. For head h and concept slot j , the layer has a unit address

$$a_{hj} \in \mathbb{R}^{d_a}, \quad \|a_{hj}\|_2 = 1.$$

180 In the code path used for CoM and MQAR, these addresses are trainable layer parameters initialized
 181 as random unit vectors. They are *not* online sequence state: they are shared across examples and
 182 updated only by ordinary training. The online state for a sequence is the concept-value memory

$$M_t \in \mathbb{R}^{H \times C \times d_v^c}.$$

183 Thus the persistent runtime state per layer is

$$S_{CA} = CHd_v^c,$$

184 independent of prefix length. With delayed writes, the implementation also keeps a small key buffer
 185 of size $O(\delta Hd_a)$; the language-model setting uses $\delta = 0$.

186 **Token recurrence.** Given hidden states $x_t \in \mathbb{R}^{d_{\text{model}}}$, CA forms normalized read queries and write
 187 keys, concept values, and retention rates:

$$q_{th} = \text{normalize}(Q_h x_t), \quad k_{th} = \text{normalize}(K_h x_t),$$

188

$$v_{th} = V_h x_t, \quad \rho_{th} = \exp(-\lambda_{th}) \in (0, 1].$$

189 Let $\gamma > 0$ be the learned read/write scale. Token t first reads the current concept memory:

$$\alpha_{thj} = \text{softmax}_j(\gamma q_{th}^\top a_{hj}), \quad z_{th} = \sum_{j=1}^C \alpha_{thj} M_{t,hj}, \quad y_t = O[z_{t1}; \dots; z_{tH}].$$

190 Then it updates the memory. For a write delay $\delta \geq 0$, define $\bar{k}_{th} = k_{t-\delta,h}$ and a valid-write mask
 191 $m_t = \mathbf{1}\{t \geq \delta\}$. The language-model experiments use $\delta = 0$; MQAR can use $\delta = 1$ to bind a value
 192 token to the preceding key token. The write distribution is

$$\beta_{thj} = \text{softmax}_j(\gamma \bar{k}_{th}^\top a_{hj}),$$

193 and the read-before-write recurrence is

$$M_{t+1,hj} = \rho_{th} M_{t,hj} + m_t \beta_{thj} v_{th}. \quad (1)$$

194 This is the algorithm implemented by the streaming `step API`: each token consumes and returns a
 195 constant-size recurrent state, exactly as in linear attention or state-space decoding.

196 **Proposition 5.1** (Linear-time CA state). *Fix the learned CA parameters and concept addresses. For
 197 any prefix length T , all prefix dependence needed by future CA outputs is contained in M_T and, when
 198 $\delta > 0$, the delayed-key buffer. The per-layer online state is $CHd_v^c + O(\delta Hd_a)$ and the autoregressive
 199 update cost is $O(HC(d_a + d_v^c))$ per token, independent of T .*

200 *Proof sketch.* Equation (1) is Markov in $(M_t, \bar{k}_t)$. The next output reads only M_t and the current
 201 hidden state, and the next memory is a deterministic function of the same state and current projections.
 202 Induction over tokens gives the claim. The memory tensor has HCd_v^c scalars and the delayed-write
 203 buffer stores only δ normalized keys. $\square$

204 **Parallel training by chunk scan.** The token recurrence is sequential, but it has the same affine-scan
 205 structure used by linear recurrent attention. For token t define the additive update

$$U_{t,hj} = m_t \beta_{thj} v_{th}.$$

206 The transition $M \mapsto \rho_t M + U_t$ is an affine map. For a chunk $b = [s, e)$, the code computes the exact
 207 chunk summary

$$R_{b,h} = \prod_{t=s}^{e-1} \rho_{th}, \quad U_{b,hj} = \sum_{t=s}^{e-1} \left(\prod_{u=t+1}^{e-1} \rho_{uh} \right) m_t \beta_{thj} v_{th}.$$

208 The summary maps the entering memory to the exiting memory:

$$M_{e,hj} = R_{b,h} M_{s,hj} + U_{b,hj}.$$

209 Chunk summaries compose associatively:

$$(R_2, U_2) \odot (R_1, U_1) = (R_2 R_1, U_2 + R_2 U_1),$$

210 where R broadcasts over concepts and values. Therefore all chunk-entry memories can be computed
 211 by an exclusive associative scan over (R_b, U_b) .

212 After the scan recovers the entering memory for each chunk, the chunk read kernel also includes all
 213 earlier writes inside the same chunk. For token $t \in [s, e)$,

$$M_{t,hj} = \left(\prod_{u=s}^{t-1} \rho_{uh} \right) M_{s,hj} + \sum_{r=s}^{t-1} \left(\prod_{u=r+1}^{t-1} \rho_{uh} \right) m_r \beta_{r,hj} v_{rh}.$$

214 The output read uses this M_t . Thus chunking changes the evaluation order and exposes parallel dense
 215 kernels, but it is mathematically equivalent to the token recurrence in Equation (1). The block size is
 216 a tiling parameter, not a modeling approximation.

Algorithm 1 CAFORWARD as implemented

- 1: **Input:** hidden states $x_{1:L}$, learned concept identities $a_{1:C}$, initial memory $M_0 = 0$
 - 2: project all q_t, k_t, v_t, ρ_t and normalize q_t, k_t, a_j
 - 3: form dense concept read weights α_t and write weights β_t
 - 4: split the sequence into chunks of size B
 - 5: build exact affine summaries (R_b, U_b) for all chunks in parallel
 - 6: compute chunk-entry memories by exclusive associative scan over (R_b, U_b)
 - 7: for each chunk, compute exact read-before-write token outputs using its entry memory and within-chunk prior writes
 - 8: **return** output projection of the multi-head concept reads
-

217 **Trainable retention.** The factor ρ_{th} controls how much old concept memory survives a token write.
218 CA computes positive decay rates λ_{th} from the hidden state. Our default parameterization uses a
219 PoST-style ordered spectrum for base decay rates,

$$A_h = A_0 + \sum_{i < h} \text{softplus}(\Delta_i),$$

220 combined with learned input-dependent step sizes and optional position-adaptive scaling. This gives
221 CA a trainable forgetting spectrum, but the recurrence acts on concept-value memory rather than a
222 token-indexed KV cache.

223 **Training and inference.** During training, slow parameters are optimized by ordinary backpropaga-
224 tion through the read weights α , write weights β , values v_t , retention factors ρ_t , output projection,
225 and the scan recurrence. During inference, model weights and concept identities are fixed. The only
226 changing state is the forward-updated concept memory M_t and the optional delayed-key buffer. This
227 is the sense in which CA performs test-time learning: future predictions condition on a memory
228 learned from the observed prefix, without running backpropagation at inference time.

229 This fixed-bank recurrence is the additive repair form used in the reported experiments. CoM
230 replaces the unnormalized value write by closed-form sufficient-statistic updates in the same concept
231 coordinates, such as responsibility-weighted counts, means, or recursive least-squares statistics. That
232 extension preserves the same read/write concept interface while making the test-time update explicitly
233 gradient-free and consequence-aligned.

234 **Relation to ordinary attention.** Standard causal attention computes

$$\text{Attn}(X)_t = \sum_{s < t} \text{softmax}_s(q_t^\top K_s) V_s,$$

235 where the memory basis is the set of prefix token positions. CA computes

$$\text{CA}(X)_t = \sum_{j=1}^C \text{softmax}_j(q_t^\top a_j) M_{t,j},$$

236 where the memory basis is a learned concept bank and M_t is a recurrent value state. Thus CA is not a
237 different optimizer around attention. It is an attention replacement that moves the memory basis from
238 tokens to concepts while retaining differentiable attention-style reads and writes.

239 6 Experiments

240 We evaluate the fixed-bank CA layer from Section 5 on state-equalized MQAR and matched language-
241 model pretraining.

242 6.1 Experiment 1: Multi-Query Associative Recall

243 MQAR asks a model to output the value associated with a queried key [Arora et al., 2024]. We
244 state-equalize CA against efficient sequence mixers with $(d_{\text{model}}, H) \in \{(512, 4), (512, 8), (256, 4)\}$

and $d_{\text{state}} = d_{\text{model}}/(2H)$, giving online-state budgets 64K, 32K, and 16K. The evaluated implementation is fixed-bank and edge-free: $C = 128$ learned concept identities are active from the start, the chunk size is 32, the MQAR write key is shifted by one token, and the exact chunk-scan kernels are enabled. The per-head concept value width is chosen so that $S_{\text{CA}} = CHd_v^c$ stays within the matched budget, giving $d_v^c = 128$ for 64K and $d_v^c = 32$ for the 32K and 16K settings.

Table 1: **MQAR capacity accuracy (%)**. Test length is T with $K=T/4$ key-value pairs; all models train at $T=512$.

Model	state= 64K					state= 32K					state= 16K				
	512	1K	2K	4K	Avg	512	1K	2K	4K	Avg	512	1K	2K	4K	Avg
Mamba-2	100.0	96.8	62.2	18.9	69.5	99.2	85.2	41.3	11.6	59.4	99.3	80.6	31.2	5.7	54.2
RWKV-7	100.0	100.0	96.1	39.2	83.8	100.0	100.0	80.1	9.5	72.4	100.0	95.2	46.0	10.8	63.0
Gated DeltaNet	100.0	100.0	92.0	42.4	83.6	100.0	96.4	56.7	15.9	67.2	99.8	82.7	31.7	7.4	55.4
GLA	100.0	97.8	67.2	20.8	71.5	100.0	97.7	50.3	7.6	63.9	99.8	88.5	38.7	7.8	58.7
RetNet	99.9	47.1	2.3	0.0	37.3	99.9	63.2	6.0	0.3	42.3	96.8	16.8	0.7	0.0	28.6
CA (ours)	100.0	100.0	99.9	96.1	99.0	100.0	100.0	97.7	72.8	92.6	100.0	98.9	86.4	45.8	82.8

6.2 Experiment 2: Matched Language Modeling

CoM replaces only the sequence mixer by fixed-bank CA, keeps the PoST FineWeb-Edu setup, and evaluates base checkpoints with the Language Model Evaluation Harness. The current LM code uses chunk size 128, write-key shift 0, PoST-style position-adaptive retention, learned concept identities, unit write mass for valid tokens, and exact chunk-scan kernels. The full 180M CoM preset uses $C = 128$, $H = 12$, and $d_v^c = 128$ inside a fused mixer block, giving $S_{\text{CA}} = CHd_v^c = 196,608$ online state variables per CA layer. A separate CoM prototype uses a parameter-matched wide local/gated backbone and a dense global Concept Memory with $C = 128$; every token queries and updates every concept, while the consequence payload is compressed by the feature map described in Section K. The best current prototype uses a large sketched consequence width $d_u = 768 < d_{\text{model}} = 960$ with a learned decoder back to the residual stream. Layerwise, in-block, and multi-time-scale dense CoM variants are non-sparse ablations. The 440M preset uses $H = 32$ and $d_v^c = 64$, giving 262,144 state variables per CA layer.

Table 2 reports the current 180M training-curve probe. These are logged next-token training losses on the same streaming FineWeb-Edu protocol with batch size 2, sequence length 1024, seed 42, and no held-out perplexity evaluation; they are included to document the architectural direction before the final tail-sample PPL and zero-shot runs. The comparison is nevertheless useful because all rows use the same corpus stream and tokenizer, and Mamba-2 is run in the same suite with the stronger 10^{-3} learning rate. The result isolates the main empirical lesson: dense global CoM improves the wide local/gated backbone; increasing the consequence sketch from $d_u = 128$ to $d_u = 768$ closes the 16k-step gap to Mamba-2 while preserving a substantial speed advantage. Simple multi-scale CoM and in-block CoM did not improve this frontier in the current implementation.

Table 2: **CoM 180M training probe**. Loss is the last logged next-token training loss, not held-out PPL. All CoM rows use dense concept responsibilities with $C = 128$ and no sparse routing. Speed is trainer steps per second on one H100.

Model	Steps	d_u	Loss	Steps/s
Wide local/gated backbone, no CoM	16k	—	5.4758	37.34
Dense global CoM	16k	128	5.3295	34.90
Dense global CoM	16k	256	5.3245	34.11
Dense global CoM	16k	768	5.2823	30.96
Mamba-2 baseline	16k	—	5.2839	22.32
Dense global CoM	8k	128	5.5746	34.80
Dense global CoM	8k	256	5.5370	33.91
Dense global CoM	8k	768	5.5221	30.71
Mamba-2 baseline	8k	—	5.6096	22.04

Table 3 transcribes the PoST zero-shot accuracy table and reserves CoM rows; final same-corpus PPL uses the tail-sample protocol in Appendix L.

Table 3: **Downstream zero-shot LM accuracies.** PoST-protocol rows are reproduced; CoM rows are pending under the same harness.

Model	LAMB.	Hella.	PIQA	ARC-E	ARC-C	Wino.	OBQA	Avg
Mamba-2 189M	21.6	31.1	62.9	50.4	24.7	49.6	30.6	38.7
RWKV-7 189M	27.9	32.1	63.1	49.7	25.7	51.3	29.0	39.8
Gated DeltaNet 212M	23.8	31.9	62.7	49.6	24.2	51.1	30.6	39.1
CoM 212M	—	—	—	—	—	—	—	—
Mamba-2 440M	24.1	37.7	65.3	57.7	27.2	50.4	32.8	42.2
CoM 440M	—	—	—	—	—	—	—	—

7 Conclusion

We formulated concept updating as a risk-sensitive test-time decision: a representation should be refined only when the induced information split has positive predictive value after estimation error and complexity costs are paid. This yields two structural consequences: a repair-only memory cannot create distinctions absent from its observable state, and a valid refinement must be guarded by an empirical certificate tied to the downstream loss. Conceptual Attention instantiates the principle in the main experiments as a fixed-bank sequence layer that reads from learned concept identities and updates their value memory during the forward pass; the implementation evaluates the exact streaming recurrence by chunked affine scan. The predictive CoM prototype sharpens the test-time-learning claim: its fast state is a dense, gradient-free sufficient-statistic repair of observed consequences. The current training probes suggest that the main practical bottleneck is not sparse routing or a write gate, but the consequence-feature capacity and decoder used by this state. The appendix gives the graph-growing refinement extension. The remaining empirical question is quantitative: after the final zero-shot and same-corpus PPL evaluations are filled, CoM either matches the relevant recurrent baselines under the same protocol or exposes the concrete optimization and systems gaps that the theory does not by itself remove.

References

- Simran Arora, Sabri Eyuboglu, Aman Timalsina, Isys Johnson, Michael Poli, James Zou, Atri Rudra, and Christopher Ré. Zoology: Measuring and improving recall in efficient language models. In *International Conference on Learning Representations*, 2024.
- Jimmy Ba, Geoffrey E. Hinton, Volodymyr Mnih, Joel Z. Leibo, and Catalin Ionescu. Using fast weights to attend to the recent past. In *Advances in Neural Information Processing Systems*, volume 29, 2016.
- James O. Berger. *Statistical Decision Theory and Bayesian Analysis*. Springer, 2 edition, 1985.
- Albert Bifet and Ricard Gavalda. Adaptive learning from evolving data streams. In *Advances in Intelligent Data Analysis VIII*, volume 5772 of *Lecture Notes in Computer Science*, pages 249–260. Springer, 2009. doi: 10.1007/978-3-642-03915-7_22.
- Patrick Billingsley. *Probability and Measure*. Wiley, 3 edition, 1995.
- David Blackwell. Equivalent comparisons of experiments. *The Annals of Mathematical Statistics*, 24(2):265–272, 1953. doi: 10.1214/aoms/1177729032.
- Sebastian Borgeaud, Arthur Mensch, Jordan Hoffmann, Trevor Cai, Eliza Rutherford, Katie Millican, George B. van den Driessche, Jean-Baptiste Lespiau, Bogdan Damoc, Aidan Clark, Diego de Las Casas, Aurelia Guy, Jacob Menick, Roman Ring, Tom Hennigan, Saffron Huang, Loren Maggiore, Chris Jones, Albin Cassirer, Andy Brock, Michela Paganini, Geoffrey Irving, Oriol Vinyals, Simon Osindero, Karen Simonyan, Jack Rae, Erich Elsen, and Laurent Sifre. Improving language models by retrieving from trillions of tokens. In *Proceedings of the 39th International Conference on Machine Learning*, volume 162 of *Proceedings of Machine Learning Research*, pages 2206–2240, 2022.
- Lev M. Bregman. The relaxation method of finding the common point of convex sets and its application to the solution of problems in convex programming. *USSR Computational Mathematics and Mathematical Physics*, 7(3):200–217, 1967. doi: 10.1016/0041-5553(67)90040-7.
- Leo Breiman, Jerome H. Friedman, Richard A. Olshen, and Charles J. Stone. *Classification and Regression Trees*. Wadsworth, 1984.
- Glenn W. Brier. Verification of forecasts expressed in terms of probability. *Monthly Weather Review*, 78(1):1–3, 1950. doi: 10.1175/1520-0493(1950)078<0001:VOFEIT>2.0.CO;2.
- Gail A. Carpenter and Stephen Grossberg. ART 2: Self-organization of stable category recognition codes for analog input patterns. *Applied Optics*, 26(23):4919–4930, 1987a. doi: 10.1364/AO.26.004919.
- Gail A. Carpenter and Stephen Grossberg. A massively parallel architecture for a self-organizing neural pattern recognition machine. *Computer Vision, Graphics, and Image Processing*, 37(1):54–115, 1987b. doi: 10.1016/S0734-189X(87)80014-2.
- Nicolò Cesa-Bianchi and Gábor Lugosi. *Prediction, Learning, and Games*. Cambridge University Press, 2006. doi: 10.1017/CBO9780511546921.
- Thomas M. Cover and Joy A. Thomas. *Elements of Information Theory*. Wiley, 2 edition, 2006.
- Koby Crammer, Ofer Dekel, Joseph Keshet, Shai Shalev-Shwartz, and Yoram Singer. Online passive-aggressive algorithms. *Journal of Machine Learning Research*, 7:551–585, 2006.
- Luc Devroye, László Györfi, and Gábor Lugosi. *A Probabilistic Theory of Pattern Recognition*. Springer, 1996.
- Pedro Domingos and Geoff Hulten. Mining high-speed data streams. In *Proceedings of the Sixth ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, pages 71–80, 2000. doi: 10.1145/347090.347107.

335 Abhimanyu Dubey, Abhinav Jauhri, Abhinav Pandey, Abhishek Kadian, Ahmad Al-Dahle, Aiesha
336 Letman, Akhil Mathur, Alan Schelten, Amy Yang, Angela Fan, et al. The Llama 3 herd of models.
337 *arXiv preprint arXiv:2407.21783*, 2024.

338 Cynthia Dwork, Vitaly Feldman, Moritz Hardt, Toniann Pitassi, Omer Reingold, and Aaron Roth.
339 Generalization in adaptive data analysis and holdout reuse. In *Advances in Neural Information*
340 *Processing Systems*, volume 28, pages 2350–2358, 2015a.

341 Cynthia Dwork, Vitaly Feldman, Moritz Hardt, Toniann Pitassi, Omer Reingold, and Aaron Roth.
342 The reusable holdout: Preserving validity in adaptive data analysis. *Science*, 349(6248):636–638,
343 2015b. doi: 10.1126/science.aaa9375.

344 William Fedus, Barret Zoph, and Noam Shazeer. Switch transformers: Scaling to trillion parameter
345 models with simple and efficient sparsity. *Journal of Machine Learning Research*, 23(120):1–39,
346 2022.

347 Yoav Freund and Robert E. Schapire. A decision-theoretic generalization of on-line learning and
348 an application to boosting. *Journal of Computer and System Sciences*, 55(1):119–139, 1997. doi:
349 10.1006/jcss.1997.1504.

350 Bernd Fritzsche. A growing neural gas network learns topologies. In *Advances in Neural Information*
351 *Processing Systems*, volume 7, pages 625–632, 1995.

352 Bernhard Ganter and Rudolf Wille. *Formal Concept Analysis: Mathematical Foundations*. Springer,
353 1999.

354 Leo Gao, Jonathan Tow, Baber Abbasi, Stella Biderman, Sid Black, Anthony DiPofi, Charles Foster,
355 Laurence Golding, Jeffrey Hsu, Alain Le Noac’h, et al. A framework for few-shot language model
356 evaluation. <https://github.com/EleutherAI/lm-evaluation-harness>, 2024.

357 Tilmann Gneiting and Adrian E. Raftery. Strictly proper scoring rules, prediction, and es-
358 timation. *Journal of the American Statistical Association*, 102(477):359–378, 2007. doi:
359 10.1198/016214506000001437.

360 Edouard Grave, Moustapha M. Cisse, and Armand Joulin. Improving neural language models with a
361 continuous cache. In *International Conference on Learning Representations*, 2017.

362 Alex Graves, Greg Wayne, and Ivo Danihelka. Neural turing machines, 2014.

363 Alex Graves, Greg Wayne, Malcolm Reynolds, Tim Harley, Ivo Danihelka, Agnieszka Grabska-
364 Barwińska, Sergio Gómez Colmenarejo, Edward Grefenstette, Tiago Ramalho, John Agapiou,
365 Adrià Puigdomènech Badia, Karl Moritz Hermann, Yori Zwols, Georg Ostrovski, Adam Cain,
366 Helen King, Christopher Summerfield, Phil Blunsom, Koray Kavukcuoglu, and Demis Hassabis.
367 Hybrid computing using a neural network with dynamic external memory. *Nature*, 538(7626):
368 471–476, 2016. doi: 10.1038/nature20101.

369 Stephen Grossberg. Adaptive resonance theory: How a brain learns to consciously attend, learn, and
370 recognize a changing world. *Neural Networks*, 37:1–47, 2013. doi: 10.1016/j.neunet.2012.09.017.

371 Chuan Guo, Geoff Pleiss, Yu Sun, and Kilian Q. Weinberger. On calibration of modern neural
372 networks. In *Proceedings of the 34th International Conference on Machine Learning*, volume 70
373 of *Proceedings of Machine Learning Research*, pages 1321–1330, 2017.

374 Wassily Hoeffding. Probability inequalities for sums of bounded random variables. *Journal of the*
375 *American Statistical Association*, 58(301):13–30, 1963. doi: 10.1080/01621459.1963.10500830.

376 Arthur E. Hoerl and Robert W. Kennard. Ridge regression: Biased estimation for nonorthogonal
377 problems. *Technometrics*, 12(1):55–67, 1970. doi: 10.1080/00401706.1970.10488634.

378 Neil Houlsby, Andrei Giurgiu, Stanislaw Jastrzebski, Bruna Morrone, Quentin de Laroussilhe, Andrea
379 Gesmundo, Mona Attariyan, and Sylvain Gelly. Parameter-efficient transfer learning for NLP. In
380 *Proceedings of the 36th International Conference on Machine Learning*, volume 97 of *Proceedings*
381 *of Machine Learning Research*, pages 2790–2799, 2019.

Edward J. Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yuanzhi Li, Shean Wang, Lu Wang, and Weizhu Chen. LoRA: Low-rank adaptation of large language models. In *International Conference on Learning Representations*, 2022. URL <https://openreview.net/forum?id=nZeVKeeFYf9>.

Piotr Indyk and Rajeev Motwani. Approximate nearest neighbors: Towards removing the curse of dimensionality. In *Proceedings of the Thirtieth Annual ACM Symposium on Theory of Computing*, pages 604–613, 1998. doi: 10.1145/276698.276876.

Jeff Johnson, Matthijs Douze, and Hervé Jégou. Billion-scale similarity search with GPUs. *IEEE Transactions on Big Data*, 7(3):535–547, 2019. doi: 10.1109/TBDATA.2019.2921572.

Olav Kallenberg. *Foundations of Modern Probability*. Springer, 2 edition, 2002.

Angelos Katharopoulos, Apoorv Vyas, Nikolaos Pappas, and François Fleuret. Transformers are RNNs: Fast autoregressive transformers with linear attention. In *Proceedings of the 37th International Conference on Machine Learning*, volume 119 of *Proceedings of Machine Learning Research*, pages 5156–5165, 2020.

Urvashi Khandelwal, Omer Levy, Dan Jurafsky, Luke Zettlemoyer, and Mike Lewis. Generalization through memorization: Nearest neighbor language models. In *International Conference on Learning Representations*, 2020.

Diederik P. Kingma and Jimmy Ba. Adam: A method for stochastic optimization. In *International Conference on Learning Representations*, 2015.

James Kirkpatrick, Razvan Pascanu, Neil Rabinowitz, Joel Veness, Guillaume Desjardins, Andrei A. Rusu, Kieran Milan, John Quan, Tiago Ramalho, Agnieszka Grabska-Barwinska, Demis Hassabis, Claudia Clopath, Dharshan Kumaran, and Raia Hadsell. Overcoming catastrophic forgetting in neural networks. *Proceedings of the National Academy of Sciences*, 114(13):3521–3526, 2017. doi: 10.1073/pnas.1611835114.

Pang Wei Koh, Thao Nguyen, Yew Siang Tang, Stephen Mussmann, Emma Pierson, Been Kim, and Percy Liang. Concept bottleneck models. In *Proceedings of the 37th International Conference on Machine Learning*, volume 119 of *Proceedings of Machine Learning Research*, pages 5338–5348, 2020.

Teuvo Kohonen. The self-organizing map. *Proceedings of the IEEE*, 78(9):1464–1480, 1990. doi: 10.1109/5.58325.

Teuvo Kohonen. *Self-Organizing Maps*, volume 30 of *Springer Series in Information Sciences*. Springer, 1995.

Balaji Lakshminarayanan, Daniel M. Roy, and Yee Whye Teh. Mondrian forests: Efficient online random forests. In *Advances in Neural Information Processing Systems*, volume 27, 2014.

Guillaume Lample, Alexandre Sablayrolles, Marc’Aurelio Ranzato, Ludovic Denoyer, and Herve Jegou. Large memory layers with product keys. In *Advances in Neural Information Processing Systems*, volume 32, 2019.

Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Küttler, Mike Lewis, Wen-tau Yih, Tim Rocktäschel, Sebastian Riedel, and Douwe Kiela. Retrieval-augmented generation for knowledge-intensive NLP tasks. In *Advances in Neural Information Processing Systems*, volume 33, pages 9459–9474, 2020.

David Lopez-Paz and Marc’Aurelio Ranzato. Gradient episodic memory for continual learning. In *Advances in Neural Information Processing Systems*, volume 30, 2017.

Ilya Loshchilov and Frank Hutter. Decoupled weight decay regularization. In *International Conference on Learning Representations*, 2019.

Anton Lozhkov, Loubna Ben Allal, Leandro von Werra, and Thomas Wolf. FineWeb-Edu: The finest collection of educational content the web has to offer. *Hugging Face Blog*, 2024. <https://huggingface.co/spaces/HuggingFaceFW/blogpost-fineweb-v1>.

430 Chaitanya Manapragada, Geoffrey I. Webb, and Mahsa Salehi. Extremely fast decision tree. In
431 *Proceedings of the 24th ACM SIGKDD International Conference on Knowledge Discovery and*
432 *Data Mining*, pages 1953–1962, 2018. doi: 10.1145/3219819.3220005.

433 Thomas M. Martinetz and Klaus J. Schulten. A neural-gas network learns topologies. In *Artificial*
434 *Neural Networks*, pages 397–402, 1991.

435 Shuaicheng Niu, Jiayang Wu, Yifan Zhang, Yaofo Chen, Shihua Zheng, Peilin Zhao, and Minghui
436 Tan. Efficient test-time model adaptation without forgetting. In *Proceedings of the 39th Inter-*
437 *national Conference on Machine Learning*, volume 162 of *Proceedings of Machine Learning*
438 *Research*, pages 16888–16905, 2022.

439 Roger Penrose. A generalized inverse for matrices. *Mathematical Proceedings of the Cambridge*
440 *Philosophical Society*, 51(3):406–413, 1955. doi: 10.1017/S0305004100030401.

441 J. R. Quinlan. Induction of decision trees. *Machine Learning*, 1(1):81–106, 1986. doi: 10.1007/
442 BF00116251.

443 Jack W. Rae, Jonathan J. Hunt, Tim Harley, Ivo Danihelka, Andrew Senior, Greg Wayne, Alex Graves,
444 and Timothy Lillicrap. Scaling memory-augmented neural networks with sparse reads and writes.
445 In *Advances in Neural Information Processing Systems*, volume 29, 2016.

446 David E. Rumelhart, Geoffrey E. Hinton, and Ronald J. Williams. Learning representations by
447 back-propagating errors. *Nature*, 323(6088):533–536, 1986. doi: 10.1038/323533a0.

448 Andrei A. Rusu, Neil C. Rabinowitz, Guillaume Desjardins, Hubert Soyer, James Kirkpatrick, Koray
449 Kavukcuoglu, Razvan Pascanu, and Raia Hadsell. Progressive neural networks, 2016.

450 Imanol Schlag, Kazuki Irie, and Jürgen Schmidhuber. Linear transformers are secretly fast weight
451 programmers. In *Proceedings of the 38th International Conference on Machine Learning*, volume
452 139 of *Proceedings of Machine Learning Research*, pages 9355–9366, 2021.

453 Shai Shalev-Shwartz. Online learning and online convex optimization. *Foundations and Trends in*
454 *Machine Learning*, 4(2):107–194, 2012. doi: 10.1561/22000000018.

455 Noam Shazeer, Azalia Mirhoseini, Krzysztof Maziarz, Andy Davis, Quoc Le, Geoffrey Hinton, and
456 Jeff Dean. Outrageously large neural networks: The sparsely-gated mixture-of-experts layer. In
457 *International Conference on Learning Representations*, 2017.

458 Jake Snell, Kevin Swersky, and Richard S. Zemel. Prototypical networks for few-shot learning. In
459 *Advances in Neural Information Processing Systems*, volume 30, 2017.

460 Sainbayar Sukhbaatar, Jason Weston, Rob Fergus, et al. End-to-end memory networks. In *Advances*
461 *in Neural Information Processing Systems*, volume 28, 2015.

462 Yu Sun, Xiaolong Wang, Zhuang Liu, John Miller, Alexei A. Efros, and Moritz Hardt. Test-time
463 training with self-supervision for generalization under distribution shifts. In *Proceedings of the*
464 *37th International Conference on Machine Learning*, volume 119 of *Proceedings of Machine*
465 *Learning Research*, pages 9229–9248, 2020.

466 Yu Sun, Xinhao Li, Karan Dalal, Jiarui Xu, Arjun Vikram, Genghan Zhang, Yann Dubois, Xinlei
467 Chen, Xiaolong Wang, Sanmi Koyejo, Tatsunori Hashimoto, and Carlos Guestrin. Learning to
468 (learn at test time): RNNs with expressive hidden states. In *Proceedings of the 42nd International*
469 *Conference on Machine Learning*, volume 267 of *Proceedings of Machine Learning Research*,
470 pages 57503–57522, 2025.

471 Andrey N. Tikhonov and Vasilii Y. Arsenin. *Solutions of Ill-Posed Problems*. Winston, 1977.

472 Vladimir G. Vovk. Aggregating strategies. In *Proceedings of the Third Annual Workshop on*
473 *Computational Learning Theory*, pages 371–383. Morgan Kaufmann, 1990.

474 Volodya Vovk. Competitive on-line linear regression. In *Advances in Neural Information Processing*
475 *Systems*, volume 10, 1997.

- 476 Dequan Wang, Evan Shelhamer, Shaoteng Liu, Bruno Olshausen, and Trevor Darrell. Tent: Fully test-
477 time adaptation by entropy minimization. In *International Conference on Learning Representations*,
478 2021.
- 479 Jason Weston, Sumit Chopra, and Antoine Bordes. Memory networks. In *International Conference*
480 *on Learning Representations*, 2015.
- 481 Yuhuai Wu, Markus N. Rabe, DeLesley Hutchins, and Christian Szegedy. Memorizing transformers.
482 In *International Conference on Learning Representations*, 2022.
- 483 Songlin Yang, Jan Kautz, and Ali Hatamizadeh. Gated delta networks: Improving mamba2 with
484 delta rule. In *International Conference on Learning Representations*, 2025.
- 485 Jaehong Yoon, Eunho Yang, Jeongtae Lee, and Sung Ju Hwang. Lifelong learning with dynamically
486 expandable networks. In *International Conference on Learning Representations*, 2018.
- 487 Marvin Zhang, Sergey Levine, and Chelsea Finn. MEMO: Test time robustness via adaptation
488 and augmentation. In *Advances in Neural Information Processing Systems*, volume 35, pages
489 38629–38642, 2022.
- 490 Martin Zinkevich. Online convex programming and generalized infinitesimal gradient ascent. In
491 *Proceedings of the 20th International Conference on Machine Learning*, pages 928–936, 2003.

Appendix

A Appendix Roadmap

The appendix follows the same logical chain as the main text, but with all modeling assumptions, proof details, and implementation bridges made explicit. It first states what the results do not prove, then places the contribution relative to prior work, and only then develops the formal machinery. The proof appendices are ordered so that each later argument can reuse objects already defined earlier: partitions and joins, value identities, graph-generated information structures, guarded refinement, and finally projection-memory bridges.

- **Part 1.** Appendix B fixes the claim boundary: the lower bound is information-theoretic, the guarded rule is a sample-split validation statement, and the large-model layer is an architectural proposal rather than a scaling theorem.
- **Part 2.** Appendix C compares CA to repair-only memories, refinement-only category systems, routing architectures, and decision-theoretic value-of-information results.
- **Part 3.** Appendices D–I give the full abstract model. They define partition-measurable predictors, repair and refinement, Bayes-risk gaps, minimal joins, value of a distinction, greedy realizability, and finite-class oracle inequalities.
- **Part 4.** Appendix J turns the abstract model into a graph data structure. It defines hard-route CA, the edge-generated partition, the soft-routing approximation, the refine-or-repair acceptance rule, and the data-structure costs.
- **Part 5.** Appendices K, M, and N connect the theory to sequence models. They state the CoM read/write layer, the Projection Memory and CPM bridge, tree special cases, systems notes, and closed-form local repair.
- **Part 6.** Appendices O, P, and Q collect the formal proofs in dependency order: first minimal information updates, then Bayes-risk value identities, then graph-routing, validation, data-structure, and projection-memory statements.

B Limitations

The theory identifies when a distinction is necessary or valuable at the population-information level. It does not solve the search problem of finding the best distinction in a rich hypothesis class. The online algorithm is greedy and local: its refinement rule concerns local concepts and their nearby conflicts, not global optimality over all information structures.

The risk identities assume correctly estimated conditional distributions or conditional means. With finite data, refinement can increase estimation error, so useful population distinctions may be statistically unsafe. Complexity penalties, thresholds, validation, Bayesian priors, or minimum leaf sizes are needed in practical systems.

The finite-sample validation theorem assumes a genuinely held-out scoring buffer: fresh samples, or samples not previously used to choose proposals or update the state. Reusing the same validation buffer after previous accept/reject decisions makes the candidate class adaptive to that buffer and requires reusable-holdout or other adaptive-validation machinery beyond the Hoeffding argument stated here.

The base-information convention is restricted. If the full hidden state is made freely conditionable in the Bayes-risk benchmark, then all CA edge predicates and anchor-order events are already measurable and the sigma-algebra refinement gap vanishes. The strict partition theory therefore applies to the current runtime information or to restricted hard-route prediction classes; h-dependent soft memories are implementation approximations or restricted-class extensions.

The graph-growing CA extension specifies one canonical refinement mechanism, based on concepts and local distinction edges, but this is not claimed to be optimal. Rich domains may require learned concept-creation rules, approximate nearest-neighbor indices, nonlinear distinction edges, and better merge, forgetting, or retention-decay policies than the default budget notes in Appendix N. The bridge to Projection Memory and CPM is exact for one-hot finite partition memories. Continuous

neural representations need additional modeling choices: when a continuous conflict should create a new concept, how local projection memories should be inherited, how retention time scales should be allocated, and how refined information structures interact with differentiable backbones.

CA’s dense read avoids sparse candidate-recall assumptions at the concept budgets used in the main experiments, and the current implementation includes fused kernels for the dense concept operations and chunk scan. Large-budget CA will still need sparse concept retrieval, sharded concept banks, and additional fused write/read kernels; at that point candidate recall again becomes an engineering assumption that should be measured in large systems. Edge gates can reject incompatible candidates in the graph-growing extension, but they cannot select a compatible concept that the retrieval layer never exposes.

The large-model layer is an architectural path rather than a completed scaling theorem. It gives a trainable read path, a continuous write path, a trainable retention decay, and a default budget policy with explicit complexity, but it does not prove optimization guarantees for deep nonconvex backbones, nor does it solve distributed systems issues such as load balancing or stale concept shards. The reported fixed-bank implementation now evaluates the exact token-level CA recurrence by chunked affine scan: chunk size affects kernel tiling and parallel depth, not the mathematical causal rule. The remaining systems limitation is efficiency rather than causality: dense CA in every layer is expensive relative to highly optimized Mamba-2/RWKV kernels, so current LM prototypes allocate the dense CoM memory globally and treat layerwise or in-block CoM placements as ablations. The Concept Memory results also show that consequence-feature capacity and the learned decoder are important systems choices: too small a d_u underfits the repaired consequence statistic, while the direct $d_u = d_{\text{model}}$ endpoint is not automatically superior in the present residual interface. Budget maintenance in the graph-growing extension is outside the formal refinement theorem: merge, eviction, decay, and compression may discard distinctions, so their effect must be evaluated as a resource-management approximation.

The experiments are a focused sequence-modeling benchmark suite, not a full large-model scaling result. The CoM training probe in Table 2 is a same-stream training-loss comparison, not a final held-out perplexity or downstream zero-shot evaluation. It should be read as evidence for the dense CoM design direction, especially the consequence-sketch capacity result, not as evidence that CA already dominates optimized attention, Mamba, RWKV, GDN, or TTT implementations at production scale. A complete empirical evaluation should include state-equalized MQAR, language modeling at multiple parameter and token budgets, held-out same-corpus PPL, zero-shot task metrics, latency and memory curves, and ablations of chunk size, write-key shift, retention parameterization, consequence-feature dimension, fixed-bank versus dynamic concepts, and edge-free versus edge-gated variants.

C Related Work

The relevant comparison is not whether CA uses centers, memory, routing, or online updates. All of these ingredients have long histories. The question is whether a runtime learner distinguishes two operations that are often collapsed: repairing state inside the current representation and refining the representation when consequences reveal a missing distinction.

C.1 Methods that repair without refining

Cache language models and k NN-LMs store or retrieve context-token examples at inference time [Grave et al., 2017, Khandelwal et al., 2020]. Memorizing Transformers, RETRO, RAG, and sparse retrieval systems extend this idea with larger or external stores [Wu et al., 2022, Borgeaud et al., 2022, Lewis et al., 2020, Rae et al., 2016]. These methods make past examples available, but the default response to surprise is more storage or retrieval, not the creation of a new runtime concept forced by consequence conflict.

Fast weights, linear attention memories, delta-rule updates, passive-aggressive learning, online ridge/RLS, and Projection Memory repair associative state inside a fixed geometry [Ba et al., 2016, Katharopoulos et al., 2020, Schlag et al., 2021, Crammer et al., 2006, Vovk, 1997, Bregman, 1967, Tikhonov and Arsenin, 1977, Hoerl and Kennard, 1970]. Adapter modules and LoRA provide parameter-efficient adaptation state in large pretrained models [Houlsby et al., 2019, Hu et al., 2022]. Conservative Projection Memory (CPM) is especially close to our repair side because it detects zero

593 innovation with nonzero residual as a conflict. CA uses this diagnosis differently: a zero-innovation
594 conflict is evidence that the current information cell is too coarse, so projection-style repair should
595 stop until the information structure is refined.

596 Test-time adaptation and test-time training update model state or parameters by optimizing an
597 inference-time objective [Sun et al., 2020, Wang et al., 2021, Niu et al., 2022, Zhang et al., 2022, Sun
598 et al., 2025]. These methods ask what surrogate loss should be optimized at test time. CA asks a prior
599 structural question: is the error repairable under the current distinctions, or does it reveal a missing
600 distinction?

601 These methods solve the repair-gap side of runtime learning. They do not, by themselves, provide an
602 explicit consequence-forced refinement operation.

603 C.2 Methods that refine without local repair

604 Decision trees, online trees, Hoeffding trees, adaptive trees, extremely fast decision trees, and Mon-
605 drian forests refine leaves using information gain, variance reduction, or streaming split tests [Quinlan,
606 1986, Breiman et al., 1984, Domingos and Hulten, 2000, Bifet and Gavaldà, 2009, Manapragada
607 et al., 2018, Lakshminarayanan et al., 2014]. These are the closest algorithmic relatives of CA’s
608 refinement test. The difference is structural: trees store distinctions in a rooted hierarchy, while CA
609 stores local pairwise distinction edges between runtime concepts and attaches repairable memories to
610 the resulting nodes.

611 Adaptive Resonance Theory, including ART 1 for binary patterns and ART 2 for analog input
612 patterns, addresses stable incremental category creation under mismatch and vigilance [Carpenter
613 and Grossberg, 1987b,a, Grossberg, 2013]. Prototype methods, LVQ, SOMs, neural gas, growing
614 neural gas, and Prototypical Networks represent regions or classes by stored centers or exemplars
615 [Kohonen, 1990, 1995, Martinetz and Schulten, 1991, Fritzke, 1995, Snell et al., 2017]. CA shares
616 the vocabulary of category creation and prototypes, but its concepts are operational hidden-state
617 concepts whose edges are consequence-forced binary generators. The node is not only a region; it is
618 a place where local memory can be repaired after the distinction is accepted.

619 Concept Bottleneck Models and Formal Concept Analysis emphasize explicit concept structures [Koh
620 et al., 2020, Ganter and Wille, 1999]. CA does not claim human-interpretable concepts by default.
621 Its concepts are runtime concepts: hidden-state anchors, centroids, local memories, and pairwise
622 distinctions created from prediction consequences.

623 These methods solve the refinement half in various forms. They typically do not pair refinement with
624 graph-gated neural local memories that continue to be repaired after the split.

625 C.3 Methods that route without runtime concept creation

626 Mixture-of-experts, Switch Transformers, and Product-Key Memory route tokens or queries to sparse
627 trained capacity [Shazeer et al., 2017, Fedus et al., 2022, Lample et al., 2019]. Memory Networks,
628 End-to-End Memory Networks, Neural Turing Machines, and Differentiable Neural Computers learn
629 external read/write controllers [Weston et al., 2015, Sukhbaatar et al., 2015, Graves et al., 2014, 2016].
630 These systems are important precedents for sparse reads and external memory. Fixed-bank CA differs
631 in write semantics: the routed objects are learned concept identities whose runtime value memories
632 are repaired by the observed prefix. The graph-growing extension goes further by creating runtime
633 concepts and edges whose predicates generate an information structure.

634 Continual learning studies forgetting, replay, regularization, and capacity growth across task se-
635 quences [Kirkpatrick et al., 2017, Lopez-Paz and Ranzato, 2017, Rusu et al., 2016, Yoon et al., 2018].
636 CA is complementary: it is an instance-level runtime data structure for local concept growth and
637 memory repair, not a complete task-sequence continual learning system.

638 C.4 CA’s position

639 CA combines refinement and repair but keeps them distinct:

$$\Delta_{\text{refine}} \text{ dominates } \Rightarrow \text{refine}, \quad \Delta_{\text{repair}} \text{ dominates } \Rightarrow \text{repair}.$$

640 The measure-theoretic formalism follows standard sigma-algebra and conditional-expectation lan-
641 guage [Billingsley, 1995, Kallenberg, 2002]. The Bayes-risk order is related to Blackwell comparison

Symbol	Meaning
Ω	finite case space
P	distribution over cases
$Y : \Omega \rightarrow \mathcal{Y}$	consequence or label variable
$\mathbf{A}$	action space
$L : \mathcal{Y} \times \mathbf{A} \rightarrow \mathbb{R}$	loss
Π	finite information partition of Ω
$\text{Pred}(\Pi)$	predictors measurable with respect to Π
$R_L(f)$	risk of predictor f
$R_L(\Pi)$	Bayes risk achievable under partition Π
$I : \Omega \rightarrow \mathcal{Z}$	information map inducing a partition $\Pi(I)$
$U : \Omega \rightarrow \{0, 1\}$	binary distinction
$\Pi \vee U$	partition obtained by splitting Π by U
$\mathcal{F}, \mathcal{G}$	sigma-algebras or information structures
$V_L(\mathcal{F}, \mathcal{G})$	value of refining $\mathcal{F}$ to $\mathcal{G}$
d_L^e	loss-dependent distinction complexity
a_j, c_j, M_j	CA anchor, centroid, and local memory for concept j
$E_t, g_e, \mathcal{P}_t^G$	CA edge set, edge generator, and graph-generated partition
$\mathcal{B}_t^{\text{score}}$	held-out scoring buffer for empirical refinement tests
$\hat{\Delta}_{\text{repair}}, \hat{\Delta}_{\text{refine}}$	empirical repair and refinement gains

Table 4: Notation used throughout the paper. We say Π' is finer than Π if every atom of Π' is contained in an atom of Π .

and statistical decision theory [Blackwell, 1953, Berger, 1985]. The log-loss and squared-loss value identities are standard information-theoretic and Hilbert-space facts [Cover and Thomas, 2006, Gneiting and Raftery, 2007]. The contribution is to place these facts inside a runtime-learning model in which some errors are repair signals and others are evidence of missing distinctions.

D Mathematical Model: Partitions, Predictors, and Bayes Risk

We first formalize the information available to a runtime learner as a partition of the case space, using the standard link between information, measurability, and admissible decision rules [Blackwell, 1953, Berger, 1985, Billingsley, 1995, Kallenberg, 2002]. The measurable-space notation used later is the direct sigma-algebra generalization, but the computational object is a finite partition.

D.1 Partitions, predictors, and risk

Let Ω be a finite case space, let P be a distribution on Ω , let $Y : \Omega \rightarrow \mathcal{Y}$ be the consequence variable, let $\mathbf{A}$ be an action space, and let $L : \mathcal{Y} \times \mathbf{A} \rightarrow \mathbb{R}$ be a loss with finite expectations for the predictors under discussion.

Definition D.1 (Information partition and partition-measurable predictor). *A finite information partition is a collection of nonempty disjoint atoms*

$$\Pi = \{B_1, \dots, B_m\}$$

whose union is Ω . It induces the equivalence relation $\omega \sim_\Pi \omega'$ iff ω, ω' lie in the same atom $B \in \Pi$.

A predictor is Π -measurable if it cannot distinguish states inside the same atom:

$$\text{Pred}(\Pi) = \{f : \Omega \rightarrow \mathbf{A} : \omega \sim_\Pi \omega' \Rightarrow f(\omega) = f(\omega')\}.$$

Equivalently, a Π -measurable predictor is a map $a : \Pi \rightarrow \mathbf{A}$ evaluated on the atom containing ω .

Definition D.2 (Bayes risk under a partition). *For $f : \Omega \rightarrow \mathbf{A}$, write $R_L(f) = \mathbb{E}[L(Y, f)]$. The Bayes risk achievable under partition Π is*

$$R_L(\Pi) = \inf_{f \in \text{Pred}(\Pi)} \mathbb{E}[L(Y, f)] = \sum_{B \in \Pi} P(B) \inf_{a \in \mathbf{A}} \mathbb{E}[L(Y, a) \mid \omega \in B],$$

where atoms with $P(B) = 0$ may be assigned arbitrary actions. Thus $R_L(\Pi)$ is the best possible risk after the representation has committed to the equivalence relation $\sim_\Pi$, i.e., the Bayes risk under constrained information [Berger, 1985].

Definition D.3 (Information map and induced partition). *An information map is a runtime representation $I : \Omega \rightarrow \mathcal{Z}$. It induces the partition*

$$\Pi(I) = \{I^{-1}(z) : z \in \mathcal{Z}, I^{-1}(z) \neq \emptyset\}.$$

Any predictor that factors through I , written $f(\omega) = a(I(\omega))$, is measurable with respect to $\Pi(I)$.

E Online Prediction with Repair and Refinement

The online learner has two update types: repair preserves the current partition, while refinement joins it with a new binary distinction.

Definition E.1 (Binary distinction and partition join). *A binary distinction is a function $U : \Omega \rightarrow \{0, 1\}$. Joining Π with U means splitting every atom of Π according to the value of U :*

$$\Pi \vee U = \{B \cap U^{-1}(0), B \cap U^{-1}(1) : B \in \Pi\} \setminus \{\emptyset\}. \quad (2)$$

Definition E.2 (Refinable online prediction problem). *A refinable online prediction problem is a tuple*

$$\mathfrak{P} = (\Omega, \mathcal{Y}, \mathcal{A}, P, L, \mathcal{O}, \mathcal{U}, c),$$

where Ω is finite, $Y : \Omega \rightarrow \mathcal{Y}$ is the consequence variable, P is the sampling distribution, $L : \mathcal{Y} \times \mathcal{A} \rightarrow \mathbb{R}$ has finite risk for admissible predictors, $\mathcal{O} \subseteq 2^\Omega$ is the pre-outcome observable sigma-algebra, $\mathcal{U} \subseteq \{0, 1\}^\Omega$ is a class of admissible binary distinctions that are $\mathcal{O}$ -measurable, and $c : \mathcal{U} \rightarrow \mathbb{R}_+$ assigns refinement costs. Bounded losses are required only in finite-sample concentration statements, where they are stated explicitly. The $\mathcal{O}$ -measurability condition prevents a prediction-time distinction from depending directly on the consequence label; pure value-of-information examples can be recovered by taking $\mathcal{O} = 2^\Omega$.

The initial runtime partition is assumed to be prediction-time observable:

$$\sigma(\Pi_0) \subseteq \mathcal{O}.$$

Since every accepted distinction $U_t \in \mathcal{U}$ is $\mathcal{O}$ -measurable, induction gives $\sigma(\Pi_t) \subseteq \mathcal{O}$ for every reachable partition. Thus observing the current atom is itself a pre-outcome observation, not an additional label-dependent channel.

At round t , the learner maintains $S_t = (\Pi_t, a_t)$ with $a_t : \Pi_t \rightarrow \mathcal{A}$. A case $\omega_t \sim P$ is drawn, the learner observes only the atom $B_t \in \Pi_t$ containing ω_t , predicts $a_t(B_t)$, observes $Y_t = Y(\omega_t)$, and then chooses one of two updates:

$$\begin{aligned} \text{Repair : } & \Pi_{t+1} = \Pi_t, \quad a_{t+1} \text{ may change,} \\ \text{Refine : } & \Pi_{t+1} = \Pi_t \vee U_t \text{ for some } U_t \in \mathcal{U}. \end{aligned}$$

The cumulative objective is prediction loss plus refinement cost,

$$\mathcal{L}_T = \sum_{t=1}^T L(Y_t, a_t(B_t)) + \lambda \sum_{t:\text{Refine}} c(U_t).$$

Definition E.3 (Fixed-information repair update). *Let a parameterized runtime predictor factor through a fixed information map $I : \Omega \rightarrow \mathcal{Z}$:*

$$f_\psi(\omega) = a_\psi(I(\omega)).$$

An update of ψ is a fixed-information repair update if I is unchanged. The update rule may be a gradient step, a closed-form sufficient-statistic update, a cache write, or any other optimizer [Rumelhart et al., 1986, Zinkevich, 2003, Shalev-Shwartz, 2012, Grave et al., 2017, Khandelwal et al., 2020, Ba et al., 2016, Schlag et al., 2021]; what makes it repair is the invariance of the induced partition $\Pi(I)$, not the optimizer used.

F The Repair-Only Obstruction

If the partition is fixed, the learner cannot beat the Bayes risk of that partition, no matter how sophisticated the repair update is. This is the fixed-information specialization of the classical decision-theoretic fact that a decision rule cannot exploit information it is not allowed to condition on [Blackwell, 1953, Berger, 1985].

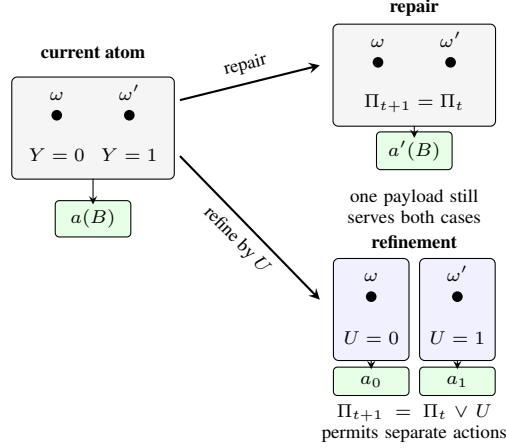


Figure 2: Repair versus refinement. Repair changes the payload assigned to an existing atom but preserves the equivalence relation. Refinement joins the partition with a distinction U , splitting the old atom only where the new bit matters and allowing different local actions.

Definition F.1 (Repair gap and refinement gap). For a concrete predictor $f \in \text{Pred}(\Pi)$ and a candidate refined partition Π^* that refines Π , define

$$\begin{aligned}\Delta_{\text{repair}}(f, \Pi) &= R_L(f) - R_L(\Pi), \\ \Delta_{\text{refine}}(\Pi, \Pi^*) &= R_L(\Pi) - R_L(\Pi^*).\end{aligned}$$

The first gap is the residual error of the current predictor at fixed equivalence relation. The second is the value of changing the equivalence relation.

Theorem F.2 (Runtime error decomposition, formal version of Theorem 2.2). Let $f_t \in \text{Pred}(\Pi_t)$ and let Π^* refine Π_t . Then

$$R_L(f_t) - R_L(\Pi^*) = \Delta_{\text{repair}}(f_t, \Pi_t) + \Delta_{\text{refine}}(\Pi_t, \Pi^*).$$

Both terms are nonnegative.

Proof. Add and subtract $R_L(\Pi_t)$. The repair gap is nonnegative because f_t is feasible for the infimum defining $R_L(\Pi_t)$. The refinement gap is nonnegative because every Π_t -measurable predictor is also measurable with respect to any refinement Π^* . $\square$

F.1 Repair-only lower bound

The pointwise contradiction inside a cell becomes a sequential lower bound. If the current partition collapses states that a refined oracle separates, no repair-only learner can avoid paying the corresponding Bayes-risk gap every round.

Theorem F.3 (Repair-only linear regret, formal version of Theorem 2.3). Let Π be a fixed partition and let Π^* be a refinement. Consider any learner whose round- t prediction $\hat{f}_t$ is always in $\text{Pred}(\Pi)$, possibly chosen adaptively from all previous observations. Then, for iid $\omega_t \sim P$,

$$\mathbb{E} \left[\sum_{t=1}^T L(Y_t, \hat{f}_t(\omega_t)) \right] \geq T R_L(\Pi).$$

Consequently, if Bayes actions exist under Π^* , then relative to an oracle using Π^* ,

$$\mathbb{E}[\text{Regret}_T] \geq T (R_L(\Pi) - R_L(\Pi^*)).$$

If the infimum under Π^* is not attained, the same statement holds in the epsilon-oracle sense: for every $\epsilon > 0$, regret against an ϵ -optimal Π^* -measurable oracle is at least $T(R_L(\Pi) - R_L(\Pi^*) - \epsilon)$. Thus if $R_L(\Pi) > R_L(\Pi^*)$, every repair-only learner suffers $\Omega(T)$ expected regret against refined or epsilon-refined oracles.

724 *Proof.* Condition on the learner's history before round t . The chosen predictor $\hat{f}_t$ is Π -measurable,
 725 so by definition of $R_L(\Pi)$,

$$\mathbb{E}[L(Y_t, \hat{f}_t(\omega_t)) \mid \text{history}] \geq R_L(\Pi).$$

726 Taking expectation and summing over t gives the first inequality. If Bayes actions exist on the atoms
 727 of Π^* , the Π^* oracle achieves $R_L(\Pi^*)$ per round by using them, so subtracting its cumulative risk
 728 gives the regret bound. Without attained Bayes actions, for every $\epsilon > 0$ choose an ϵ -optimal atomwise
 729 predictor under Π^* ; subtracting its cumulative risk gives the epsilon-oracle bound. $\square$

730 The displayed comparator gives the refined partition to the oracle for free. If regret includes a
 731 one-time refinement cost $C(\Pi^*)$ for reaching Π^* , the same argument gives

$$\mathbb{E}[\text{Regret}_T] \geq T(R_L(\Pi) - R_L(\Pi^*)) - \lambda C(\Pi^*).$$

732 **Corollary F.4** (Fixed-information gradient repair lower bound). *Consider any online optimizer,*
 733 *including backpropagation, whose predictions at every round factor through a fixed information map*
 734 *I :*

$$\hat{f}_t(\omega) = a_{\psi_t}(I(\omega)).$$

735 *Then $\hat{f}_t \in \text{Pred}(\Pi(I))$ for all t , so Theorem F.3 applies with $\Pi = \Pi(I)$. In particular, if a refined*
 736 *partition Π^* has $R_L(\Pi(I)) > R_L(\Pi^*)$, this fixed-information gradient repair suffers linear regret*
 737 *against the refined oracle. Backpropagation through parameters that change I is outside this*
 738 *corollary; in the present model, such an update changes the information structure rather than merely*
 739 *repairing actions on fixed atoms.*

740 **Example F.5** (Two-state separation). *Let $\Omega = \{0, 1\}$, $P(0) = P(1) = 1/2$, and $Y = \omega$. Under*
 741 *the trivial partition $\Pi = \{\{0, 1\}\}$, the best log-loss predictor is $q(Y = 0) = q(Y = 1) = 1/2$, so*
 742 *$R_{\log}(\Pi) = \log 2$. The refined partition $\Pi^* = \{\{0\}, \{1\}\}$ has $R_{\log}(\Pi^*) = 0$ in the noiseless case*
 743 *when point-mass predictions are allowed. Therefore any repair-only learner has expected regret*
 744 *$T \log 2$ against the refined oracle. This example is not about neural networks, caches, or trees. It*
 745 *is about equivalence relations: a learner that identifies two states cannot assign them two different*
 746 *consequences.*

747 **Corollary F.6** (Contradiction requires distinction, informal version of Theorem O.3). *Let Π be a*
 748 *partition and let $y : \Omega \rightarrow \mathcal{A}$ be a required deterministic output. If ω, ω' lie in the same atom of Π but*
 749 *$y(\omega) \neq y(\omega')$, then no predictor in $\text{Pred}(\Pi)$ satisfies both requirements.*

750 *Proof.* Every $f \in \text{Pred}(\Pi)$ is constant on the atom containing ω and ω' , so $f(\omega) = f(\omega')$. Satisfying
 751 both requirements would imply $y(\omega) = y(\omega')$, a contradiction. $\square$

752 F.2 Sigma-algebra extension

753 The finite partition model is the main computational model. The sigma-algebra notation is the
 754 measure-theoretic extension used for value identities and language-model interpretations [Billingsley,
 755 1995, Kallenberg, 2002]. In this formulation, L is measurable with finite expectation for the actions
 756 considered.

757 **Definition F.7** ($\mathcal{F}$ -measurable predictor and Bayes action). *On a probability space $(\Omega, \mathcal{E}, \mathbb{P})$, an*
 758 *information structure is a sub-sigma-algebra $\mathcal{F} \subseteq \mathcal{E}$. For a measurable action space $(\mathcal{A}, \Sigma_{\mathcal{A}})$, a*
 759 *predictor $a : \Omega \rightarrow \mathcal{A}$ is $\mathcal{F}$ -measurable if*

$$a^{-1}(C) \in \mathcal{F} \quad \text{for every } C \in \Sigma_{\mathcal{A}}.$$

760 *The admissible predictor class and Bayes risk are*

$$\text{Pred}(\mathcal{F}) = \{a : \Omega \rightarrow \mathcal{A} : a \text{ is } (\mathcal{F}, \Sigma_{\mathcal{A}})\text{-measurable}\}, \quad R_L(\mathcal{F}) = \inf_{a \in \text{Pred}(\mathcal{F})} \mathbb{E}[L(Y, a)].$$

761 *When the infimum is attained, any minimizer is an $\mathcal{F}$ -Bayes action and is denoted $a_{\mathcal{F}}^*$. A random*
 762 *variable b has an $\mathcal{F}$ -measurable version if there exists an $\mathcal{F}$ -measurable random variable $\tilde{b}$ such that*
 763 *$\tilde{b} = b$ almost surely.*

764 For a finite partition Π , Definition F.7 recovers Definition D.1 by taking $\mathcal{F} = \sigma(\Pi)$ and the action
 765 space discrete.

766 Implementations often have fixed pre-existing prediction outputs that are not created by the runtime
 767 refinement mechanism, such as a backbone distribution or other fixed pre-outcome features used
 768 outside CA routing. We write this base information as a sigma-algebra $\mathcal{B}$ and measure runtime
 769 refinements through the total information $\mathcal{B} \vee \mathcal{F}_t$. The full hidden state used to propose or instantiate
 770 distinctions should not automatically be included as freely conditionable base information: if $\mathcal{B}$
 771 contains h , then every edge predicate $g_e(h)$ and anchor-order event is already $\mathcal{B}$ -measurable, so the
 772 unrestricted sigma-algebra refinement gap becomes trivial. Thus the refinement gap relative to a
 773 target $\mathcal{F}^*$ is

$$R_L(\mathcal{B} \vee \mathcal{F}_t) - R_L(\mathcal{B} \vee \mathcal{F}^*).$$

774 Repairs keep $\mathcal{F}_t$ fixed, while refinements replace $\mathcal{F}_t$ by $\mathcal{F}_t \vee \sigma(U_t)$. All monotonicity and value
 775 identities below apply after joining both sides with the same fixed $\mathcal{B}$.

776 Section H uses this notation to characterize exactly when a richer information structure is useful.

777 G Minimal Refinement

778 Adding a distinction has a canonical minimal form: split exactly the atoms crossed by that distinction.
 779 Once a contradiction shows that the current atom is too coarse, the next question is not how to
 780 grow a large model. It is which information must be added. The primitive operation in this paper is
 781 minimal refinement: given an information structure and evidence that a new event or variable must be
 782 observable, add that observability and nothing else. We state the finite partition theorem first, then the
 783 sigma-algebra form. The latter is the standard measure-theoretic join; the former is its finite special
 784 case [Billingsley, 1995, Kallenberg, 2002].

785 G.1 The finite refinement rule

786 **Definition G.1** (Partition-measurable event and finite split). *Let $\mathcal{P}$ be a finite partition of Ω . An event*
 787 *$D \subseteq \Omega$ is $\mathcal{P}$ -measurable if it is a union of atoms of $\mathcal{P}$. For any event D , the finite split of $\mathcal{P}$ by D is*

$$\begin{aligned} \mathcal{P}_D = \{ & B \cap D : B \in \mathcal{P}, B \cap D \neq \emptyset \} \\ & \cup \{ B \cap D^c : B \in \mathcal{P}, B \cap D^c \neq \emptyset \}. \end{aligned}$$

788 *This keeps exactly the nonempty pieces obtained by cutting each old atom by D .*

789 **Theorem G.2** (Minimal finite refinement, informal version of Theorem O.1). *Let $\mathcal{P}$ be a finite*
 790 *partition of Ω and let $D \subseteq \Omega$. The partition $\mathcal{P}_D$ is the unique minimal partition that refines $\mathcal{P}$ and*
 791 *makes D measurable. More precisely, if $\mathcal{Q}$ refines $\mathcal{P}$ and D is $\mathcal{Q}$ -measurable, then $\mathcal{Q}$ refines $\mathcal{P}_D$.*

792 *Proof.* Each old atom $B \in \mathcal{P}$ is replaced by the nonempty sets among $B \cap D$ and $B \cap D^c$, so the
 793 resulting sets are disjoint, cover Ω , and each lies inside an atom of $\mathcal{P}$. Hence $\mathcal{P}_D$ is a refinement of
 794 $\mathcal{P}$. Moreover,

$$D = \bigcup_{B \in \mathcal{P}} (B \cap D),$$

795 so D is a union of $\mathcal{P}_D$ -atoms. Now let $\mathcal{Q}$ refine $\mathcal{P}$ and make D measurable. For every $Q \in \mathcal{Q}$, there
 796 is $B \in \mathcal{P}$ with $Q \subseteq B$, and measurability of D with respect to $\mathcal{Q}$ implies either $Q \subseteq D$ or $Q \subseteq D^c$.
 797 Thus Q is contained in either $B \cap D$ or $B \cap D^c$, i.e., in an atom of $\mathcal{P}_D$. Therefore $\mathcal{Q}$ refines $\mathcal{P}_D$. $\square$

798 The theorem gives a canonical update. The evidence determines the least refined representation:
 799 atoms untouched by D remain unchanged, while atoms crossed by D split into the two pieces that
 800 the evidence distinguishes.

801 G.2 The sigma-algebra form

802 For general spaces, the same operation is the join of sigma-algebras. If the current information is $\mathcal{F}_t$
 803 and the evidence is a variable U_t , the minimal updated information is

$$\mathcal{F}_{t+1} = \mathcal{F}_t \vee \sigma(U_t).$$

Definition G.3 (Generated sigma-algebra and join). For a random variable U , $\sigma(U)$ denotes the smallest sigma-algebra with respect to which U is measurable. For two sigma-algebras $\mathcal{F}$ and $\mathcal{H}$, their join is

$$\mathcal{F} \vee \mathcal{H} = \sigma(\mathcal{F} \cup \mathcal{H}),$$

the smallest sigma-algebra containing both. In particular, $\mathcal{F} \vee \sigma(U)$ is the information structure obtained by adding observability of U to $\mathcal{F}$.

Theorem G.4 (Minimal sigma-algebra refinement, informal version of Theorem O.2). Let $\mathcal{F} \subseteq \mathcal{E}$ be a sigma-algebra and let U be a random variable. The join $\mathcal{F} \vee \sigma(U)$ is the smallest sigma-algebra $\mathcal{G}$ such that $\mathcal{F} \subseteq \mathcal{G}$ and U is $\mathcal{G}$ -measurable.

Proof. By definition, $\mathcal{F} \vee \sigma(U) = \sigma(\mathcal{F} \cup \sigma(U))$, so it contains $\mathcal{F}$ and makes U measurable. If another sigma-algebra $\mathcal{G}$ contains $\mathcal{F}$ and makes U measurable, then $\mathcal{F} \cup \sigma(U) \subseteq \mathcal{G}$. Since $\mathcal{G}$ is a sigma-algebra, $\sigma(\mathcal{F} \cup \sigma(U)) \subseteq \mathcal{G}$. $\square$

Thus the same object governs finite decision-tree splits and measurable-space refinement. A distinction is not an auxiliary feature added after learning; it is a change in the sigma-algebra of what the learner may condition on. The obstruction in Section F says when such a change is necessary; minimal refinement says how little information must be added.

H The Value of a Distinction

Minimal refinement says how to add a distinction; it does not license every possible distinction. The value question is task-dependent: when does the new distinction actually change the best achievable prediction? We measure this by the decrease in optimal risk after the information structure is refined, in the same decision-theoretic sense used in classical comparisons of statistical experiments and value of information [Blackwell, 1953, Berger, 1985].

The identities in this section are population Bayes-risk statements and require the relevant expectations, entropies, or second moments to be finite. Bounded-loss assumptions enter only in the empirical concentration and validation results.

Definition H.1 (Distinction value). For information structures $\mathcal{F} \subseteq \mathcal{G}$, the value of refining $\mathcal{F}$ to $\mathcal{G}$ under loss L is

$$V_L(\mathcal{F}, \mathcal{G}) = R_L(\mathcal{F}) - R_L(\mathcal{G}).$$

For a binary distinction U , its conditional value at $\mathcal{F}$ is

$$V_L(U \mid \mathcal{F}) = R_L(\mathcal{F}) - R_L(\mathcal{F} \vee \sigma(U)).$$

In the finite partition model, the analogous value is $V_L(U \mid \Pi) = R_L(\Pi) - R_L(\Pi \vee U)$.

H.1 Risk monotonicity

The first fact is purely order-theoretic. If $\mathcal{G}$ contains all events in $\mathcal{F}$, then every $\mathcal{F}$ -measurable predictor is also $\mathcal{G}$ -measurable. The decision class has only grown; this is the Bayes-risk analogue of Blackwell's monotonicity of decision value under more informative experiments [Blackwell, 1953].

Theorem H.2 (Bayes-risk monotonicity, informal version of Theorem P.2). If $\mathcal{F} \subseteq \mathcal{G}$, then for every loss L for which both risks are well-defined,

$$R_L(\mathcal{G}) \leq R_L(\mathcal{F}).$$

Proof. Every $\mathcal{F}$ -measurable action is also $\mathcal{G}$ -measurable. Taking the infimum of the same expected loss over the larger feasible set gives the claim. $\square$

This is a population statement. It says that information cannot hurt the best possible decision rule. It does not say that a finite implementation should refine without restraint. Statistical estimation, finite samples, and model selection can overfit; those are algorithmic questions, not failures of the information order. This is why the CA algorithm later uses penalties, thresholds, and sample-split scoring before committing a new concept.

845 H.2 Loss-relevant refinement

846 A distinction is useful only if it changes the Bayes-optimal decision. The next theorem is the decision-
847 theoretic core of the paper: zero refinement value means the refined optimum was already expressible
848 under the old information.

849 **Theorem H.3** (Loss-relevant refinement, informal version of Theorem P.1). *Let $\mathcal{F} \subseteq \mathcal{G} \subseteq \mathcal{E}$. Assume*
850 *Bayes actions exist under $\mathcal{F}$ and $\mathcal{G}$, and that the $\mathcal{G}$ -Bayes action is almost surely unique. Then*

$$R_L(\mathcal{F}) - R_L(\mathcal{G}) = 0$$

851 *if and only if $a_{\mathcal{G}}^*$ has an $\mathcal{F}$ -measurable version.*

852 *Proof.* If $a_{\mathcal{G}}^*$ is $\mathcal{F}$ -measurable, then it is feasible for $R_L(\mathcal{F})$, so

$$R_L(\mathcal{F}) \leq \mathbb{E}[L(Y, a_{\mathcal{G}}^*)] = R_L(\mathcal{G}).$$

853 Monotonicity gives the reverse inequality. Conversely, if the risks are equal and $a_{\mathcal{F}}^*$ is an $\mathcal{F}$ -Bayes
854 action, then $a_{\mathcal{F}}^*$ is also feasible under $\mathcal{G}$ and achieves $R_L(\mathcal{G})$. Almost sure uniqueness under $\mathcal{G}$ forces
855 $a_{\mathcal{F}}^* = a_{\mathcal{G}}^*$ almost surely, so $a_{\mathcal{G}}^*$ has an $\mathcal{F}$ -measurable version. $\square$

856 H.3 Log loss

857 For a finite target variable Y , log loss asks the learner to output a conditional distribution. The optimal
858 prediction under $\mathcal{F}$ is the conditional law of Y given $\mathcal{F}$, and the optimal risk is conditional entropy,
859 a standard identity for logarithmic scoring rules and information theory [Cover and Thomas, 2006,
860 Gneiting and Raftery, 2007].

861 **Theorem H.4** (Log-loss value, informal version of Theorem P.3). *Let Y be finite and let the loss be*
862 *$-\log q(Y)$. If $\mathcal{F} \subseteq \mathcal{G}$, then*

$$R_{\log}(\mathcal{F}) - R_{\log}(\mathcal{G}) = I(Y; \mathcal{G} | \mathcal{F}).$$

863 Here $I(Y; \mathcal{G} | \mathcal{F})$ denotes $H(Y | \mathcal{F}) - H(Y | \mathcal{G})$. Writing

$$p_{\mathcal{F}}(y) = \mathbb{P}(Y = y | \mathcal{F}), \quad p_{\mathcal{G}}(y) = \mathbb{P}(Y = y | \mathcal{G}),$$

864 the same identity has the conditional-KL form

$$R_{\log}(\mathcal{F}) - R_{\log}(\mathcal{G}) = \mathbb{E}[D_{\text{KL}}(p_{\mathcal{G}}(\cdot) \| p_{\mathcal{F}}(\cdot))].$$

865 Thus, for classification and language modeling, a refinement is useful exactly when it changes the
866 conditional consequence distribution with positive probability. If the new bit does not tell us anything
867 about the next consequence beyond the old information, it is a gratuitous distinction, even if it
868 separates inputs.

869 **Corollary H.5** (Binary distinction value, informal version of Corollary P.4). *Let $U \in \{0, 1\}$ be a*
870 *candidate distinction and let $\mathcal{G} = \mathcal{F} \vee \sigma(U)$. Under log loss,*

$$\begin{aligned} R_{\log}(\mathcal{F}) - R_{\log}(\mathcal{G}) &= I(Y; U | \mathcal{F}) \\ &= \mathbb{E} \left[\sum_{u \in \{0, 1\}} \mathbb{P}(U = u | \mathcal{F}) D_{\text{KL}}(\mathbb{P}(Y \in \cdot | \mathcal{F}, U = u) \| \mathbb{P}(Y \in \cdot | \mathcal{F})) \right]. \end{aligned}$$

871 H.4 Squared loss

872 For $Y \in L^2(\Omega; \mathbb{R}^d)$ and squared loss, the optimal predictor under $\mathcal{F}$ is $\mathbb{E}[Y | \mathcal{F}]$, the L^2 projection
873 onto the closed subspace of $\mathcal{F}$ -measurable random variables [Billingsley, 1995, Kallenberg, 2002].
874 Refinement helps precisely when it changes the conditional mean.

875 **Theorem H.6** (Squared-loss value, informal version of Theorem P.5). *Let $Y \in L^2(\Omega; \mathbb{R}^d)$ and let*
876 *$L(Y, a) = \|Y - a\|_2^2$. If $\mathcal{F} \subseteq \mathcal{G}$, then*

$$R_2(\mathcal{F}) - R_2(\mathcal{G}) = \mathbb{E} \left[\|\mathbb{E}[Y | \mathcal{G}] - \mathbb{E}[Y | \mathcal{F}]\|_2^2 \right].$$

877 The identity is the Hilbert-space analogue of the log-loss result. A distinction has squared-loss value
878 when the cells it creates have different conditional means. In regression language, refinement is worth
879 paying for when it separates consequences, not merely when it separates inputs.

880 H.5 Minimal loss-sufficient refinement

881 Minimal refinement in Section G adds the least information needed to observe a proposed variable.
 882 The loss-sufficient question is slightly different: among the refinements inside $\mathcal{G}$, what is the least
 883 information needed to attain the $\mathcal{G}$ -Bayes risk?

884 **Theorem H.7** (Minimal loss-sufficient refinement, informal version of Theorem P.6). *Work in the*
 885 *measurable-action convention of Section D. Let $\mathcal{F} \subseteq \mathcal{G}$ and assume the $\mathcal{G}$ -Bayes action $a_{\mathcal{G}}^* : \Omega \rightarrow \mathcal{A}$*
 886 *exists as a $(\mathcal{G}, \Sigma_{\mathcal{A}})$ -measurable map and is almost surely unique. Also assume the Bayes infimum is*
 887 *attained for every intermediate sigma-algebra $\mathcal{H}$ with $\mathcal{F} \subseteq \mathcal{H} \subseteq \mathcal{G}$ that is considered below. Up to*
 888 *null sets, the smallest sigma-algebra $\mathcal{H}$ satisfying*

$$\mathcal{F} \subseteq \mathcal{H} \subseteq \mathcal{G}, \quad R_L(\mathcal{H}) = R_L(\mathcal{G})$$

889 *is*

$$\mathcal{F} \vee \sigma(a_{\mathcal{G}}^*).$$

890 *For finite Y under log loss, the corresponding smallest sufficient refinement is*

$$\mathcal{F} \vee \sigma(p_{\mathcal{G}}), \quad p_{\mathcal{G}}(\cdot) = \mathbb{P}(Y \in \cdot \mid \mathcal{G}).$$

891 *Proof.* The action $a_{\mathcal{G}}^*$ is measurable with respect to $\mathcal{F} \vee \sigma(a_{\mathcal{G}}^*)$, so this refinement attains $R_L(\mathcal{G})$. If
 892 any $\mathcal{H}$ with $\mathcal{F} \subseteq \mathcal{H} \subseteq \mathcal{G}$ also attains $R_L(\mathcal{G})$, Theorem H.3 applied to $\mathcal{H} \subseteq \mathcal{G}$ implies that $a_{\mathcal{G}}^*$ has an
 893 $\mathcal{H}$ -measurable version. Hence $\mathcal{F} \vee \sigma(a_{\mathcal{G}}^*) \subseteq \mathcal{H}$ up to null sets. Under log loss, the Bayes action is
 894 the conditional law $p_{\mathcal{G}}$, giving the final statement. $\square$

895 I Empirical Split Scores and Finite-Sample Refinement

896 Population refinement value becomes empirical information gain or variance reduction, but finite
 897 samples require penalties, margins, and held-out validation.

898 I.1 Finite empirical scores

899 **Definition I.1** (Empirical distinction scores). *Suppose an atom B contains samples (x_j, y_j) and a*
 900 *candidate distinction $g : B \rightarrow \{0, 1\}$ splits it into B_0 and B_1 . The formulas below are evaluated*
 901 *for nontrivial empirical splits with $|B_0|, |B_1| > 0$. A candidate with an empty empirical child is*
 902 *assigned score zero and is not accepted by a positive-gain split rule.*

903 *For classification, let $\hat{p}_B$, $\hat{p}_{B_0}$, and $\hat{p}_{B_1}$ be empirical label distributions. The empirical log-loss*
 904 *improvement is*

$$\Delta_{\log}(B, g) = |B| H(\hat{p}_B) - |B_0| H(\hat{p}_{B_0}) - |B_1| H(\hat{p}_{B_1}).$$

905 *Equivalently,*

$$\Delta_{\log}(B, g) = |B| \hat{I}(Y; g \mid B).$$

906 *The normalized empirical edge score is therefore*

$$\hat{I}(Y; g \mid B) = H(\hat{p}_B) - \sum_{b \in \{0, 1\}} \frac{|B_b|}{|B|} H(\hat{p}_{B_b}).$$

907 *For regression, define*

$$\hat{\mu}_B = \frac{1}{|B|} \sum_{j \in B} y_j, \quad \text{SSE}(B) = \sum_{j \in B} \|y_j - \hat{\mu}_B\|_2^2.$$

908 *The empirical squared-loss improvement is*

$$\Delta_2(B, g) = \text{SSE}(B) - \text{SSE}(B_0) - \text{SSE}(B_1).$$

909 *It also has the between-cell form*

$$\Delta_2(B, g) = |B_0| \|\hat{\mu}_{B_0} - \hat{\mu}_B\|_2^2 + |B_1| \|\hat{\mu}_{B_1} - \hat{\mu}_B\|_2^2.$$

910 The population identities lead directly to these standard empirical split scores: information gain
 911 for classification and variance reduction for regression trees [Quinlan, 1986, Breiman et al., 1984,
 912 Domingos and Hulten, 2000]. They implement the same principle as Definition H.1: split only when
 913 the proposed distinction has measurable consequence value. The CA algorithm below turns this
 914 population rule into a guarded empirical decision.

915 I.2 Distinction complexity

916 **Definition I.2** (Distinction complexity). *Let Π be an initial partition, let Π^* be a target refinement,*
 917 *and let $\mathcal{U} \subseteq \{0, 1\}^\Omega$ be a distinction class. For $\epsilon \geq 0$, the loss-dependent distinction complexity is*

$$d_L^\epsilon(\Pi, \Pi^*; \mathcal{U}) = \min \{m : \exists U_1, \dots, U_m \in \mathcal{U} \text{ such that } R_L(\Pi \vee U_1 \vee \dots \vee U_m) \leq R_L(\Pi^*) + \epsilon\},$$

918 *with value $+\infty$ if no such sequence exists.*

919 The refinement gap measures the value of a target information structure. Distinction complexity
 920 measures how many admissible bits are needed to reach it. Repair-only learning is the case $m = 0$.
 921 Decision trees, graph refinements, and prototype split systems differ partly in the distinction classes
 922 and ordering constraints they impose. Graph-growing CA corresponds to a graph-generated class of
 923 local binary predicates proposed from confused runtime states. The fixed-bank CA layer used in the
 924 main experiments can be viewed as preallocating a finite concept basis and learning how to repair its
 925 value payloads, rather than accepting new bits online during the run.

926 I.3 Greedy recovery under realizability

927 The next theorem is a clean realizable upper bound for greedy refinement. It is intentionally stated
 928 for the abstract partition model rather than for Soft-CA retrieval. The distinction class is understood
 929 to contain only prediction-time observable distinctions, as in Definition E.2.

930 **Definition I.3** (Target-respecting distinction). *Let Π^* be a target partition. A distinction $U : \Omega \rightarrow$*
 931 *$\{0, 1\}$ is Π^* -respecting if it does not split any atom of Π^* : for every $C \in \Pi^*$, either $C \subseteq U^{-1}(0)$ or*
 932 *$C \subseteq U^{-1}(1)$.*

933 **Definition I.4** (Impure atom and local split value). *For an atom $B \in \Pi$, write $\Pi^*|_B = \{C \in \Pi^* :$*
 934 *$C \cap B \neq \emptyset\}$. The atom B is impure relative to Π^* if $|\Pi^*|_B| > 1$. For a candidate distinction U , its*
 935 *local value on B is*

$$V_L(B, U) = R_L(\{B\}) - R_L(\{B\} \vee U),$$

936 *where the risks are conditional on $\omega \in B$. Under log loss, $V_{\log}(B, U) = I(Y; U \mid \omega \in B)$.*

937 **Theorem I.5** (Greedy recovery under target-respecting splits). *Let*

$$K^+ = |\{C \in \Pi^* : P(C) > 0\}|, \quad m_0^+ = |\{B \in \Pi_0 : P(B) > 0\}|.$$

938 *Suppose the current partition Π_0 is initially a coarsening of Π^* . Assume that every positive-mass*
 939 *impure current atom B admits a Π^* -respecting distinction $U \in \mathcal{U}$ such that U splits $\Pi^*|_B$ into two*
 940 *groups whose unions both have positive conditional probability under $P(\cdot \mid B)$ and*

$$V_L(B, U) \geq \gamma.$$

941 *Consider a greedy refinement algorithm that repeatedly selects a positive-mass impure current atom*
 942 *whenever one exists. Assume its candidate set contains at least one such distinction for every selected*
 943 *impure atom, its accepted candidates are Π^* -respecting, and its empirical local values satisfy*

$$\left| \widehat{V}_L(B, U) - V_L(B, U) \right| \leq \gamma/4$$

944 *for all candidates it scores. If the algorithm accepts a split whenever $\widehat{V}_L(B, U) > \gamma/2$, then it*
 945 *accepts a useful split for every selected impure atom and recovers Π^* on the positive-mass support*
 946 *after at most $K^+ - m_0^+ \leq K^+ - 1$ accepted splits.*

947 *Proof.* Because every accepted split is Π^* -respecting, the current partition remains a coarsening of
 948 Π^* after each accepted refinement. A candidate with true value at least γ has empirical value at least
 949 $3\gamma/4$, so it exceeds the $\gamma/2$ threshold and is accepted when proposed. Conversely, any accepted
 950 candidate has true value greater than $\gamma/4$ by the uniform estimation event. A Π^* -respecting split
 951 with positive value cannot be trivial on the selected atom: if one side had zero conditional mass, the
 952 conditional Bayes risk would be unchanged. Thus any accepted candidate separates the selected
 953 impure atom into two positive-mass unions of target atoms. It replaces one positive-mass current atom
 954 by two positive-mass unions of target atoms and increases the number of represented positive-mass
 955 current atoms by one. The algorithm keeps selecting impure positive-mass atoms until none remain.
 956 Starting from m_0^+ positive-mass current atoms and ending with at most K^+ positive-mass target
 957 atoms, at most $K^+ - m_0^+ \leq K^+ - 1$ accepted splits are needed. $\square$

For finite $\mathcal{U}$, finite action class A , and bounded loss, the event in Theorem I.5 follows from a standard held-out union bound over scored split-action pairs [Hoeffding, 1963, Devroye et al., 1996]. It suffices to score each accepted split with

$$m = O\left(\frac{1}{\gamma^2} \log \frac{K^+ |\mathcal{U}| |A|^3}{\delta}\right)$$

independent validation samples, up to constants and the bounded-loss range. For finite-label log-loss split scores or regression scores with infinite action spaces, the same conclusion requires the corresponding entropy, variance, or empirical-risk concentration assumption; bounded loss and finite $\mathcal{U}$ alone do not control the Bayes-action estimation error.

I.4 A finite-class oracle inequality

The same finite model gives a structural-risk bound for searching over refinements. Let $\mathfrak{P}_B(\mathcal{U})$ be the set of partitions obtainable from an initial partition Π_0 by at most B global distinctions from finite $\mathcal{U}$, where each distinction is applied by the join operation in (2). If Π_0 has m_0 atoms, every $\Pi \in \mathfrak{P}_B(\mathcal{U})$ has at most $m_0 2^B$ atoms, because one binary distinction can cut every current atom. The sharper $m_0 + B$ count applies only to a different local-split class in which each step is restricted to split a single current atom.

Theorem I.6 (Finite-class refinement oracle inequality). *Assume $L \in [0, 1]$ and A is finite. Let $\hat{R}_L(\Pi)$ be the empirical risk of the best atomwise action for Π on n iid validation samples, and let*

$$\hat{\Pi} \in \operatorname{argmin}_{\Pi \in \mathfrak{P}_B(\mathcal{U})} \left\{ \hat{R}_L(\Pi) + \lambda |\Pi| \right\}.$$

Then with probability at least $1 - \delta$,

$$\begin{aligned} R_L(\hat{\Pi}) + \lambda |\hat{\Pi}| &\leq \min_{\Pi \in \mathfrak{P}_B(\mathcal{U})} \{R_L(\Pi) + \lambda |\Pi|\} \\ &\quad + 2 \left(\frac{B \log |\mathcal{U}| + m_0 2^B \log |A| + \log(B+1) + \log(2/\delta)}{2n} \right)^{1/2}. \end{aligned}$$

Proof. For any fixed sequence of at most B distinctions and any assignment of actions to the resulting at most $m_0 2^B$ atoms, Hoeffding's inequality controls the empirical-population risk deviation. The number of sequence lengths is at most $B+1$, the number of distinction sequences is at most $|\mathcal{U}|^B$, and the number of action assignments for any resulting partition is at most $|A|^{m_0 2^B}$. Thus the number of partition-action pairs is at most $(B+1) |\mathcal{U}|^B |A|^{m_0 2^B}$. A union bound gives a uniform deviation ϵ equal to the square-root term without the leading factor 2. On this event, empirical penalized risk and population penalized risk differ by at most ϵ for every candidate. Applying the usual empirical-minimizer argument gives the displayed 2ϵ bound. $\square$

Theorem I.7 (Local-split refinement oracle inequality). *Let $\mathfrak{P}_B^{\text{loc}}(\mathcal{U})$ be the set of partitions reachable from an initial partition Π_0 with m_0 atoms by at most B local splits, where each split chooses one current atom and one distinction $U \in \mathcal{U}$ and applies the split only inside that atom. Assume $L \in [0, 1]$ and A is finite. Let $\hat{\Pi}$ minimize empirical penalized risk $\hat{R}_L(\Pi) + \lambda |\Pi|$ over $\mathfrak{P}_B^{\text{loc}}(\mathcal{U})$ on n iid validation samples. Then, with probability at least $1 - \delta$,*

$$\begin{aligned} R_L(\hat{\Pi}) + \lambda |\hat{\Pi}| &\leq \min_{\Pi \in \mathfrak{P}_B^{\text{loc}}(\mathcal{U})} \{R_L(\Pi) + \lambda |\Pi|\} \\ &\quad + 2 \sqrt{\frac{B \log |\mathcal{U}| + B \log(m_0 + B) + (m_0 + B) \log |A| + \log(2(B+1)/\delta)}{2n}}. \end{aligned}$$

Proof. Every local-split partition has at most $m_0 + B$ atoms. For a sequence length at most B , there are at most $|\mathcal{U}|^B$ choices of distinctions and at most $(m_0 + B)^B$ choices of current atoms to split, since the number of atoms never exceeds $m_0 + B$. For any resulting partition, the number of atomwise action assignments is at most $|A|^{m_0 + B}$. Hoeffding's inequality and a union bound over sequence lengths, local split sequences, and action assignments give a uniform deviation with the square-root term without the leading factor 2. The empirical-minimizer argument then adds the second copy of the deviation. $\square$

J Conceptual Attention as a Data Structure for Refinable Partitions

This appendix describes the graph-growing CA extension that realizes literal partition refinement. The main experiments use the simpler fixed-bank specialization from Section 5: concept identities are preallocated, all slots are active, edge gates are disabled, and the online operation is retained value-memory repair inside a bounded concept bank. The distinction matters for claims. The fixed-bank layer tests whether preallocated concept capacity and forward-time repair can replace token-indexed memory; the graph-growing extension below is the implementation route for accepting new distinctions online.

In the graph-growing extension, CA implements refinable information structures as a graph: edges generate distinctions and nodes store repairable memories. The ingredients are related to decision-tree refinement, adaptive category creation, prototype memories, and external neural memories, but CA uses them to implement the repair/refinement distinction [Breiman et al., 1984, Domingos and Hulten, 2000, Carpenter and Grossberg, 1987b, Grossberg, 2013, Kohonen, 1990, Weston et al., 2015, Graves et al., 2016]. The theory so far gives a diagnostic: decide whether an observed error is better explained by Δ_{repair} at fixed information or by Δ_{refine} from missing distinctions. A runtime implementation therefore needs a state representation that can both repair local statistics and add distinctions. Graph-growing CA is the data-structure realization of the abstract refinable-partition model.

J.1 Fixed-bank specialization and chunk kernels

The fixed-bank layer used for MQAR and CoM can be read as a finite concept-memory state machine. For a token t , heads H , concepts C , address width d_a , and value width d_v^c , the dense read computes

$$S_{thj} = \gamma q_{th}^\top a_{hj}, \quad \alpha_{thj} = \text{softmax}_j S_{thj}, \quad z_{th} = \sum_{j=1}^C \alpha_{thj} M_{t,hj}.$$

The dense write computes delayed write scores

$$R_{thj} = \gamma \bar{k}_{th}^\top a_{hj}, \quad \beta_{thj} = \text{softmax}_j R_{thj},$$

and updates

$$M_{t+1,hj} = \rho_{th} M_{t,hj} + m_t \beta_{thj} v_{th},$$

where m_t masks invalid delayed writes. The online state is only M_t , with size CHd_v^c ; the concept identities a_{hj} are trainable parameters or fixed buffers of the layer, not sequence state.

For training, the implementation groups tokens into chunks and forms the exact affine summary

$$R_{b,h} = \prod_{t=s}^{e-1} \rho_{th}, \quad U_{b,hj} = \sum_{t=s}^{e-1} \left(\prod_{u=t+1}^{e-1} \rho_{uh} \right) m_t \beta_{thj} v_{th}.$$

An exclusive scan over the associative composition

$$(R_2, U_2) \odot (R_1, U_1) = (R_2 R_1, U_2 + R_2 U_1)$$

recovers the entry memory for every chunk. The chunk read then adds all previous writes inside the chunk, so the result is exactly the token recurrence above. This is the implementation described in Section 5.

The arithmetic cost per chunk is

$$O(BHCd_a + BHCd_v^c)$$

for the dense read and the same order for the dense write summary, plus projection costs shared with the surrounding decoder block. The Triton kernels tile the concept, address, value, and chunk dimensions and apply the same equations under the implementation bounds used by the CUDA path, namely chunk length at most 128, concept count at most 256, address width at most 128, and value width at most 256. These bounds are engineering constraints of the fused dense kernels, not assumptions in the partition theory. Proposition 5.1 gives the mathematical interface: CA has a constant-size recurrent state and linear autoregressive decoding cost.

1033 The Concept Memory variant has the same scan shape with a different payload. Its state is

$$(n_t, B_t) \in \mathbb{R}^C \times \mathbb{R}^{C \times d_u},$$

1034 so the online memory is $C(d_u + 1)$. A dense block computes query/write responsibilities in $O(BCd_u)$,
 1035 updates count and moment summaries in $O(BCd_u)$, reads normalized means in $O(BCd_u)$, and
 1036 decodes the feature by a map in $O(Bd_u d_{\text{model}})$ when $d_u < d_{\text{model}}$. Thus d_u is the primary systems
 1037 and statistical-capacity knob. Setting $d_u \ll d_{\text{model}}$ gives a compressed sufficient statistic in the
 1038 chosen consequence feature space, not an approximation to the causal scan; increasing d_u increases
 1039 the amount of consequence information the state can repair at fixed concept count. The direct
 1040 $d_u = d_{\text{model}}$ endpoint is not automatically best in the present architecture, because it removes the
 1041 learned decoder used by the sketched variants and injects raw embedding moments directly into the
 1042 residual stream.

1043 With R retained time scales, the state becomes

$$(n_t^{(r)}, B_t^{(r)})_{r=1}^R,$$

1044 and memory/cost scale by R in the count and moment terms. This is a straightforward dense
 1045 extension of the same causal scan. It is not sparse retrieval and it does not change the read-before-
 1046 write semantics; it only allocates several decay horizons to the same consequence statistic.

1047 J.2 Why a graph, not a tree?

1048 **Definition J.1** (CA concept). *A CA concept is a persistent address for a prediction-relevant informa-*
 1049 *tion cell, equipped with a local repairable memory. In the graph-growing formalization it is selected*
 1050 *by a graph-generated hard-route map. It is not claimed to be a human-interpretable semantic concept*
 1051 *and need not correspond one-to-one with an atom of the edge partition. Local memories live at graph*
 1052 *nodes, and distinction predicates live on edges between nodes that have been confused.*

1053 The central object is

$$\text{CA} = \text{graph-generated information structure} + \text{local memory repair}.$$

1054 The graph is not metadata: its edges generate a finite edge-skeleton information structure and gate
 1055 the read path, while anchor order completes the exact hard route. The partition and generated-
 1056 sigma-algebra language follows standard finite measurable-space constructions [Billingsley, 1995,
 1057 Kallenberg, 2002]. The main-text statements below give the operational theorem forms; full proofs
 1058 are in Appendix Q.

1059 **Definition J.2** (Hard-CA and Soft-CA). *Hard-CA is the formal algorithm: fixed anchors and accepted*
 1060 *binary edges generate an edge skeleton, while the exact hard route is measured by the join of this edge*
 1061 *skeleton with the anchor-distance ordering. A repair changes only local actions or memories and*
 1062 *leaves the hard-route information fixed; a refinement adds an anchored concept, new edge generators,*
 1063 *and the anchor-order events involving the new anchor. Soft-CA is the practical relaxation used for*
 1064 *sparse neural hidden states: movable centroids retrieve candidates, smooth edge gates approximate*
 1065 *hard eligibility, and routing weights mix local memories.*

1066 The partition-level guarantees are exact for Hard-CA. Soft-CA is covered only as an approximation
 1067 under the candidate-recall and margin conditions in Theorem J.9.

1068 J.3 Hard-CA state and invariants

1069 **Definition J.3** (Hard-CA state and anchor edge). *Let $\phi_\theta : \mathcal{X} \rightarrow \mathbb{R}^d$ be an encoder, write $h = \phi_\theta(x)$,*
 1070 *and let $H \subseteq \mathbb{R}^d$ denote the hidden-state space on which the runtime structure is evaluated. A*
 1071 *Conceptual Attention state is*

$$\mathcal{S}_t = \left(\{a_j, c_j, M_j, n_j\}_{j=1}^{N_t}, G_t \right), \quad G_t = (V_t, E_t), \quad V_t = \{1, \dots, N_t\}.$$

1072 *Each component has a role forced by the refine-or-repair split. A runtime concept needs a stable*
 1073 *identity, so $a_j \in \mathbb{R}^d$ is a frozen distinction anchor. Retrieval may adapt as examples arrive, so*
 1074 *$c_j \in \mathbb{R}^d$ is a movable centroid used only by the soft implementation. State inside an adequate*
 1075 *concept must be repairable, so M_j is a local memory and n_j is a usage count. If a concept is created*

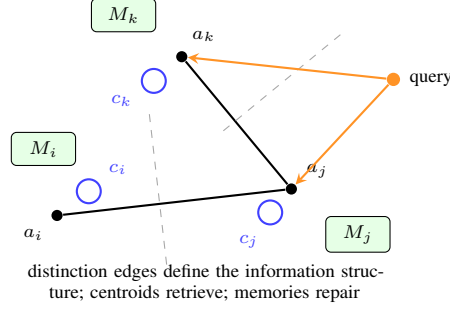


Figure 3: CA state. Anchors are stable concept identities, centroids are movable retrieval statistics, distinction edges generate the information structure, and concept memories are repaired after a concept is judged adequate.

1076 *because it was confused with existing concepts, that conflict becomes an edge in E_t . A new concept is*
 1077 *initialized with $a_j = c_j = h$. Subsequent repair may move c_j , but anchors remain fixed once created,*
 1078 *so the hard information structure is unchanged by centroid repair.*

1079 *An edge $e = (i, j)$ stores a linear distinction*

$$u_e = \frac{a_i - a_j}{\|a_i - a_j\|_2}, \quad b_e = \frac{1}{2}u_e^\top(a_i + a_j),$$

1080 *with side labels $s_{e,i} = +1$ and $s_{e,j} = -1$. Its binary generator is*

$$g_e(h) = \mathbf{1}\{u_e^\top h > b_e\}.$$

1081 **Definition J.4** (Graph signature and edge-generated partition). *The edge set induces the edge skeleton*
 1082 *information structure on $\mathbf{H}$. The graph signature is*

$$\sigma_t^G(h) = (g_e(h))_{e \in E_t} \in \{0, 1\}^{E_t}.$$

1083 *Two hidden states are indistinguishable by the current graph if their signatures agree. The nonempty*
 1084 *level sets*

$$A_a^t = \{h \in \mathbf{H} : \sigma_t^G(h) = a\}, \quad a \in \{0, 1\}^{E_t},$$

1085 *form a finite partition $\mathcal{P}_t^G$ of hidden-state space, and the generated sigma-algebra is $\sigma(\mathcal{P}_t^G)$. Adding*
 1086 *a new edge is therefore a join with one binary generator:*

$$\mathcal{P}_{t+1}^G = \mathcal{P}_t^G \vee \sigma(g_e).$$

1087 This edge partition is only the skeleton. The exact hard route also needs the ordering of eligible
 1088 anchors by anchor distance; that order is added below as $\mathcal{F}_t^{\text{anchor-order}}$. Centroids and local memories
 1089 are adaptive statistics attached to hard-route states or sparse neighborhoods, not generators of the
 1090 exact Hard-CA information structure. Concept nodes and graph-generated partition atoms are related
 1091 but not identical: a node is a data-structure representative with memory and retrieval statistics; the
 1092 formal information structure is generated by edge signatures and anchor-order events.

1093 **Theorem J.5** (Anchor distinction separates its endpoints, informal version of Theorem Q.1). *If*
 1094 *$a_i \neq a_j$ and u_e, b_e are defined as above for edge $e = (i, j)$, then*

$$u_e^\top a_i - b_e = \frac{1}{2} \|a_i - a_j\|_2, \quad u_e^\top a_j - b_e = -\frac{1}{2} \|a_i - a_j\|_2.$$

1095 *Thus each edge distinction separates the two anchors that caused it.*

1096 Implementations should create such an edge only when the anchor separation clears a numerical
 1097 guard,

$$\|a_i - a_j\|_2 > \eta_{\text{anchor}},$$

1098 for a fixed $\eta_{\text{anchor}} > 0$. If a proposed new anchor is duplicate or nearly duplicate with a confused
 1099 neighbor, the algorithm should skip that edge and fall back to repair, merge, or another admissible
 1100 distinction.

1101 **Theorem J.6** (CA graph generates a finite information structure, informal version of Theorem Q.2).
 1102 *For any finite edge set E_t , equality of signatures $\sigma_t^G(h)$ is an equivalence relation whose classes form*
 1103 *a finite edge-skeleton partition $\mathcal{P}_t^G$. Adding one edge g refines this skeleton by $\mathcal{P}_t^G \vee \sigma(g)$.*

1104 J.4 Soft-CA edge-gated prediction

1105 The hard route selects the nearest eligible anchor using fixed anchors, accepted edges, and anchor-
 1106 distance order, so repairs that change only memories and centroids do not change $\mathcal{F}_t^{\text{CA}}$. Given h ,
 1107 Soft-CA instead computes retrieval distances $d_j(h) = \|h - c_j\|_2^2$ to movable centroids. Moving
 1108 centroids can change the smooth routing weights and therefore the actual Soft-CA predictor, even
 1109 when the Hard-CA skeleton is fixed. Thus centroid repair is a repair in the formal hard-route structure,
 1110 not a strict claim that the soft read map remains partition-measurable. A centroid-only top- k set is not
 1111 sufficient for the formal hard-route guarantee: if the hard eligible anchor never enters the candidate
 1112 set, edge gates cannot recover it. Approximate-nearest-neighbor systems can make the retrieval term
 1113 sublinear or hardware-efficient, but the formal guarantee still needs recall of the hard-route candidate
 1114 [Indyk and Motwani, 1998, Johnson et al., 2019]. We therefore use a recall-oriented candidate set

$$N(h) = N_{k_c}^{\text{centroid}}(h) \cup N_{k_a}^{\text{anchor}}(h) \cup \text{GraphNeighbors}(N_{k_c}^{\text{centroid}}(h)),$$

1115 with duplicates removed and an optional budget cap after the union. The first term gives adaptive
 1116 locality, the second protects the anchor-defined hard route, and the third recovers concepts connected
 1117 by recent consequence conflicts. In exact small-scale implementations one may take $N(h) =$
 1118 $\{1, \dots, N_t\}$. If $\text{Inc}_t(j)$ is the set of edges incident to concept j , define the soft compatibility of h
 1119 with concept j by

$$\log \chi_j(h) = \frac{1}{\sqrt{\max\{1, |\text{Inc}_t(j)|\}}} \sum_{e \in \text{Inc}_t(j)} \beta_e \log \text{sigmoid} \left(\frac{s_{e,j}(u_e^\top h - b_e)}{\tau_e} \right),$$

1120 where structural distinction edges have $\beta_e \in (0, 1]$ and the empty sum gives $\chi_j(h) = 1$. The degree
 1121 normalization avoids systematically suppressing high-degree concepts while preserving hard rejection
 1122 in the low-temperature limit. The edge-gated routing weights are

$$\alpha_j(h) = \frac{\exp(-d_j(h)/T) \chi_j(h)}{\sum_{r \in N(h)} \exp(-d_r(h)/T) \chi_r(h)}, \quad j \in N(h). \quad (3)$$

1123 The prediction is the local mixture

$$\hat{y} = \sum_{j \in N(h)} \alpha_j(h) M_j(h).$$

1124 The hard route makes the information-structure connection explicit. A concept j is eligible for h if
 1125 every incident edge bit places h on the side recorded for j :

$$\begin{cases} g_e(h) = 1, & s_{e,j} = +1, \\ g_e(h) = 0, & s_{e,j} = -1, \end{cases} \quad \text{for every } e \in \text{Inc}_t(j).$$

1126 This convention assigns boundary points to the nonpositive side because $g_e(h) = \mathbf{1}\{u_e^\top h > b_e\}$. The
 1127 exact hard route used in the refinement theory routes to the nearest eligible anchor, falling back to the
 1128 nearest anchor if no eligible concept is available. The soft read in (3) is the trainable implementation:
 1129 it may retrieve by movable centroids, but those centroids are retrieval statistics rather than formal
 1130 information cells.

1131 The distinction partition alone does not determine the whole exact hard route, because eligible
 1132 concepts still need to be ordered. Anchor distances provide this stable order geometry. Let $\mathcal{F}_t^{\text{edge}} =$
 1133 $\sigma(g_e : e \in E_t)$ and let $\mathcal{F}_t^{\text{anchor-order}}$ be the finite sigma-algebra generated by all pairwise anchor-
 1134 distance comparison events

$$\{h : \|h - a_i\|_2^2 \leq \|h - a_j\|_2^2\}, \quad 1 \leq i, j \leq N_t,$$

1135 with deterministic tie-breaking. The hard routing skeleton is measurable with respect to

$$\mathcal{F}_t^{\text{CA}} = \mathcal{F}_t^{\text{edge}} \vee \mathcal{F}_t^{\text{anchor-order}}.$$

1136 The exact CA route map can be written as

$$I_{\mathcal{S}_t}(h) = ((g_e(h))_{e \in E_t}, \text{HardRoute}_{\mathcal{S}_t}(h)).$$

1137 This map is measurable with respect to $\mathcal{F}_t^{\text{CA}}$, so

$$\sigma(I_{S_t}) \subseteq \mathcal{F}_t^{\text{edge}} \vee \mathcal{F}_t^{\text{anchor-order}}.$$

1138 The inclusion can be strict: the full anchor-order sigma-algebra records pairwise order comparisons
 1139 that may not affect the final selected concept. One may use $\mathcal{F}_t^{\text{CA}}$ as a stable upper information
 1140 structure with simple repair-stability proofs, or use the coarser exact-route sigma-algebra $\sigma(I_{S_t})$
 1141 when the selected memory location itself is the desired state. Nodes are memory locations selected
 1142 by this measurable route map; they need not be in one-to-one correspondence with atoms of the
 1143 edge-skeleton partition. The soft read keeps the same anchored graph skeleton but may use movable
 1144 centroids and smooth gates for locality; repairs can therefore change its effective smooth read
 1145 geometry. Soft-CA should be read as a differentiable approximation to the anchor-generated hard
 1146 information map, with exact partition claims recovered only in the hard-route limit or under explicit
 1147 approximation assumptions.

1148 **Theorem J.7** (Anchor-based hard routing is graph-plus-anchor-order measurable, informal version of
 1149 Theorem Q.3). *With finite E_t , finite frozen anchors $a_1, \dots, a_{N_t}$, and deterministic tie-breaking, the
 1150 exact hard-route selected concept index, defined as the nearest eligible anchor with fallback to the
 1151 nearest anchor, is measurable with respect to $\mathcal{F}_t^{\text{CA}} = \mathcal{F}_t^{\text{edge}} \vee \mathcal{F}_t^{\text{anchor-order}}$.*

1152 The next statement is analytic rather than structural: it applies to whichever fixed distances a soft
 1153 read uses. With anchor distances it recovers the exact hard route; with centroid distances it describes
 1154 the behavior of the soft retrieval approximation. It also exposes the candidate-recall requirement that
 1155 links the formal hard route to the sparse implementation.

1156 **Condition J.8** (Candidate recall). *Let $j^*(h)$ be the unique concept selected by the exact hard route
 1157 for query h . The sparse soft read satisfies candidate recall at h if*

$$j^*(h) \in N(h).$$

1158 *If this condition fails, the soft read has no guarantee of selecting the hard route, regardless of how
 1159 sharp the edge gates are.*

1160 **Theorem J.9** (Soft routing approaches hard routing, informal version of Theorem Q.4). *Assume
 1161 candidate recall holds for h , and let j^* be the recalled hard-route concept. Split the remaining
 1162 candidates into eligible competitors, which satisfy the hard structural constraints, and wrong-side
 1163 competitors, which violate at least one structural edge. Eligible competitors are removed by distance:
 1164 if j^* has distance margin $\delta > 0$ over every eligible competitor and the eligible/reference compatibility
 1165 ratios are uniformly bounded, their total soft mass decays exponentially as $T \rightarrow 0$. Wrong-side
 1166 competitors are removed by edge gates: if every wrong-side candidate violates some incident
 1167 structural edge with weight at least $\beta_{\min} > 0$, signed margin at least $\delta_e > 0$, and degree at most
 1168 $D_{\max}$, then its compatibility decays uniformly after degree normalization. The hard-route limit is
 1169 therefore an ordered or controlled joint limit: edge temperature must suppress wrong-side candidates
 1170 before distance temperature makes nearest-neighbor selection decisive. Equivalently, either the
 1171 wrong-side/reference distance-score ratios remain uniformly bounded, or for distance advantage Δ_d
 1172 of wrong-side candidates over an eligible reference,*

$$\frac{\beta_{\min} \delta_e}{\sqrt{D_{\max} \tau_e}} - \frac{\Delta_d}{T} \rightarrow +\infty.$$

1173 *Under either condition, total wrong-side routing mass vanishes. Together with the distance margin for
 1174 eligible competitors, this gives convergence to the hard route. Without candidate recall, the theorem
 1175 gives no correctness guarantee for an omitted hard-route concept.*

Algorithm 2 CA-PREDICT

$\mathcal{S}_t = (\{a_j, c_j, M_j, n_j\}_{j=1}^{N_t}, G_t), \quad x \in \mathcal{X}$
 1: $h \leftarrow \phi_\theta(x)$
 2: $N(h) \leftarrow N_{k_c}^{\text{centroid}}(h) \cup N_{k_a}^{\text{anchor}}(h) \cup \text{GraphNeighbors}(N_{k_c}^{\text{centroid}}(h))$
 3: $\chi_j(h) \leftarrow \exp \left\{ \frac{1}{\sqrt{\max\{1, |\text{Inc}_t(j)|\}}} \sum_{e \in \text{Inc}_t(j)} \beta_e \log \text{sigmoid}(s_{e,j}(u_e^\top h - b_e)/\tau_e) \right\}, \quad j \in N(h)$
 4: $\alpha_j(h) \leftarrow \frac{\exp(-\|h - c_j\|_2^2/T) \chi_j(h)}{\sum_{r \in N(h)} \exp(-\|h - c_r\|_2^2/T) \chi_r(h)}, \quad j \in N(h)$
 5: $\hat{a}_t(h) \leftarrow \sum_{j \in N(h)} \alpha_j(h) M_j(h)$
 6: **return** $\hat{a}_t(h)$

1176 J.5 Refine or repair on a graph

1177 After observing consequence y , CA scores empirical versions of the two gaps in Theorem F.2. A
 1178 repair candidate keeps the CA information map fixed and updates only local state. A refinement
 1179 candidate changes the information map by adding an anchored concept and distinction edges. The
 1180 committed operation is the one whose held-out gain justifies its cost and safety margin.

1181 Let j^* be the highest-weight concept under (3). Define local surprise

$$\rho_t = L(y, M_{j^*}(h)).$$

1182 Define neighborhood consequence disagreement on $N(h)$ by the task-appropriate local prediction
 1183 divergence. For classification one may use

$$D_t = \max_{i,r \in N(h)} \text{TV}(p_i(\cdot | h), p_r(\cdot | h)),$$

1184 where p_j is the distribution represented by memory M_j . For regression one may use

$$D_t = \max_{i,r \in N(h)} \|\mu_i(h) - \mu_r(h)\|_2,$$

1185 and for language modeling the log-loss surprise $-\log p_{j^*}(x_{t+1} | x_{\leq t})$ is the local disagreement
 1186 signal. These quantities are proposal signals. They decide which candidates are worth scoring; they
 1187 do not define repair or refinement.

1188 All gap estimates below are computed on a held-out scoring buffer $\mathcal{B}_t^{\text{score}}$. Refinement proposals
 1189 may use a disjoint proposal buffer $\mathcal{B}_t^{\text{prop}}$ together with the current observation; the scoring buffer
 1190 is held out from proposal generation and from previous accept/reject decisions that determine the
 1191 current state. Theorem J.12 applies when each scoring decision uses fresh held-out samples, or
 1192 samples not previously used to choose proposals or update the state. Reservoir sampling can maintain
 1193 disjoint proposal and scoring streams, but reusing the same validation buffer adaptively requires
 1194 reusable-holdout or adaptive-validation control [Dwork et al., 2015b,a]. Without that extra control,
 1195 the empirical rule is a heuristic rather than the Hoeffding guarantee.

1196 The edges created by a refinement are attached to a concrete confused-neighbor set. Let
 1197 $\text{Div}_{\mathcal{Y}}(M_i, M_r; h)$ denote a task-appropriate consequence divergence, such as total variation for
 1198 classifiers, Euclidean mean distance for regression memories, or next-token distribution divergence
 1199 for language models. Define

$$\text{Confused}(h, y) = \{i \in N(h) : L(y, M_i(h)) > \tau_{\text{conflict}} \text{ or } \text{Div}_{\mathcal{Y}}(M_i, M_{j^*}; h) > \tau_{\text{disagree}}\}.$$

1200 When a budget of at most q new edges is used, CA takes $\text{Confused}_q(h, y)$ to be the q highest-scoring
 1201 members of this set under

$$\text{score}_i(h, y) = \alpha_i(h) L(y, M_i(h)) + \gamma_{\text{div}} \text{Div}_{\mathcal{Y}}(M_i, M_{j^*}; h).$$

1202 If the set is empty but refinement is otherwise accepted, the implementation falls back to the highest-
 1203 score candidates in $N(h) \setminus \{j^*\}$. Thus “confused neighbors” denotes a concrete operator rather than
 1204 an informal phrase.

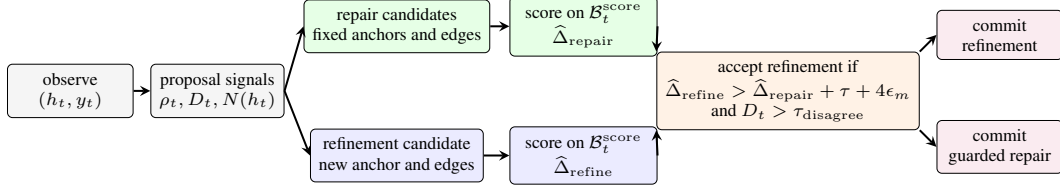


Figure 4: CA refine-or-repair decision. Proposal signals only choose candidates to score; the committed update is selected by held-out empirical gains. A refinement changes the information structure only when its penalized gain beats the best non-worsening repair by the safety margin; otherwise CA repairs local state while preserving anchors and edges.

1205 Repair is guarded rather than blindly committed. The implementation proposes a small set of repair
 1206 candidates, such as a memory-only update, a memory-plus-centroid update, and one or more reduced
 1207 centroid steps:

$$\mathcal{C}_{\text{repair}}(\mathcal{S}_t, h, y) = \{\mathcal{S}_t^{\text{mem}}, \mathcal{S}_t^{\text{mem}+\text{centroid}}, \mathcal{S}_t^{\text{shrink}}, \mathcal{S}_t\}.$$

1208 Each candidate keeps anchors and edges fixed, so it preserves $I_{\mathcal{S}_t}$ and $\mathcal{F}_t^{\text{CA}}$. On the scoring buffer,
 1209 the empirical repair gain is

$$\hat{\Delta}_{\text{repair}} = \hat{R}(\mathcal{S}_t; \mathcal{B}_t^{\text{score}}) - \min_{\mathcal{S}' \in \mathcal{C}_{\text{repair}}} \hat{R}(\mathcal{S}'; \mathcal{B}_t^{\text{score}}).$$

1210 CA's guarded repair candidate is

$$\mathcal{S}_{t+1} \in \underset{\mathcal{S}' \in \mathcal{C}_{\text{repair}}}{\text{argmin}} \hat{R}(\mathcal{S}'; \mathcal{B}_t^{\text{score}}) \quad \text{subject to} \quad \hat{R}(\mathcal{S}'; \mathcal{B}_t^{\text{score}}) \leq \hat{R}(\mathcal{S}_t; \mathcal{B}_t^{\text{score}}). \quad (4)$$

1211 The identity state is in $\mathcal{C}_{\text{repair}}$, so $\hat{\Delta}_{\text{repair}} \geq 0$. Thus monotone repair is an enforced algorithmic
 1212 property, not merely a theorem assumption. The anchor a_{j^*} is never moved by repair. Consequently,
 1213 local memory repair and centroid adaptation do not rewrite previously accepted edge generators.

1214 For a triggered refinement, define the candidate index $j_{\text{new}} = N_t + 1$ and the proposed edge set

$$E_{\text{new}}(h, y) = \{(j_{\text{new}}, i) : i \in \text{Confused}_q(h, y), \|h - a_i\|_2 > \eta_{\text{anchor}}\}.$$

1215 The corresponding candidate state creates

$$a_{j_{\text{new}}} = h, \quad c_{j_{\text{new}}} = h, \quad M_{j_{\text{new}}} = \text{InitMemory}(h, y),$$

1216 with edge set $E_t \cup E_{\text{new}}(h, y)$. Let $\mathcal{S}_t^{\text{new}}$ denote this state, and let $\hat{R}(\mathcal{S}; \mathcal{B}_t^{\text{score}})$ be the empirical local
 1217 loss on the scoring buffer after routing with state $\mathcal{S}$. The empirical penalized refinement gain is

$$\hat{\Delta}_{\text{refine}} = \hat{R}(\mathcal{S}_t; \mathcal{B}_t^{\text{score}}) - \hat{R}(\mathcal{S}_t^{\text{new}}; \mathcal{B}_t^{\text{score}}) - \lambda_{\text{concept}} - \lambda_{\text{edge}} |E_{\text{new}}(h, y)|.$$

1218 Here λ_{concept} is the penalty for creating the anchored concept and its induced anchor-order book-
 1219 keeping, while $\lambda_{\text{edge}} |E_{\text{new}}(h, y)|$ accounts for the accepted edge generators. Using the edge budget
 1220 q instead of the actual $|E_{\text{new}}(h, y)|$ is a conservative implementation choice that only makes the
 1221 acceptance test stricter. The candidate is accepted only if it beats the repair gain by a safety margin:

$$\hat{\Delta}_{\text{refine}} > \max\{\hat{\Delta}_{\text{repair}}, 0\} + \tau + 4\epsilon_m \quad \text{and} \quad D_t > \tau_{\text{disagree}}. \quad (5)$$

1222 Here ϵ_m is a statistical safety margin for a scoring buffer of size $m = |\mathcal{B}_t^{\text{score}}|$; it can be set to zero
 1223 for a purely empirical implementation, or to the adaptive validation margin in Theorem J.12. If
 1224 the refinement test fails, CA commits the guarded repair candidate in (4). This makes refinement a
 1225 statistically guarded gap comparison rather than an unscored growth heuristic.

Algorithm 3 CA-OBSERVE

```

 $\mathcal{S}_t, (x_t, y_t), \mathcal{B}_t^{\text{prop}}, \mathcal{B}_t^{\text{score}}$ 
1:  $h_t \leftarrow \phi_\theta(x_t)$ 
2: if  $N_t = 0$  then
3:    $\mathcal{S}_{t+1} \leftarrow (\{(h_t, h_t, \text{InitMemory}(h_t, y_t), 1)\}, (\{1\}, \emptyset))$ 
4:   return  $\mathcal{S}_{t+1}$ 
5: end if
6:  $\mathcal{N}_t \leftarrow N_{k_c}^{\text{centroid}}(h_t) \cup N_{k_a}^{\text{anchor}}(h_t) \cup \text{GraphNeighbors}(N_{k_c}^{\text{centroid}}(h_t))$ 
7:  $\alpha_{t,j} \leftarrow \alpha_j(h_t), j \in \mathcal{N}_t$ 
8:  $j_t^* \leftarrow \arg\max_{j \in \mathcal{N}_t} \alpha_{t,j}$ 
9:  $\rho_t \leftarrow L(y_t, M_{j_t^*}(h_t))$ 
10:  $D_t \leftarrow \max_{i,r \in \mathcal{N}_t} \text{Div}_Y(M_i, M_r; h_t)$ 
11:  $\mathcal{C}_t^{\text{repair}} \leftarrow \mathcal{C}_{\text{repair}}(\mathcal{S}_t, h_t, y_t)$ 
12:  $\hat{\Delta}_t^{\text{repair}} \leftarrow \hat{R}(\mathcal{S}_t; \mathcal{B}_t^{\text{score}}) - \min_{S' \in \mathcal{C}_t^{\text{repair}}} \hat{R}(S'; \mathcal{B}_t^{\text{score}})$ 
13:  $\mathcal{S}_t^{\text{repair}} \in \arg\min_{S' \in \mathcal{C}_t^{\text{repair}}} \hat{R}(S'; \mathcal{B}_t^{\text{score}})$ 
14:  $z_t \leftarrow \mathbf{1}\{\rho_t > \tau_{\text{repair}} \vee D_t > \tau_{\text{disagree}}\}$ 
15: if  $z_t = 1$  then
16:    $E_t^{\text{new}} \leftarrow E_{\text{new}}(h_t, y_t), \mathcal{S}_t^{\text{new}} \leftarrow \text{AddConcept}(\mathcal{S}_t, h_t, y_t, E_t^{\text{new}}; \mathcal{B}_t^{\text{prop}})$ 
17:    $\hat{\Delta}_t^{\text{refine}} \leftarrow \hat{R}(\mathcal{S}_t; \mathcal{B}_t^{\text{score}}) - \hat{R}(\mathcal{S}_t^{\text{new}}; \mathcal{B}_t^{\text{score}}) - \lambda_{\text{concept}} - \lambda_{\text{edge}} |E_t^{\text{new}}|$ 
18:    $r_t \leftarrow \mathbf{1}\{\hat{\Delta}_t^{\text{refine}} > \max\{\hat{\Delta}_t^{\text{repair}}, 0\} + \tau + 4\epsilon_m \wedge D_t > \tau_{\text{disagree}}\}$ 
19:   if  $r_t = 1$  then
20:      $\mathcal{S}_{t+1} \leftarrow \mathcal{S}_t^{\text{new}}$ 
21:     return  $\mathcal{S}_{t+1}$ 
22:   end if
23: end if
24:  $\mathcal{S}_{t+1} \leftarrow \mathcal{S}_t^{\text{repair}}$ 
25: return  $\mathcal{S}_{t+1}$ 

```

1226 **Definition J.10** (Guarded monotone local repair). *A guarded repair operator is monotone on scoring*
 1227 *buffer $\mathcal{B}_t^{\text{score}}$ if it commits a candidate state $\mathcal{S}_{t+1}$ only when local memories and retrieval centroids*
 1228 *may change, anchors and edges remain fixed, and*

$$\hat{R}(\mathcal{S}_{t+1}; \mathcal{B}_t^{\text{score}}) \leq \hat{R}(\mathcal{S}_t; \mathcal{B}_t^{\text{score}}).$$

1229 *The identity candidate $\mathcal{S}_{t+1} = \mathcal{S}_t$ is always allowed, so the guard is feasible even when every*
 1230 *nontrivial repair would hurt the scoring buffer.*

1231 **Theorem J.11** (Refine-or-repair objective descent, informal version of Theorem Q.5). *Let the local*
 1232 *scoring objective on a fixed scoring buffer $\mathcal{B}_t^{\text{score}}$ be*

$$J_{\mathcal{B}_t^{\text{score}}}(\mathcal{S}) = \hat{R}(\mathcal{S}; \mathcal{B}_t^{\text{score}}) + \lambda_{\text{concept}} N_{\text{concept}}(\mathcal{S}) + \lambda_{\text{edge}} |E(\mathcal{S})|.$$

1233 *If repair is committed by the guarded rule (4), then $J_{\mathcal{B}_t^{\text{score}}}$ does not increase. If a refinement adding*
 1234 *one anchored concept and actual edge set E_{new} is accepted only under (5), then the empirical*
 1235 *penalized objective $J_{\mathcal{B}_t^{\text{score}}}$ decreases by more than $\hat{\Delta}_{\text{repair}} + \tau + 4\epsilon_m$ and in particular by more than*
 1236 *$\tau + 4\epsilon_m$.*

1237 This is a local-buffer statement. It does not imply monotone descent of a global objective when the
 1238 scoring buffer changes over time; a global finiteness claim requires a common potential or validation
 1239 objective.

1240 **Theorem J.12** (Statistically safe adaptive refinement, informal version of Theorem Q.6). *Assume*
 1241 *losses lie in $[0, 1]$. Let $(\mathfrak{H}_t)$ be the filtration containing the past stream, the current CA state, and all*
 1242 *proposal-buffer information available before scoring at time t . Let $\mathcal{C}_t$ be a finite candidate class that*
 1243 *is predictable with respect to $\mathfrak{H}_t$, and let $\mathcal{B}_t^{\text{score}}$ contain m_t fresh held-out samples, or samples not*
 1244 *previously used to choose proposals or update the state, independent of $\mathfrak{H}_t$ conditional on the current*
 1245 *scoring distribution. For candidate g , let $\hat{G}_t(g)$ be the empirical raw risk decrease on $\mathcal{B}_t^{\text{score}}$ before*

1246 complexity cost, and let $G_t(g)$ be the corresponding population raw gain. Define

$$\epsilon_t = \sqrt{\frac{\log(4t^2(|\mathcal{C}_t| + 1)/\delta)}{2m_t}}.$$

1247 Then, with probability at least $1 - \delta$, for every time t and every $g \in \mathcal{C}_t$,

$$|\hat{G}_t(g) - G_t(g)| \leq 2\epsilon_t.$$

1248 Hence an accepted refinement satisfying

$$\hat{G}_t(g) > \lambda \text{cost}(g) + \tau + 2\epsilon_t$$

1249 equivalently $\hat{\Delta}_t^{\text{pen}}(g) = \hat{G}_t(g) - \lambda \text{cost}(g) > \tau + 2\epsilon_t$, has positive population penalized gain after
1250 paying complexity cost and margin τ , uniformly over all accepted times.

1251 **Corollary J.13** (Bounded language-model refinement score, informal version of Corollary Q.7). For
1252 language modeling, raw next-token negative log likelihood is unbounded. Fix $L_{\max} > 0$ and define
1253 the clipped normalized score

$$\ell_{\text{clip}}(x_{\leq t}, x_{t+1}) = \frac{1}{L_{\max}} \min\{-\log p(x_{t+1} \mid x_{\leq t}), L_{\max}\}.$$

1254 If refinement proposal and scoring use ℓ_{clip} , then Theorem J.12 applies to clipped-loss refinement
1255 gain. An accepted refinement has positive clipped population gain after cost and margin with the
1256 stated high probability. This corollary does not claim positive gain for the raw unbounded negative
1257 log likelihood without additional tail assumptions.

1258 **Theorem J.14** (Anchor-stable CA information structure, informal version of Theorem Q.8). Let

$$\mathcal{F}_t^{\text{CA}} = \sigma(g_e : e \in E_t) \vee \sigma\left(\{h : \|h - a_i\|_2^2 \leq \|h - a_j\|_2^2\} : 1 \leq i, j \leq N_t\right).$$

1259 If a repair step changes only retrieval centroids c_j , local memories M_j , and counts n_j , while keeping
1260 anchors a_j and edges E_t fixed, then $\mathcal{F}_{t+1}^{\text{CA}} = \mathcal{F}_t^{\text{CA}}$. If a refinement adds a new anchor and new edge
1261 generators, then the updated information structure is the join of $\mathcal{F}_t^{\text{CA}}$ with the new anchor-order
1262 events and the new edge generators.

1263 Thus the edge skeleton realizes the minimal join operation from Section G. The exact hard-route CA
1264 adds anchor-order events as routing infrastructure whenever a new anchor is inserted. Consequently,
1265 Hard-CA is a structured implementation of refinement, but its exact-route information update is not
1266 always the single-event minimal refinement in the strict Section G sense.

1267 **Corollary J.15** (Finite number of accepted refinements, informal version of Corollary Q.9). If there
1268 is a common nonnegative potential $\Phi(\mathcal{S})$ and every accepted refinement decreases Φ by at least
1269 $\tau > 0$, then the number of accepted refinements is at most $\Phi(\mathcal{S}_0)/\tau$. In particular, this applies to a
1270 fixed scoring buffer or validation objective. The local-buffer descent theorem alone should not be
1271 read as a global finiteness result when the scoring buffer changes over time.

1272 J.6 Data-structure operations

1273 **Definition J.16** (CA data-structure operations). Viewed as a dynamic partition data structure, CA
1274 implements four operations:

$$\text{Query}(h), \quad \text{Repair}(h, y), \quad \text{Refine}(h, y, g), \quad \text{Score}(g, \mathcal{B}).$$

1275 The maintained edge-skeleton invariant is

$$\begin{aligned} \text{Repair} &\implies \mathcal{P}^G(\mathcal{S}_{t+1}) = \mathcal{P}^G(\mathcal{S}_t), \\ \text{Refine}(g) &\implies \mathcal{P}^G(\mathcal{S}_{t+1}) = \mathcal{P}^G(\mathcal{S}_t) \vee \sigma(g), \end{aligned}$$

1276 where $\mathcal{P}^G(\mathcal{S}_t)$ is the edge-generated skeleton partition. The exact hard-route invariant is

$$\begin{aligned} \text{Repair} &\implies \mathcal{F}^{\text{CA}}(\mathcal{S}_{t+1}) = \mathcal{F}^{\text{CA}}(\mathcal{S}_t), \\ \text{Refine} &\implies \mathcal{F}^{\text{CA}}(\mathcal{S}_{t+1}) = \mathcal{F}^{\text{CA}}(\mathcal{S}_t) \vee \sigma(g_e : e \in E_{\text{new}}) \vee \sigma(\text{new anchor-order events}). \end{aligned}$$

1277 Thus repair modifies payloads on the current structure, while refinement changes the structure by
1278 adding both edge generators and the order information induced by new anchors.

Theorem J.17 (CA data-structure cost, informal version of Theorem Q.11). Assume N_t concepts in $\mathbb{R}^d$, candidate set size $|N(h)| = k$, selected incident edge count $Q(h)$, retrieval cost $C_{\text{ret}}(N_t, d)$, and selected local-memory evaluation cost C_{mem} . Then

$$\text{Query}(h) = O(C_{\text{ret}}(N_t, d) + Q(h)d + kd + C_{\text{mem}}).$$

Here $Q(h)$ is an edge count and each dense linear edge gate costs $O(d)$ to evaluate. A repair adds the cost of scoring the proposed local-memory candidates on the scoring buffer. A refinement creating one anchor-centroid pair and q guarded linear distinction edges adds $O(d + qd)$ state-update time, plus the indexing or comparison bookkeeping used to maintain anchor-order queries. With sparse categorical memories of average support $\bar{s}$, the state uses

$$O(N_t(2d + \bar{s}) + |E_t|d)$$

memory, up to indexing overhead.

The theorem is an implementation bound rather than a statistical guarantee. It says that CA realizes the abstract operation $\Pi \mapsto \Pi \vee U$ on the edge skeleton and the corresponding join update on $\mathcal{F}^{\text{CA}}$ with local graph updates and sparse routing, provided the candidate retrieval layer supplies the relevant hard-route concepts.

Comparison to gradient and memory baselines. Backpropagation is not itself a repair or refinement operation; it is an optimizer. If it updates only parameters downstream of a fixed information map, it is a fixed-information repair update in the sense of Definition E.3. If it updates the encoder or routing representation, it may change the induced partition and is outside the repair-only lower bound. Table 5 summarizes the resulting computational comparison against common online optimizers, caches, trees, and parameter-efficient adaptation modules [Rumelhart et al., 1986, Kingma and Ba, 2015, Grave et al., 2017, Khandelwal et al., 2020, Domingos and Hulten, 2000, Houlsby et al., 2019, Hu et al., 2022]. Let P_{tr} be the number of trainable parameters updated by backpropagation, A_{bp} the activation storage required for the backward pass over the adaptation window, M the number of stored examples in a cache or k NN memory, D_{tree} the tree depth, S_{leaf} the per-leaf sufficient statistics, and $\bar{s}$ the average sparse categorical support per CA concept. The table lists runtime-adaptation overhead beyond any shared frozen forward pass that computes h .

J.7 Canonical local memory

Definition J.18 (Calibrated categorical local memory). The local memory M_j is the repair mechanism inside an already chosen concept. For classification and language modeling, the main-text canonical memory is a calibrated categorical memory rather than an unconstrained embedding score, consistent with proper probabilistic scores and calibration diagnostics for probability forecasts [Brier, 1950, Gneiting and Raftery, 2007, Guo et al., 2017]. Concept j stores token or label counts $n_{j,y}$ and $n_j = \sum_y n_{j,y}$. Given a backbone distribution $p_0(y | h)$ and smoothing strength $\alpha_0 > 0$, define

$$p_j(y | h) = \frac{n_{j,y} + \alpha_0 p_0(y | h)}{n_j + \alpha_0}.$$

The routed memory distribution is

$$p_{\text{mem}}(y | h) = \sum_{j \in N(h)} \alpha_j(h) p_j(y | h),$$

and the final prediction is the convex mixture

$$p_{\text{CA}}(y | h) = (1 - \lambda_t) p_0(y | h) + \lambda_t p_{\text{mem}}(y | h), \quad \lambda_t \in [0, 1].$$

This keeps the read path a valid probability distribution. It also gives the simple pointwise safety bound

$$-\log p_{\text{CA}}(y | h) \leq -\log p_0(y | h) - \log(1 - \lambda_t)$$

whenever $\lambda_t < 1$. For log-loss experiments we use a stronger cumulative safeguard based on standard prediction-with-expert-advice and aggregating-algorithm bounds [Vovk, 1990, Freund and Schapire, 1997, Cesa-Bianchi and Lugosi, 2006]. Let $\Lambda = \{\lambda_1, \dots, \lambda_K\} \subseteq [0, 1]$ with $0 \in \Lambda$, define

$$q_{t,k}(y) = (1 - \lambda_k) p_0(y | h_t) + \lambda_k p_{\text{mem}}(y | h_t),$$

Method	Information operation	Per-round adaptation / read overhead	Runtime state memory
Full backprop + Adam	May change information map if encoder/routing parameters move	Backward/update through P_{tr} parameters; activation storage for the adaptation window	Weights, gradients, and Adam moments: $O(P_{\text{tr}})$ each, plus $O(A_{\text{bp}})$ activations
Head-only or linear-probe backprop	Fixed-information repair when features $h = I(\omega)$ are frozen	Backward/update through head parameters P_{head}	$O(P_{\text{head}})$ weights plus optimizer state and head activations
Adapters / LoRA online	Repair if downstream of fixed I ; otherwise may change the information map	Backward/update through adapter parameters P_{ad} in selected layers	$O(P_{\text{ad}})$ weights plus optimizer state and selected-layer activations
Cache or budgeted k NN	Stores examples under a fixed representation; no explicit refinement test	Retrieval $C_{\text{ret}}(M, d)$ plus aggregation over k neighbors	$O(M(d + \bar{s}))$ keys and values, plus index overhead
Online tree / Hoeffding tree	Refines a rooted leaf partition	Query $O(D_{\text{tree}})$; update split statistics on visited leaf	$O(\#\text{leaves} \cdot S_{\text{leaf}})$
CA, exact Hard-CA	Refines graph-generated partition by adding binary generators	Query $O(N_t d + Q(h)d + kd + C_{\text{mem}})$ with exact retrieval; accepted refinement adds $O(d + qd)$	$O(N_t(2d + \bar{s}) + E_t d)$
CA, sparse Soft-CA	Approximate Hard-CA under candidate recall and margins	Query $O(C_{\text{ret}}(N_t, d) + Q(h)d + kd + C_{\text{mem}})$; accepted refinement adds $O(d + qd)$	$O(N_t(2d + \bar{s}) + E_t d)$ plus index overhead

Table 5: Runtime-adaptation cost comparison. CA does not claim to dominate unconstrained full backpropagation statistically. The theoretical separation applies to fixed-information repair methods: when the representation quotient is fixed and has positive refinement gap, they incur the lower bound in Theorem F.3, while CA can pay local graph-memory cost to add a distinction. Full backpropagation through the representation can also change the quotient, but pays backward-pass, activation, and optimizer-state costs.

1318 and predict with a mixture $p_t(y) = \sum_k w_{t,k} q_{t,k}(y)$. After observing y_t , update the trust weights by

$$w_{t+1,k} = \frac{w_{t,k} q_{t,k}(y_t)}{\sum_{\ell=1}^K w_{t,\ell} q_{t,\ell}(y_t)}. \quad (6)$$

1319 **Theorem J.19** (No-harm log-loss trust aggregation, standard). *Initialize $w_{1,k} = 1/K$ and update by*
1320 *(6). For any outcome sequence,*

$$\sum_{t=1}^T -\log p_t(y_t) \leq \min_{k \in [K]} \sum_{t=1}^T -\log q_{t,k}(y_t) + \log K.$$

1321 *Since $0 \in \Lambda$, one expert is the frozen backbone predictor, so*

$$\sum_{t=1}^T -\log p_t(y_t) \leq \sum_{t=1}^T -\log p_0(y_t | h_t) + \log K.$$

1322 *Proof.* Let $Z_t = \sum_k w_{1,k} \prod_{s=1}^t q_{s,k}(y_s)$. The posterior update implies $p_t(y_t) = Z_t/Z_{t-1}$, hence
1323 $\prod_{t=1}^T p_t(y_t) = Z_T$. For every k , $Z_T \geq K^{-1} \prod_{t=1}^T q_{t,k}(y_t)$. Taking negative logarithms and
1324 minimizing over k gives the first bound. The second follows from the $\lambda = 0$ expert. $\square$

1325 Because $p_0(y | h)$, $M_j(h)$, and the soft weights can depend on the hidden state, the practical
1326 predictor need not be constant on atoms of the edge partition alone. The formal interpretation is the

1327 base-information convention from Section D: let

$$\mathcal{B} = \sigma(p_0(\cdot \mid h), \text{fixed pre-outcome features used outside CA routing}).$$

1328 Then CA refinements are claims about the additional runtime information in $\mathcal{F}_t^{\text{CA}}$, and predictors
 1329 are measured as $\mathcal{B} \vee \mathcal{F}_t^{\text{CA}}$ -measurable. The hidden state may be used by the algorithm to compute
 1330 admissible distinctions, proposal scores, and restricted routing functions, but it is not treated as
 1331 arbitrary freely conditionable information in the unrestricted Bayes-risk theory. If full h is admitted
 1332 into $\mathcal{B}$, then all edge predicates and anchor-order events are already $\mathcal{B}$ -measurable and the partition-
 1333 level refinement gap collapses to zero. In the hard one-hot special case, where each memory returns
 1334 an atomwise constant action and no extra base features are used, this reduces to strict partition
 1335 measurability; h -dependent soft memories should be read as restricted-class or approximation
 1336 statements.

1337 Repair is the sufficient-statistic update. For every selected concept $j \in N(h)$,

$$n_{j,y}^+ = n_{j,y} + \alpha_j(h), \quad n_j^+ = n_j + \alpha_j(h),$$

1338 with all unselected counts unchanged. The low-cost hard-update variant is recovered by setting
 1339 $\alpha_{j^*} = 1$ for the highest-weight concept and $\alpha_j = 0$ for all $j \neq j^*$. Embedding-mean memories and
 1340 ridge or projection-style local repair are useful implementation variants, but they should be evaluated
 1341 with calibration metrics before being used for log-loss claims.

1342 J.8 From CA to CoM

1343 The generic CA object above can be attached to many local memories. For sequence models,
 1344 Appendix K gives CoM as an attention-replacement architecture with a causal read-before-write
 1345 protocol and forward-updated concept-value memory.

1346 K CoM: Concept Memory Language Modeling

1347 For language modeling, the world state is a prefix and the consequence is the next token. Concept
 1348 Memory (CoM) replaces the causal attention sublayer inside a decoder block with a concept-addressed
 1349 memory update:

$$\text{CABlock}(X) = X + \text{CA}(\text{Norm}(X)).$$

1350 The block has the same external shape contract as attention, $[B, L, d_{\text{model}}] \mapsto [B, L, d_{\text{model}}]$. The
 1351 residual stream, layer normalization, embeddings, and language-model head are otherwise standard
 1352 decoder components. In a Transformer-style decoder, CA can be paired with the usual MLP sublayer.
 1353 For the state-matched PoST LM runs, the default implementation uses the same mixer-only residual
 1354 scaffold as the Mamba-2 baseline; an MLP sublayer is an explicit configuration option rather than
 1355 part of the main comparison.

1356 K.1 Fixed concept memory

1357 At layer ℓ , CoM keeps a fixed bank of C concept identities $a_{\ell h j} \in \mathbb{R}^{d_a}$ for each head h and concept
 1358 slot j . These identities are sequence-independent layer parameters, initialized as random unit vectors
 1359 and trained by ordinary backpropagation. They are addresses shared across sequences, not online
 1360 state. The online state is the value memory

$$M_t^\ell \in \mathbb{R}^{H \times C \times d_v^c}.$$

1361 Each token produces read queries, write keys, write values, and retention-decay projections:

$$q_t = Q_\ell x_t, \quad k_t = K_\ell x_t, \quad v_t = V_\ell x_t.$$

1362 The read is causal: token t reads only concept values produced by earlier writes. For concept slots
 1363 $j = 1, \dots, C$,

$$\alpha_{thj} = \text{softmax}_j(\gamma_\ell q_{th}^\top a_{\ell h j}), \quad z_{th} = \sum_{j=1}^C \alpha_{thj} M_{t,hj}^\ell, \quad y_t = O_\ell[z_{t1}; \dots; z_{tH}].$$

1364 Multi-head CA uses independent address and value subspaces per head and concatenates the head
 1365 outputs before the output projection, exactly as multi-head attention does.

1366 K.2 Read before write

1367 The update order is the same during training and autoregressive inference:

$$\text{read from } M_t^\ell \rightarrow \text{emit } y_t \rightarrow \text{decay and write } (k_t, v_t) \rightarrow \text{update } M_{t+1}^\ell.$$

1368 This prevents a token from reading its own write. The inference-time update is the test-time learning
1369 mechanism in CoM: the model weights are fixed, but the concept value memory changes after the
1370 causal write and future tokens condition on that updated memory.

1371 For a delayed write key $\bar{k}_t = k_{t-\delta}$, CA computes write weights

$$\beta_{thj} = \text{softmax}_j (\gamma_\ell \bar{k}_{th}^\top a_{\ell h j}).$$

1372 With retention factor ρ_{th} , the token-level streaming update is

$$M_{t+1,hj}^\ell = \rho_{th} M_{t,hj}^\ell + \mathbf{1}_{t \geq \delta} \beta_{thj} v_{th}.$$

1373 For parallel training, CA groups tokens into chunks but keeps the same token-level recurrence. For
1374 chunk $b = [s, e)$, define the exact affine summary

$$U_{b,hj}^\ell = \sum_{t=s}^{e-1} \left(\prod_{u=t+1}^{e-1} \rho_{uh}^\ell \right) \mathbf{1}_{t \geq \delta} \beta_{thj} v_{th}, \quad \rho_{bh}^\ell = \prod_{t=s}^{e-1} \rho_{th}^\ell.$$

1375 The chunk transition is

$$M_{e,hj}^\ell = \rho_{bh}^\ell M_{s,hj}^\ell + U_{b,hj}^\ell.$$

1376 Thus each chunk is summarized by an affine map (ρ_b, U_b) . These summaries compose associatively:

$$(\rho_2, U_2) \odot (\rho_1, U_1) = (\rho_2 \rho_1, U_2 + \rho_2 U_1),$$

1377 with identity $(1, 0)$. The prefix memory before every chunk can therefore be recovered by an exclusive
1378 associative scan over the chunk summaries. After the scan, the implementation computes exact causal
1379 reads inside each chunk:

$$M_{t,hj}^\ell = \left(\prod_{u=s}^{t-1} \rho_{uh}^\ell \right) M_{s,hj}^\ell + \sum_{r=s}^{t-1} \left(\prod_{u=r+1}^{t-1} \rho_{uh}^\ell \right) \mathbf{1}_{r \geq \delta} \beta_{r h j} v_{rh}.$$

1380 Thus the chunked implementation is mathematically equivalent to the streaming read-before-write
1381 recurrence. The block size controls kernel tiling and scan granularity, not a different causal model.

1382 The current fixed-bank CoM path writes every valid token with unit mass through the concept write
1383 distribution, while delayed-key runs use only the causal valid-write mask. The experiments use a
1384 trainable retention factor with a PoST-style ordered decay parameterization, input-dependent step
1385 sizes in a bounded range, and optional position-adaptive scaling. These are parameterization choices
1386 for stable allocation of forgetting time scales, not the definition of CA itself; CA can be run with a
1387 free retention parameterization for ablations.

1388 **First-principles update constraint.** The state update must be more than a compressed token table.
1389 Under log loss, a useful runtime state is one that preserves or creates prediction-relevant distinctions
1390 in the prefix; a rule that only pools token values into slots can repair local averages, but it has no
1391 reason to form a local predictive model. The right test-time object is therefore a predictive statistic in
1392 concept coordinates: after observing a prefix event, the state should update the conditional predictor
1393 used by future concept queries, without running backpropagation at test time.

1394 **Gradient-free Concept Memory.** Let $c_t \in \Delta^{C-1}$ be the concept responsibility of the context
1395 used to predict a consequence Y_t ; in a language model, Y_t is the next observed token or a learned
1396 consequence feature of that token. A concept-memory update is first-principled when it is the closed-
1397 form online estimator for the local prediction problem. For squared loss in a target feature space
1398 $u(Y_t) \in \mathbb{R}^{d_u}$, the sufficient statistics are discounted responsibility mass and first moment:

$$n_{t+1} = \lambda_t n_t + c_t, \quad B_{t+1} = \lambda_t B_t + c_t u(Y_t)^\top.$$

1399 The read used by a future query q_t is the normalized conditional mean

$$z_t = q_t^\top [\text{diag}(n_t + \alpha)^{-1} (B_t + \alpha \mathbf{1} \mu_0^\top)] .$$

1400 This update is causal, online, and gradient-free. It is not a write gate: c_t is the concept responsibility of
 1401 the observed event, n_t is the accumulated evidence for each concept, and the normalization prevents
 1402 frequent concepts from merely accumulating larger vector norms.

1403 For log loss with a finite vocabulary, the exact conjugate analogue is a concept-indexed categorical
 1404 statistic

$$A_{t+1,jv} = \lambda_t A_{t,jv} + c_{tj} \mathbf{1}\{Y_t = v\}, \quad n_{t+1,j} = \lambda_t n_{tj} + c_{tj},$$

1405 and the fast prediction is

$$p_t(v) = \sum_{j=1}^C q_{tj} \frac{A_{t,jv} + \alpha p_0(v)}{n_{tj} + \alpha}.$$

1406 This is the Bayes-optimal repair inside a fixed concept partition for the multinomial-Dirichlet model.
 1407 A full $C \times |V|$ table is usually too large, so the practical CoM form stores low-rank token features
 1408 or residual logit factors instead of full vocabulary counts, but the statistical target is the same: the
 1409 runtime state estimates a local conditional consequence distribution, not a hidden-value cache.

1410 The scalable implementation chooses a consequence feature map

$$u(Y_t) = Re(Y_t) \in \mathbb{R}^{d_u},$$

1411 where $e(Y_t)$ is the output-embedding feature of the observed token and R may be the identity or a
 1412 fixed low-dimensional sketch. The fast state size is then $C(d_u + 1)$ rather than $C(d_{\text{model}} + 1)$ or
 1413 $C|V|$. This compression is not sparse retrieval: every concept can still be read and updated. It is the
 1414 sufficient statistic for the chosen predictive feature space. A learned decoder $D : \mathbb{R}^{d_u} \rightarrow \mathbb{R}^{d_{\text{model}}}$
 1415 maps the read feature back to the residual stream, while the test-time update of (n_t, B_t) remains the
 1416 closed-form forward update above.

1417 The dimension d_u is therefore not a cosmetic hyperparameter. It is the information capacity of
 1418 the repaired consequence statistic. If d_u is too small, the concept state can identify a useful prefix
 1419 condition but cannot carry enough consequence information for the decoder. If $d_u = d_{\text{model}}$ and the
 1420 implementation simply returns the raw embedding moment, the model also loses the learned decoder
 1421 D that maps the statistic into the residual coordinates preferred by the backbone. The empirical
 1422 design point that follows from this view is intermediate: use a large consequence sketch with a
 1423 learned decoder, e.g. $d_u < d_{\text{model}}$ but not extremely compressed, while keeping the update itself
 1424 gradient-free.

1425 The linear-map variant keeps the same principle but lets concepts interact through a ridge predictor.
 1426 Let

$$S_t \in \mathbb{R}^{C \times d_v}$$

1427 be a fast concept map, read by

$$z_t = q_t^\top S_t,$$

1428 and defined as the ridge solution

$$S_t = \left(\alpha I + \sum_{s < t} \omega_s c_s c_s^\top \right)^{-1} \left(\sum_{s < t} \omega_s c_s u(Y_s)^\top \right).$$

1429 Its exact streaming update is the recursive least-squares correction

$$g_t = \frac{P_t c_t}{\lambda_t + c_t^\top P_t c_t}, \quad S_{t+1} = S_t + g_t (u(Y_t) - c_t^\top S_t)^\top,$$

$$P_{t+1} = \lambda_t^{-1} (P_t - g_t c_t^\top P_t).$$

1430 Diagonal or scalar approximations to P_t recover normalized LMS and delta-rule memories. The
 1431 commonly used delta update

$$S_{t+1} = \Lambda_t \left[S_t + \eta_h k_t (v_t - k_t^\top S_t)^\top \right].$$

is therefore best understood as a cheap approximation to a normalized predictive estimator, not as the definition of CoM. The crucial requirements are: the target is a consequence Y_t or its predictive feature, q_t and c_t are attention-style responsibilities over concept identities, and the update is a closed-form online repair of the predictor. This is the forward-only analogue of TTT, but the inner learner is a sufficient-statistic memory rather than a neural network trained by inner backpropagation [Schlag et al., 2021, Sun et al., 2025, Yang et al., 2025]. Causality fixes the order: prediction at position t reads state built from observations before the predicted consequence, and the observation is written only for future predictions.

1440 K.3 Training boundary

There are two separate questions. The first is the fast-state update at test time. For the Concept Memory above, this update is gradient-free: it is an online update of counts, moments, or a closed-form least-squares predictor in concept coordinates. No inference-time loss is differentiated and no model weight is changed. The second question is how to obtain the slow feature maps that produce concept responsibilities, target features, and output logits. The current fixed-bank CoM experiments train those slow neural parameters by ordinary next-token backpropagation, then freeze them at inference. A stricter backprop-free variant would fix or locally learn these feature maps and rely only on the sufficient-statistic updates, but that is a different systems point from the CA state update itself.

This boundary matters for claims. The older additive CA mixer is a bounded attention replacement with a forward-updated concept-value memory. CoM is stronger: its fast state is an explicit online predictor of consequences under the current concept partition. Both perform test-time state learning without inner backpropagation. Only the current slow backbone training uses backpropagation.

Algorithm 4 CONCEPTMEMORYSTEP

- 1: **Input:** query responsibilities q_t , write responsibilities c_t , consequence feature $u(Y_t)$, state (n_t, B_t)
 - 2: read normalized concept means $\mu_{t,j} = (B_{t,j} + \alpha\mu_0)/(n_{t,j} + \alpha)$
 - 3: emit fast prediction feature $z_t = \sum_j q_{tj}\mu_{t,j}$
 - 4: after the causal consequence is observed, update evidence
 - 5: $n_{t+1} \leftarrow \lambda_t n_t + c_t$
 - 6: $B_{t+1} \leftarrow \lambda_t B_t + c_t u(Y_t)^\top$
 - 7: **return** $z_t, (n_{t+1}, B_{t+1})$
-

The same interface can be implemented with categorical counts for exact log-loss repair, with low-rank token features for scalable logits, or with RLS statistics when cross-concept linear prediction is needed. In all cases the state mutation is a deterministic forward update of sufficient statistics. The systems-critical dimension is d_u : reducing it lowers the scan state, the concept read, and the activation footprint without changing the causal order or the statistical interpretation.

CoM design. These observations separate two architectural roles. The slow backbone should compute a strong context representation under the usual parameter budget. The fast Concept Memory should be a dense predictive repair module over that representation. Thus the simplest parameter-matched CoM uses a wider local/gated backbone and a single dense global Concept Memory after the final normalization:

$$h_{1:L} = \text{Backbone}_\theta(x_{1:L}), \quad \tilde{h}_t = h_t + D \left(\sum_{j=1}^C q_{tj} \frac{B_{t,j}}{n_{t,j} + \alpha} \right),$$

where all C concepts are queried and updated for every token. This is not sparse concept routing. The speed path is low-rank consequence compression $d_u \ll d_{\text{model}}$ and a fused dense scan over (n, B) , not top- k selection. A layerwise variant can attach the same dense Concept Memory after each block, but it changes the systems tradeoff: it gives more test-time learners while repeating the same consequence signal many times. The first parameter-matched design point therefore tests the global dense memory before adding layerwise memories.

1469 The same notation also permits multiple retained time scales:

$$n_{t+1}^{(r)} = \lambda_r n_t^{(r)} + c_t, \quad B_{t+1}^{(r)} = \lambda_r B_t^{(r)} + c_t u(Y_t)^\top, \quad z_t = \sum_r \pi_{tr} \sum_j q_{tj} \frac{B_{t,j}^{(r)}}{n_{t,j}^{(r)} + \alpha}.$$

1470 This is a natural extension, but it is not the current main mechanism. In preliminary LM probes,
 1471 learned global or token-dependent scale mixing often collapsed to one dominant time scale, while fixed
 1472 uniform mixing slowed training without improving loss. The stronger signal was the consequence-
 1473 feature capacity d_u , not the number of retention scales. Likewise, injecting CoM into intermediate
 1474 blocks is a useful ablation of where the fast state enters the computation, but the current dense
 1475 implementation was substantially slower and did not outperform the single global CoM. We therefore
 1476 treat multi-scale and in-block CoM as systems/design ablations, and use the global dense CoM with a
 1477 sufficiently large consequence sketch as the primary CoM prototype.

Algorithm 5 CALAYERFORWARD

- 1: **Input:** hidden states $x_{1:L}$, learned concept identities $a_{1:C}$, zero value memory M_0
 - 2: project all queries, write keys, values, and retention rates
 - 3: form dense read and write weights over concept identities
 - 4: form each exact chunk summary (ρ_b, U_b) independently
 - 5: compute all chunk prefix memories by exclusive associative scan over summaries
 - 6: read each chunk using its prefix memory plus exact within-chunk prior writes
 - 7: emit chunk outputs through the output projection
 - 8: **return** contextualized hidden states
-

1478 K.4 Scaling path

1479 The mathematical scaling path is the affine chunk scan above. The reported experiments use dense
 1480 concept reads and dense writes, which is appropriate for state-equalized MQAR and for the first
 1481 language-model runs with $C = 128$. An optimized system can fuse concept scoring, softmax, chunk-
 1482 summary construction, the associative scan, and exact chunk reads, but this is an implementation
 1483 of the same equations rather than a separate algorithm. For Concept Memory, the analogous kernel
 1484 fuses responsibility scoring, count/moment construction in dimension d_u , exact discounted prefix
 1485 scan, normalized read, and the final low-rank decode. For substantially larger concept banks, sharded
 1486 concept banks are the primary systems path; sparse retrieval is an optional acceleration rather than
 1487 the architectural claim. CA changes the memory basis of attention from tokens to concepts, while the
 1488 scan algebra supplies the parallel recurrent evaluation.

1489 L Language-Model Evaluation Details

1490 This appendix specifies the large-scale CoM experiment summarized in Section 6. The goal is to
 1491 match the PoST language-model evaluation protocol while changing only the sequence mixer. All
 1492 models are pretrained as base language models and evaluated without instruction tuning or in-context
 1493 examples.

1494 L.1 Training protocol

1495 L.2 CoM configuration

1496 For Mamba-2, the per-layer runtime state is $d_{\text{inner}} d_{\text{state}}$. For CoM, the online state is CHd_v^c . The
 1497 code exposes C , H , and d_v^c independently, so the experiment can trade concept-memory budget
 1498 against parameter count and channel-mixing capacity. The current 180M preset uses the stable
 1499 parameter-matched fused CoM block from the codebase; the 440M preset keeps the larger state-
 1500 matched CA value width.

1501 The dense full-CoM preset pays the concept-memory cost in every layer. The code also exposes layer-
 1502 budget placement: setting `ca_every_n_layers=4` keeps the same read-before-write CA recurrence
 1503 but runs it only once per four fused mixer blocks, while intervening blocks use the local gated path.
 1504 This is a compute-allocation ablation for the older layerwise CA mixer, not the main CoM direction.

Table 6: CoM training protocol for the large language-model evaluation.

Parameter	Value
Training data	FineWeb-Edu [Lozhkov et al., 2024]
Training context length T_{train}	2,048
Tokenizer	Llama-3.1 tokenizer [Dubey et al., 2024]
Hardware	8× H200
Optimizer	AdamW [Loshchilov and Hutter, 2019]
Warmup	1% of total steps
Precision	BF16 mixed precision
Gradient clipping	$\ \nabla\ _2 \leq 1.0$
Training tokens (180M / 440M)	4B / 9B
Learning rate (180M / 440M)	$6 \times 10^{-4} / 3 \times 10^{-4}$, cosine to 10^{-5}

Table 7: CoM configurations used by the current experiment scripts. Concept identities are learned layer parameters; the online state is only the concept value memory.

Parameter	180M	440M
d_{model}	768	1,024
Number of layers	12	48
Heads H	12	32
Concepts C	128	128
Concept value width d_v^c	128	64
Block size	128	128
Write-key shift	0	0
Write rule	unit mass on valid tokens	unit mass on valid tokens
Fused mixer block	yes, expansion 2	yes, expansion 2
External MLP ratio	0	0
CA online state CHd_v^c	196,608	262,144
Matched Mamba-2 state $d_{\text{inner}}d_{\text{state}}$	$1,536 \times 128$	$2,048 \times 128$
Concept identities	learned unit addresses	learned unit addresses
Concept edges	disabled in fixed-bank runs	disabled in fixed-bank runs
Read/write addressing	untied projections	untied projections
Retention	trainable, PoST-style parameterization	trainable, PoST-style parameterization
Exact chunk-scan kernel	Triton enabled	Triton enabled

The current CoM prototype instead uses a dense global CoM over all C concepts and varies the consequence width d_u . A separate residual branch-scale ablation initializes the CA output multiplier below one; this changes how strongly the CA read is injected into the block, not whether or how tokens write to memory.

L.3 Evaluation protocol

We use the Language Model Evaluation Harness [Gao et al., 2024] in a zero-shot setting. The reported tasks are LAMBADA, HellaSwag, PIQA, ARC-Easy, ARC-Challenge, WinoGrande, and OpenBookQA, matching the protocol in Section 6. LAMBADA perplexity is not mixed into the main zero-shot table; final next-token perplexity is measured by `experiments/eval_final_ppl.py` on the last n samples of the same corpus used for training, using the training tokenizer and non-overlapping 2048-token windows. For reproducibility and systems comparison, each checkpoint should report: final same-corpus negative log likelihood and perplexity, zero-shot task metrics, training tokens per second, prefill latency, decode latency, parameter count, online state size, and CA diagnostics for read scale, slot usage, retention, and decay. The latency table is generated by `scripts/benchmark_lm_latency.py`, which uses synthetic tokens so that all checkpoints are measured with identical batch size, context length, dtype, and hardware.

M From Distinctions to Projection Memory

CA uses refinement and repair in the same runtime object. Distinction learning asks what the learner can tell apart. Projection learning assumes such a representation and asks how a state inside it should be minimally repaired. The repair side is related to Bregman projection, Moore-Penrose

Table 8: Final same-corpus perplexity evaluation. Each checkpoint is scored on the last n samples from the training corpus, tokenized with the training tokenizer and evaluated with non-overlapping windows of length 2048.

Model	Corpus slice	Scored tokens	PPL↓
Mamba-2 180M	train tail	–	–
CoM 180M	train tail	–	–
Mamba-2 440M	train tail	–	–
CoM 440M	train tail	–	–

projection, ridge/Tikhonov repair, passive-aggressive learning, online regression, and fast-weight memory updates [Bregman, 1967, Penrose, 1955, Tikhonov and Arsenin, 1977, Hoerl and Kennard, 1970, Crammer et al., 2006, Vovk, 1997, Ba et al., 2016, Schlag et al., 2021]. This section makes the relation formal in the finite one-hot case.

M.1 One-hot partition memory

Let $\mathcal{P} = \{B_1, \dots, B_n\}$ be a finite partition and let $\phi_{\mathcal{P}} : \Omega \rightarrow \mathbb{R}^n$ be its one-hot feature map:

$$\phi_{\mathcal{P}}(\omega) = e_i \quad \text{if} \quad \omega \in B_i.$$

A memory matrix $W \in \mathbb{R}^{m \times n}$ defines a value map

$$f_W(\omega) = W\phi_{\mathcal{P}}(\omega).$$

This is exactly a table over the atoms of $\mathcal{P}$: every state inside the same atom receives the same value.

Theorem M.1 (One-hot partition memory, informal version of Theorem Q.13). *For every $W \in \mathbb{R}^{m \times n}$, the map $f_W(\omega) = W\phi_{\mathcal{P}}(\omega)$ is $\mathcal{P}$ -measurable. Conversely, every $\mathcal{P}$ -measurable map $f : \Omega \rightarrow \mathbb{R}^m$ is represented by some W through $f = f_W$.*

Thus a one-hot associative memory is not merely analogous to a partition. It is an exact finite representation of all vector-valued predictors measurable with respect to that partition.

M.2 CPM conflict as missing distinction

We take K to be the matrix whose columns are stored keys, so $KK^\dagger$ is the orthogonal projector onto their span. For old key matrices K , a new key k has CPM innovation

$$z = (I - KK^\dagger)k.$$

Conservative Projection Memory detects conflict when a new key has zero innovation but nonzero residual. In the one-hot finite representation, this diagnostic is exactly the statement that two world states have been placed in the same atom despite requiring different values.

Theorem M.2 (CPM conflict equals missing distinction, informal version of Theorem Q.14). *Let $\omega, \omega' \in B_i$ for an atom $B_i \in \mathcal{P}$, and suppose required values satisfy $v(\omega) \neq v(\omega')$. Let $k = \phi_{\mathcal{P}}(\omega) = \phi_{\mathcal{P}}(\omega') = e_i$. If $We_i = v(\omega)$ has already been committed, then evidence requiring $We_i = v(\omega')$ has CPM innovation $z = 0$ and residual $e \neq 0$.*

The theorem explains why a projection update should stop. The key is not new; the old information structure maps both states to the same coordinate. The residual is nonzero because the consequence differs. The missing update is a refinement of $\mathcal{P}$ that separates ω from ω' .

M.3 The hierarchy

The resulting hierarchy is

$$\text{distinction refinement} \supset \text{projection repair} \supset \text{implicit gradient} \supset \text{explicit gradient descent}.$$

Distinction refinement changes the sigma-algebra $\mathcal{F}_t$. Projection repair changes a state s_t inside a fixed information geometry. Gradient descent is the linearized ℓ_2 special case of soft projection,

and backpropagation is the mechanism that computes such gradients in differentiable static graphs [Rumelhart et al., 1986].

This is not a rejection of gradient learning or projection memory. It is a separation of roles. When the representation is adequate, repair and optimization are the right tools. When the representation identifies states that consequences require it to separate, the first learning act is to create the missing distinction.

N CA Implementation Extensions

This appendix collects CA mechanics and large-model details that are useful for implementations but secondary to the main refine-or-repair argument. The main experiments instantiate the fixed-bank CA layer from Section 5. The graph-growing material below explains how the same refine-or-repair principle can be used when concepts and distinction edges are created online.

N.1 Trees as a hard-routing special case

A tree stores distinctions as a hierarchy. A mistake near the root changes every downstream routing decision. A graph stores distinctions locally between concepts. Creating a new concept does not rewrite old paths; it adds a local alternative and edges explaining why the new concept differs from its neighbors. Trees are still contained in the view below, but as a constrained hard-routing case.

Proposition N.1 (Trees as generalized hard-CAs, informal version of Proposition Q.10). *Given a binary distinction tree, construct one generalized hard-route CA concept for each leaf and attach the path distinctions as hard eligibility constraints for that concept. If the generalized hard route selects the unique eligible leaf concept and each concept memory equals the tree leaf predictor, then the generalized CA and the tree make identical predictions.*

Thus the distinction-graph view contains trees after allowing arbitrary concept-level eligibility constraints. A decision tree is the sparse case in which distinctions are forced into a rooted hierarchy and routing is hard and single-path.

N.2 CA data-structure cost

Let

$$Q(h) = \sum_{j \in N(h)} |\text{Inc}_t(j)|$$

be the number of incident edge gates evaluated for the selected candidates. With exact nearest-neighbor search, prediction costs

$$O(N_t d + Q(h)d + |N(h)|d + C_{\text{local}}),$$

where C_{local} is the cost of evaluating the selected local memories and the $Q(h)d$ term evaluates dense linear edge gates. With approximate nearest-neighbor indexing, the $N_t d$ term can be replaced by the query cost of the index [Indyk and Motwani, 1998, Johnson et al., 2019]. Creating an anchor-centroid pair costs $O(d)$ plus the cost of initializing its memory and the bookkeeping used for anchor-order queries. Adding q guarded distinction edges costs $O(qd)$. If each concept stores a sparse categorical memory with average support size $\bar{s}$, runtime memory is

$$O(N_t(2d + \bar{s}) + |E_t|d).$$

An embedding-mean variant instead uses $O(N_t(2d + d_e) + |E_t|d)$ memory, but it is not a calibrated log-loss memory without an additional probability model.

Theorem N.2 (CA data-structure bound, informal version of Theorem Q.11). *For exact search over N_t centroids in $\mathbb{R}^d$, one edge-gated CA prediction costs $O(N_t d + Qd + |N|d + C_{\text{local}})$, where $N = N(h)$ and $Q = Q(h)$ is an edge count. One observation that repairs a selected concept costs this plus the guarded local repair scoring cost. One observation that creates a new anchor-centroid pair and q guarded distinction edges costs an additional $O(d + qd)$ plus anchor-order query bookkeeping. With sparse categorical memories of average support $\bar{s}$, the runtime state uses $O(N_t(2d + \bar{s}) + |E_t|d)$ memory.*

1598 N.3 Ridge local memory

1599 For few-shot adaptation or local readout repair, a memory stores support embeddings $X_{\text{sup}} \in \mathbb{R}^{n \times d}$
 1600 and labels $Y \in \mathbb{R}^{n \times C}$. The closed-form adapter is the dual ridge/Tikhonov solution [Tikhonov and
 1601 Arsenin, 1977, Hoerl and Kennard, 1970]:

$$\Gamma = (X_{\text{sup}} X_{\text{sup}}^\top + \lambda I)^{-1} Y, \quad \text{logits}(z) = z X_{\text{sup}}^\top \Gamma.$$

1602 Projection Memory and CPM are advanced repair rules inside M_j after the graph has fixed the
 1603 relevant local concept; Appendix M treats that relation directly.

1604 N.4 Training modes for representation drift

1605 Anchors, centroids, and edges live in query space. If the encoder or query map changes rapidly while
 1606 the CA state persists, old anchors and edges describe stale geometry. Reports should therefore state
 1607 which training mode is used.

1608 **Mode A: frozen encoder.** Train the backbone and query map first, then freeze ϕ_θ , Q , and the
 1609 backbone distribution used for CA smoothing while running CA runtime learning. This is the cleanest
 1610 protocol for the first experiments and is the default interpretation of the theory.

1611 **Mode B: EMA query encoder.** Maintain a slowly moving query encoder

$$\bar{\theta} \leftarrow \beta \bar{\theta} + (1 - \beta) \theta,$$

1612 and create anchors, centroids, and edge generators in the query space of $\phi_{\bar{\theta}}$ and $Q_{\bar{\theta}}$. The backbone
 1613 can still train, but the CA state is tied to the slower geometry.

1614 **Mode C: periodic state rebuild.** During end-to-end training, rebuild or clear the CA state every K
 1615 optimization steps, or rebuild it from a replay buffer under the current query map. The formal guaran-
 1616 tees then apply within a rebuild interval. Cross-interval state carryover is a systems approximation,
 1617 not a claim of anchor-stable refinement.

1618 N.5 Non-canonical adapter extension

1619 Beyond the categorical next-token memory in Appendix K, a richer optional adapter-style layer stores
 1620 a local linear repair map, related to parameter-efficient adapter and LoRA modules but updated as a
 1621 local CA memory [Houlsby et al., 2019, Hu et al., 2022]. It requires an explicit residual target and
 1622 should not be conflated with the calibrated next-token memory mechanism:

$$\mathcal{S}_t^{\text{adp}} = \{a_j, c_j, W_j, n_j\}_{j=1}^{N_t} \cup G_t, \quad W_j \in \mathbb{R}^{d \times r}.$$

1623 Let $K : \mathbb{R}^d \rightarrow \mathbb{R}^r$ be a learned or fixed key projection and set $k_t = K h_t$. The adapter read is

$$\Delta h_t = \sum_{j \in N(q_t)} \alpha_j(q_t) W_j k_t, \quad \tilde{h}_t = h_t + \gamma_\theta(h_t) \Delta h_t.$$

1624 Here $W_j k_t$ is a hidden-space residual, so the write rule requires an explicit residual target $v_t \in \mathbb{R}^d$.
 1625 Let $P_{\text{emb} \rightarrow h} : \mathbb{R}^{d_e} \rightarrow \mathbb{R}^d$ be a chosen projection from output-embedding space to hidden-residual
 1626 space. Natural choices include a token-embedding target $P_{\text{emb} \rightarrow h} e_{x_{t+1}}$, a teacher-forced output-error
 1627 target $P_{\text{emb} \rightarrow h} (e_{x_{t+1}} - \hat{e}_t)$, or a distillation target $h_t^{\text{teacher}} - h_t$. Without such a target, the adapter is
 1628 only an architectural extension.

1629 Given a local residual target, the projected rank-one repair for a selected concept is the same residual-
 1630 correction pattern that appears in passive-aggressive updates, RLS/ridge-style online least squares,
 1631 and delta-rule fast-weight memories [Crammer et al., 2006, Vovk, 1997, Schlag et al., 2021]:

$$W_j^+ = W_j + \frac{\eta \alpha_j(q_t)}{1 + \eta \alpha_j(q_t) \|k_t\|_2^2} (v_t - W_j k_t) k_t^\top.$$

1632 Equivalently, a memory may maintain small ridge sufficient statistics and solve the local adapter in
 1633 closed form [Tikhonov and Arsenin, 1977, Hoerl and Kennard, 1970].

1634 N.6 Cost and systems notes

1635 During sequential detached training, backpropagation sees the selected read computation, while the
 1636 CA state is updated in the same order as streaming inference. The important comparison is with
 1637 test-time gradient adaptation [Sun et al., 2020, Wang et al., 2021, Niu et al., 2022, Zhang et al., 2022,
 1638 Sun et al., 2025]. A TTT-style layer typically performs an inner backward pass at inference time. A
 1639 CA layer performs retrieval, sparse local read, and closed-form or sufficient-statistic repair. The cost
 1640 is forward-only:

$$\text{CA inference} = \text{backbone forward} + \text{retrieval} + \text{sparse local repair}.$$

1641 **Theorem N.3** (CA layer cost, informal version of Theorem Q.12). *Consider a CA adapter extension*
 1642 *with hidden dimension d , query dimension d_q , local rank r , candidate count $|N|$, selected incident*
 1643 *edge-gate count Q_N , and concept-bank query cost $C_{\text{ret}}(N_t, d_q)$. For one token, the read overhead is*

$$O(dd_q + C_{\text{ret}}(N_t, d_q) + Q_N d_q + |N| dr).$$

1644 *If the write path is detached and uses rank-one local repair, the write overhead is*

$$O(dd_q + C_{\text{ret}}(N_t, d_q) + Q_N d_q + |N| (d_q + dr)).$$

1645 *For a block of length L , the CA state history need not be stored for backpropagation in detached*
 1646 *sparse-adapter variants; the additional activation memory is the selected read computation, not*
 1647 *the full sequence of memory states. This is not the large-model CoM implementation used in the*
 1648 *main experiments. The implemented CoM mixer uses a dense static concept bank with learned*
 1649 *addresses, differentiable concept read/write weights, retained value writes, and an exact chunked*
 1650 *concept-memory scan.*

1651 The concept bank can be sharded by the same principle used in retrieval and sparse expert systems.
 1652 Each device owns a subset of concepts. A query is routed to a small set of shards, each shard returns
 1653 local top- k candidates, and a final top- k merge selects the memories used by the read path. Only
 1654 selected memories receive gradients. Detached writes are sent back to the owning shards.

1655 Budgeting is necessary for long training. A practical CA layer maintains $N_t \leq N_{\text{max}}$, merges only
 1656 compatible unprotected concepts, and otherwise evicts or compresses low-score memories. These
 1657 budget actions are maintenance operations, not learning operations in the formal refine-or-repair sense.
 1658 The refine-or-repair theorems apply between maintenance events; merge, eviction, and compression
 1659 may destroy information and must be evaluated as resource-management approximations.

1660 N.7 Closed-form local repair

1661 Ridge memory is the main closed-form adapter used in our checks. Let $X_{\text{sup}} \in \mathbb{R}^{n \times d}$ be normalized
 1662 support embeddings stored by a CA memory and let $Y \in \mathbb{R}^{n \times C}$ be one-hot consequences. The
 1663 adapter computes the dual ridge/Tikhonov solution [Tikhonov and Arsenin, 1977, Hoerl and Kennard,
 1664 1970]

$$\Gamma = (X_{\text{sup}} X_{\text{sup}}^\top + \lambda I)^{-1} Y, \quad \text{logits}(z) = z X_{\text{sup}}^\top \Gamma.$$

1665 This is projection-style repair: it fits the local adapter in one linear solve rather than by iterated
 1666 backpropagation. In a large model, the same idea can be applied per concept, per document, or per
 1667 retrieved shard after the relevant distinction structure has been fixed.

1668 O Minimal Refinement Proofs

1669 The statements in this appendix use standard facts about finite partitions, generated sigma-algebras,
 1670 and measurability [Billingsley, 1995, Kallenberg, 2002]. They are recalled only to fix notation for
 1671 the runtime-learning interpretation: refinement is the minimal information update needed by new
 1672 evidence.

1673 **Theorem O.1** (Minimal finite refinement, formal version of Proposition 3.1). *Let Ω be finite, let $\mathcal{P}$*
 1674 *be a partition of Ω , and let $D \subseteq \Omega$. Define*

$$\mathcal{P}_D = \{B \cap D : B \in \mathcal{P}, B \cap D \neq \emptyset\} \cup \{B \cap D^c : B \in \mathcal{P}, B \cap D^c \neq \emptyset\}.$$

1675 *Then:*

- 1676 • **Part 1.** $\mathcal{P}_D$ is a partition of Ω .
- 1677 • **Part 2.** $\mathcal{P}_D$ refines $\mathcal{P}$.
- 1678 • **Part 3.** D is $\mathcal{P}_D$ -measurable.
- 1679 • **Part 4.** If $\mathcal{Q}$ refines $\mathcal{P}$ and D is $\mathcal{Q}$ -measurable, then $\mathcal{Q}$ refines $\mathcal{P}_D$.

1680 Consequently, $\mathcal{P}_D$ is the unique minimal refinement of $\mathcal{P}$ making D measurable.

1681 *Proof.* For each $B \in \mathcal{P}$, the nonempty sets among $B \cap D$ and $B \cap D^c$ are disjoint and have union B .
 1682 Since the atoms of $\mathcal{P}$ are disjoint and cover Ω , the atoms of $\mathcal{P}_D$ are disjoint and cover Ω . Thus Part 1
 1683 holds.

1684 Every atom of $\mathcal{P}_D$ is contained in some $B \in \mathcal{P}$, so Part 2 holds. Also

$$D = \bigcup_{B \in \mathcal{P}} (B \cap D),$$

1685 where empty intersections are ignored. Hence D is a union of atoms of $\mathcal{P}_D$, proving Part 3.

1686 Let $\mathcal{Q}$ refine $\mathcal{P}$ and make D measurable. For any atom $Q \in \mathcal{Q}$, there exists $B \in \mathcal{P}$ with

$$Q \subseteq B.$$

1687 Since D is $\mathcal{Q}$ -measurable, each atom of $\mathcal{Q}$ is either contained in D or disjoint from D . Hence

$$Q \subseteq B \cap D \quad \text{or} \quad Q \subseteq B \cap D^c.$$

1688 Thus each atom of $\mathcal{Q}$ is contained in an atom of $\mathcal{P}_D$, so $\mathcal{Q}$ refines $\mathcal{P}_D$. This proves Part 4.

1689 If another partition has the same minimal property, it refines $\mathcal{P}_D$ and is refined by $\mathcal{P}_D$, so the two
 1690 partitions have the same atoms. $\square$

1691 **Theorem O.2** (Minimal sigma-algebra refinement, formal version of Theorem G.4). *Let $(\Omega, \mathcal{E})$ be*
 1692 *a measurable space, let $\mathcal{F} \subseteq \mathcal{E}$ be a sigma-algebra, and let $U : \Omega \rightarrow \mathcal{U}$ be a measurable map into*
 1693 *a measurable space $(\mathcal{U}, \Sigma_{\mathcal{U}})$. The join $\mathcal{F} \vee \sigma(U)$ is the smallest sigma-algebra $\mathcal{G} \subseteq \mathcal{E}$ such that*
 1694 *$\mathcal{F} \subseteq \mathcal{G}$ and U is $\mathcal{G}$ -measurable.*

1695 *Proof.* By definition,

$$\mathcal{F} \vee \sigma(U) = \sigma(\mathcal{F} \cup \sigma(U)).$$

1696 Therefore $\mathcal{F} \subseteq \mathcal{F} \vee \sigma(U)$ and $\sigma(U) \subseteq \mathcal{F} \vee \sigma(U)$, so U is measurable with respect to $\mathcal{F} \vee \sigma(U)$.

1697 Let $\mathcal{G}$ be any sigma-algebra such that $\mathcal{F} \subseteq \mathcal{G}$ and U is $\mathcal{G}$ -measurable. The latter condition means
 1698 $\sigma(U) \subseteq \mathcal{G}$. Hence

$$\mathcal{F} \cup \sigma(U) \subseteq \mathcal{G}.$$

1699 Since $\mathcal{G}$ is a sigma-algebra,

$$\sigma(\mathcal{F} \cup \sigma(U)) \subseteq \mathcal{G}.$$

1700 Thus $\mathcal{F} \vee \sigma(U) \subseteq \mathcal{G}$. $\square$

1701 **Theorem O.3** (Contradiction requires distinction, formal version of Corollary F.6). *Let $I : \Omega \rightarrow \mathcal{Z}$*
 1702 *be an information map and let $y : \Omega \rightarrow \mathcal{A}$ be a required output. Suppose $I(\omega) = I(\omega')$ but*
 1703 *$y(\omega) \neq y(\omega')$. Then no predictor $f = g \circ I$ satisfies both*

$$f(\omega) = y(\omega), \quad f(\omega') = y(\omega').$$

1704 *Proof.* Since $f = g \circ I$ and $I(\omega) = I(\omega')$,

$$f(\omega) = f(\omega').$$

1705 If both requirements held, then

$$y(\omega) = f(\omega) = f(\omega') = y(\omega'),$$

1706 contradicting $y(\omega) \neq y(\omega')$. $\square$

1707 P Value of Distinction Proofs

1708 The proofs below use standard tools from statistical decision theory, information theory, proper scoring
 1709 rules, and conditional expectation: Bayes risk under constrained information, log-loss optimality
 1710 of the conditional law, and the L^2 projection identity for conditional expectation [Blackwell, 1953,
 1711 Berger, 1985, Cover and Thomas, 2006, Gneiting and Raftery, 2007, Billingsley, 1995, Kallenberg,
 1712 2002]. The derivations are included only to specialize these standard identities to the paper's notation.

1713 Throughout the measurable-space statements, the action space A is equipped with a sigma-algebra
 1714 Σ_A , actions are measurable maps into (A, Σ_A) , and losses are measurable with finite expectation for
 1715 the actions under consideration.

1716 **Theorem P.1** (Loss-relevant refinement, formal version of Theorem 3.2). *Let $\mathcal{F} \subseteq \mathcal{G} \subseteq \mathcal{E}$. Assume*
 1717 *Bayes actions exist under $\mathcal{F}$ and $\mathcal{G}$, and that the $\mathcal{G}$ -Bayes action is almost surely unique. Then*

$$R_L(\mathcal{F}) - R_L(\mathcal{G}) = 0$$

1718 *if and only if $a_{\mathcal{G}}^*$ has an $\mathcal{F}$ -measurable version.*

1719 *Proof.* If $a_{\mathcal{G}}^*$ is $\mathcal{F}$ -measurable, then it is feasible for $R_L(\mathcal{F})$, so

$$R_L(\mathcal{F}) \leq \mathbb{E}[L(Y, a_{\mathcal{G}}^*)] = R_L(\mathcal{G}).$$

1720 The inclusion $\mathcal{F} \subseteq \mathcal{G}$ gives $R_L(\mathcal{G}) \leq R_L(\mathcal{F})$, so the risks are equal. Conversely, if the risks are
 1721 equal, let $a_{\mathcal{F}}^*$ be an $\mathcal{F}$ -Bayes action. It is also $\mathcal{G}$ -measurable and achieves $R_L(\mathcal{G})$. By almost sure
 1722 uniqueness of $a_{\mathcal{G}}^*$, $a_{\mathcal{F}}^* = a_{\mathcal{G}}^*$ almost surely. Hence $a_{\mathcal{G}}^*$ has an $\mathcal{F}$ -measurable version. $\square$

1723 **Theorem P.2** (Bayes-risk monotonicity, formal version of Theorem H.2). *Let Y be a target random*
 1724 *variable, let A be an action space, and let L be an integrable loss. If $\mathcal{F} \subseteq \mathcal{G}$, then*

$$R_L(\mathcal{G}) \leq R_L(\mathcal{F}).$$

1725 *Proof.* Define

$$\mathcal{D}_{\mathcal{F}} = \{a : \Omega \rightarrow A : a \text{ is } \mathcal{F}\text{-measurable}\}, \quad \mathcal{D}_{\mathcal{G}} = \{a : \Omega \rightarrow A : a \text{ is } \mathcal{G}\text{-measurable}\}.$$

1726 Since $\mathcal{F} \subseteq \mathcal{G}$,

$$\mathcal{D}_{\mathcal{F}} \subseteq \mathcal{D}_{\mathcal{G}}.$$

1727 Therefore

$$\begin{aligned} R_L(\mathcal{G}) &= \inf_{a \in \mathcal{D}_{\mathcal{G}}} \mathbb{E}[L(Y, a)] \\ &\leq \inf_{a \in \mathcal{D}_{\mathcal{F}}} \mathbb{E}[L(Y, a)] \\ &= R_L(\mathcal{F}). \end{aligned}$$

1728 $\square$

1729 **Theorem P.3** (Log-loss value, formal version of Theorem H.4). *Let Y be finite and let $\mathcal{F} \subseteq \mathcal{G}$. Under*
 1730 *log loss, an admissible action is a measurable conditional distribution $q(\cdot)$ on the range of Y , and*
 1731 *the loss is $-\log q(Y)$. Then*

$$R_{\log}(\mathcal{F}) - R_{\log}(\mathcal{G}) = I(Y; \mathcal{G} \mid \mathcal{F}) = \mathbb{E}[D_{\text{KL}}(\mathbb{P}(Y \in \cdot \mid \mathcal{G}) \parallel \mathbb{P}(Y \in \cdot \mid \mathcal{F}))].$$

1732 *Proof.* For any $\mathcal{F}$ -measurable conditional distribution q , write $p_{\mathcal{F}}(y) = \mathbb{P}(Y = y \mid \mathcal{F})$. Then

$$\begin{aligned} \mathbb{E}[-\log q(Y) \mid \mathcal{F}] &= \sum_y p_{\mathcal{F}}(y)(-\log q(y)) \\ &= \sum_y p_{\mathcal{F}}(y)(-\log p_{\mathcal{F}}(y)) + \sum_y p_{\mathcal{F}}(y) \log \frac{p_{\mathcal{F}}(y)}{q(y)} \\ &= H(Y \mid \mathcal{F}) + D_{\text{KL}}(p_{\mathcal{F}} \parallel q). \end{aligned}$$

1733 Taking expectation and minimizing over q gives

$$R_{\log}(\mathcal{F}) = H(Y \mid \mathcal{F}).$$

1734 Likewise,

$$R_{\log}(\mathcal{G}) = H(Y \mid \mathcal{G}).$$

1735 By definition of conditional mutual information between Y and the additional information $\mathcal{G}$ given
1736 $\mathcal{F}$,

$$I(Y; \mathcal{G} \mid \mathcal{F}) = H(Y \mid \mathcal{F}) - H(Y \mid \mathcal{G}).$$

1737 The conditional-KL form follows from the tower property:

$$\begin{aligned} & \mathbb{E}[D_{\text{KL}}(\mathbb{P}(Y \in \cdot \mid \mathcal{G}) \parallel \mathbb{P}(Y \in \cdot \mid \mathcal{F}))] \\ &= \mathbb{E}\left[\sum_y p_{\mathcal{G}}(y) \log p_{\mathcal{G}}(y) - \sum_y p_{\mathcal{G}}(y) \log p_{\mathcal{F}}(y)\right] \\ &= -H(Y \mid \mathcal{G}) - \mathbb{E}\left[\sum_y p_{\mathcal{F}}(y) \log p_{\mathcal{F}}(y)\right] \\ &= H(Y \mid \mathcal{F}) - H(Y \mid \mathcal{G}). \end{aligned}$$

1738 Combining proves the claim. $\square$

1739 **Corollary P.4** (Binary distinction value, formal version of Corollary H.5). *Let $U \in \{0, 1\}$ and*
1740 *$\mathcal{G} = \mathcal{F} \vee \sigma(U)$. Under log loss,*

$$\begin{aligned} R_{\log}(\mathcal{F}) - R_{\log}(\mathcal{G}) &= I(Y; U \mid \mathcal{F}) \\ &= \mathbb{E}\left[\sum_{u \in \{0, 1\}} \mathbb{P}(U = u \mid \mathcal{F}) D_{\text{KL}}(\mathbb{P}(Y \in \cdot \mid \mathcal{F}, U = u) \parallel \mathbb{P}(Y \in \cdot \mid \mathcal{F}))\right]. \end{aligned}$$

1741 *Proof.* Apply Theorem P.3 with $\mathcal{G} = \mathcal{F} \vee \sigma(U)$. Conditioning on $\mathcal{F} \vee \sigma(U)$ is the same as conditioning
1742 on $\mathcal{F}$ and the binary value of U , yielding the displayed expansion over $u \in \{0, 1\}$. $\square$

1743 **Theorem P.5** (Squared-loss value, formal version of Theorem H.6). *Let $Y \in L^2(\Omega; \mathbb{R}^d)$ and let*
1744 *$L(Y, a) = \|Y - a\|_2^2$. If $\mathcal{F} \subseteq \mathcal{G}$, then*

$$R_2(\mathcal{F}) - R_2(\mathcal{G}) = \mathbb{E}\left[\|\mathbb{E}[Y \mid \mathcal{G}] - \mathbb{E}[Y \mid \mathcal{F}]\|_2^2\right].$$

1745 *Proof.* The optimal $\mathcal{F}$ -measurable predictor is $\mathbb{E}[Y \mid \mathcal{F}]$, and the optimal $\mathcal{G}$ -measurable predictor is
1746 $\mathbb{E}[Y \mid \mathcal{G}]$. Let

$$\varepsilon = Y - \mathbb{E}[Y \mid \mathcal{G}], \quad \delta = \mathbb{E}[Y \mid \mathcal{G}] - \mathbb{E}[Y \mid \mathcal{F}].$$

1747 Then

$$Y - \mathbb{E}[Y \mid \mathcal{F}] = \varepsilon + \delta.$$

1748 Since δ is $\mathcal{G}$ -measurable and $\mathbb{E}[\varepsilon \mid \mathcal{G}] = 0$,

$$\mathbb{E}[\langle \varepsilon, \delta \rangle \mid \mathcal{G}] = \langle \mathbb{E}[\varepsilon \mid \mathcal{G}], \delta \rangle = 0.$$

1749 Thus

$$\begin{aligned} \mathbb{E}\|Y - \mathbb{E}[Y \mid \mathcal{F}]\|_2^2 &= \mathbb{E}\|\varepsilon + \delta\|_2^2 \\ &= \mathbb{E}\|\varepsilon\|_2^2 + \mathbb{E}\|\delta\|_2^2 \\ &= R_2(\mathcal{G}) + \mathbb{E}\left[\|\mathbb{E}[Y \mid \mathcal{G}] - \mathbb{E}[Y \mid \mathcal{F}]\|_2^2\right]. \end{aligned}$$

1750 Rearranging proves the identity. $\square$

1751 **Theorem P.6** (Minimal loss-sufficient refinement). *Let $\mathcal{F} \subseteq \mathcal{G}$ and assume the $\mathcal{G}$ -Bayes action*
 1752 *$a_{\mathcal{G}}^* : \Omega \rightarrow \mathcal{A}$ exists as a $(\mathcal{G}, \Sigma_{\mathcal{A}})$ -measurable map and is almost surely unique. Also assume the Bayes*
 1753 *infimum is attained for every intermediate sigma-algebra $\mathcal{H}$ with $\mathcal{F} \subseteq \mathcal{H} \subseteq \mathcal{G}$ that is considered*
 1754 *below. Up to null sets, the smallest sigma-algebra $\mathcal{H}$ satisfying*

$$\mathcal{F} \subseteq \mathcal{H} \subseteq \mathcal{G}, \quad R_L(\mathcal{H}) = R_L(\mathcal{G})$$

1755 *is $\mathcal{F} \vee \sigma(a_{\mathcal{G}}^*)$. For finite Y under log loss, the corresponding smallest sufficient refinement is $\mathcal{F} \vee \sigma(p_{\mathcal{G}})$,*
 1756 *where $p_{\mathcal{G}}(\cdot) = \mathbb{P}(Y \in \cdot \mid \mathcal{G})$.*

1757 *Proof.* The action $a_{\mathcal{G}}^*$ is measurable with respect to $\mathcal{F} \vee \sigma(a_{\mathcal{G}}^*)$, so this refinement attains $R_L(\mathcal{G})$.
 1758 Let $\mathcal{H}$ be any intermediate sigma-algebra with $R_L(\mathcal{H}) = R_L(\mathcal{G})$. Since $\mathcal{H} \subseteq \mathcal{G}$ and the $\mathcal{G}$ -Bayes
 1759 action is almost surely unique, Theorem P.1 implies that $a_{\mathcal{G}}^*$ has an $\mathcal{H}$ -measurable version. Therefore
 1760 $\sigma(a_{\mathcal{G}}^*) \subseteq \mathcal{H}$ up to null sets, and because $\mathcal{F} \subseteq \mathcal{H}$,

$$\mathcal{F} \vee \sigma(a_{\mathcal{G}}^*) \subseteq \mathcal{H}$$

1761 up to null sets. Under log loss, the Bayes action is the conditional law $p_{\mathcal{G}}$, giving the stated form. $\square$

1762 **Proposition P.7** (Finite squared-loss split identity). *Let $B = B_0 \cup B_1$ be a disjoint split of a finite*
 1763 *sample set. With empirical means $\hat{\mu}_B, \hat{\mu}_{B_0}, \hat{\mu}_{B_1}$ and SSE as in Section H,*

$$\text{SSE}(B) - \text{SSE}(B_0) - \text{SSE}(B_1) = |B_0| \|\hat{\mu}_{B_0} - \hat{\mu}_B\|_2^2 + |B_1| \|\hat{\mu}_{B_1} - \hat{\mu}_B\|_2^2.$$

1764 *Proof.* For each child B_b ,

$$\begin{aligned} \sum_{j \in B_b} \|y_j - \hat{\mu}_B\|_2^2 &= \sum_{j \in B_b} \|y_j - \hat{\mu}_{B_b} + \hat{\mu}_{B_b} - \hat{\mu}_B\|_2^2 \\ &= \sum_{j \in B_b} \|y_j - \hat{\mu}_{B_b}\|_2^2 + |B_b| \|\hat{\mu}_{B_b} - \hat{\mu}_B\|_2^2, \end{aligned}$$

1765 because

$$\sum_{j \in B_b} (y_j - \hat{\mu}_{B_b}) = 0.$$

1766 Summing over $b \in \{0, 1\}$ gives the result. $\square$

1767 Q CA Algorithm and Projection-Bridge Proofs

1768 This appendix combines elementary finite graph/partition arguments with standard concentration and
 1769 linear-algebra tools: Hoeffding sample-split validation bounds, adaptive-holdout caveats, nearest-
 1770 neighbor indexing costs, Moore-Penrose projections, and ridge/Tikhonov least squares [Hoeffding,
 1771 1963, Devroye et al., 1996, Dwork et al., 2015b,a, Indyk and Motwani, 1998, Johnson et al., 2019,
 1772 Penrose, 1955, Tikhonov and Arsenin, 1977, Hoerl and Kennard, 1970].

1773 **Theorem Q.1** (Anchor edge separation, formal version of Theorem J.5). *Let $a_i, a_j \in \mathbb{R}^d$ with*
 1774 *$a_i \neq a_j$. Define*

$$u = \frac{a_i - a_j}{\|a_i - a_j\|_2}, \quad b = \frac{1}{2} u^\top (a_i + a_j).$$

1775 *Then*

$$u^\top a_i - b = \frac{1}{2} \|a_i - a_j\|_2, \quad u^\top a_j - b = -\frac{1}{2} \|a_i - a_j\|_2.$$

1776 *Proof.* Compute

$$\begin{aligned} u^\top a_i - b &= u^\top a_i - \frac{1}{2} u^\top a_i - \frac{1}{2} u^\top a_j \\ &= \frac{1}{2} u^\top (a_i - a_j) \end{aligned}$$

$$= \frac{1}{2} \|a_i - a_j\|_2.$$

1777 Similarly,

$$\begin{aligned} u^\top a_j - b &= u^\top a_j - \frac{1}{2} u^\top a_i - \frac{1}{2} u^\top a_j \\ &= -\frac{1}{2} u^\top (a_i - a_j) \\ &= -\frac{1}{2} \|a_i - a_j\|_2. \end{aligned}$$

1778

□

1779 The construction is used only for separated anchors. In implementations, proposed edges with
1780 $\|a_i - a_j\|_2 \leq \eta_{\text{anchor}}$ are skipped or replaced by a fallback distinction, so the normalized direction
1781 is never formed from duplicate or numerically indistinguishable anchors.

1782 **Theorem Q.2** (CA graph generates a finite edge-skeleton information structure, formal version of
1783 Theorem J.6). *Let S be a finite set of binary generators $g_s : H \rightarrow \{0, 1\}$. Define*

$$\sigma_S(h) = (g_s(h))_{s \in S}.$$

1784 *The relation $h \sim_S h'$ iff $\sigma_S(h) = \sigma_S(h')$ is an equivalence relation. Its nonempty equivalence*
1785 *classes form a finite edge-skeleton partition $\mathcal{P}_S^G$ of H . If a new generator g is added, then*

$$\mathcal{P}_{S \cup \{g\}}^G = \mathcal{P}_S^G \vee \sigma(g),$$

1786 *where the right side denotes cutting each atom of $\mathcal{P}_S^G$ by $g^{-1}(1)$ and $g^{-1}(0)$ and keeping nonempty*
1787 *pieces.*

1788 *Proof.* Equality of signatures is reflexive, symmetric, and transitive, so $\sim_S$ is an equivalence relation.
1789 Equivalence classes are disjoint, nonempty, and cover H , hence they form a partition. Since S is
1790 finite, there are at most $2^{|S|}$ signatures, so the partition is finite.

1791 For the update, two points have the same signature under $S \cup \{g\}$ exactly when they have the same old
1792 S -signature and the same value of g . Therefore each new equivalence class is an old atom intersected
1793 with either $g^{-1}(1)$ or $g^{-1}(0)$, with empty intersections discarded. This is precisely $\mathcal{P}_S^G \vee \sigma(g)$. □

1794 **Theorem Q.3** (Anchor-based hard routing is graph-plus-anchor-order measurable, formal version of
1795 Theorem J.7). *Let E_t be finite and let $a_1, \dots, a_{N_t} \in \mathbb{R}^d$ be finite frozen anchors with deterministic*
1796 *tie-breaking. Let $\mathcal{F}_t^{\text{edge}} = \sigma(g_e : e \in E_t)$ and let $\mathcal{F}_t^{\text{anchor-order}}$ be generated by all pairwise*
1797 *anchor-distance comparison events*

$$\{h : \|h - a_i\|_2^2 \leq \|h - a_j\|_2^2\}.$$

1798 *The exact hard-route selected concept index, defined as the nearest eligible anchor with fallback to*
1799 *the nearest anchor, is measurable with respect to $\mathcal{F}_t^{\text{edge}} \vee \mathcal{F}_t^{\text{anchor-order}}$.*

1800 *Proof.* For each concept j , eligibility is an intersection of finitely many edge-bit events of the form

$$\{h : g_e(h) = 1\} \quad \text{or} \quad \{h : g_e(h) = 0\},$$

1801 depending on j 's recorded side of edge e . Hence the eligible set for j is $\mathcal{F}_t^{\text{edge}}$ -measurable.

1802 The anchor-order sigma-algebra determines the complete order of all anchors under the fixed tie-
1803 breaking rule. For any subset $R \subseteq \{1, \dots, N_t\}$, the event that concept j is the nearest anchor in R is a
1804 finite intersection of pairwise anchor-comparison events, hence belongs to $\mathcal{F}_t^{\text{anchor-order}}$. The selected
1805 hard-route index is obtained by combining finitely many eligibility events with these nearest-in-subset
1806 events. Therefore the inverse image of each selected index is a finite union of intersections of sets
1807 from $\mathcal{F}_t^{\text{edge}}$ and $\mathcal{F}_t^{\text{anchor-order}}$, and belongs to their join. □

1808 **Theorem Q.4** (Soft routing approaches hard routing, formal version of Theorem J.9). *Let N be a*
 1809 *finite candidate set and let*

$$\alpha_j = \frac{\exp(-d_j/T)\chi_j}{\sum_{r \in N} \exp(-d_r/T)\chi_r}.$$

1810 *If $j_{\text{hard}}^* \notin N$, where j_{hard}^* is the exact hard-route selected concept, then this sparse soft read gives no*
 1811 *guarantee for the omitted hard-route concept. Otherwise, assume $j^* = j_{\text{hard}}^* \in N$. For an eligible*
 1812 *competitor set $E \subseteq N \setminus \{j^*\}$, if $d_j - d_{j^*} \geq \delta > 0$ and $\chi_j/\chi_{j^*} \leq C_E < \infty$ for all $j \in E$, then*

$$\sum_{j \in E} \alpha_j \leq C_E |E| \exp(-\delta/T).$$

1813 *Moreover, suppose $\log \chi_j$ contains a term*

$$\frac{\beta_e}{\sqrt{D_j}} \log \text{sigmoid} \left(\frac{m_j}{\tau_e} \right)$$

1814 *with $\beta_e > 0$ and finite $D_j \geq 1$, and candidate j violates that edge with $m_j \leq -\delta_e < 0$. Then*
 1815 *$\chi_j \leq \exp(-\beta_e \delta_e / (\sqrt{D_j} \tau_e))$.*

1816 *For a wrong-side set $W \subseteq N$, suppose every $j \in W$ violates some incident structural edge whose*
 1817 *weight is at least $\beta_{\min} > 0$, whose signed margin is at most $-\delta_e$, and whose incident degree is at*
 1818 *most $D_{\max}$. If at least one eligible reference candidate r has $\chi_r \geq \chi_{\min} > 0$, and the distance score*
 1819 *ratio between every $j \in W$ and r is bounded by C , then the total wrong-side routing mass is at most*

$$\frac{C |N|}{\chi_{\min}} \exp \left(-\frac{\beta_{\min} \delta_e}{\sqrt{D_{\max} \tau_e}} \right).$$

1820 *More generally, if*

$$\Delta_d = \max_{j \in W} (d_r - d_j)_+,$$

1821 *then the same mass is at most*

$$\frac{|N|}{\chi_{\min}} \exp \left(\frac{\Delta_d}{T} - \frac{\beta_{\min} \delta_e}{\sqrt{D_{\max} \tau_e}} \right),$$

1822 *and therefore vanishes along any joint limit satisfying*

$$\frac{\beta_{\min} \delta_e}{\sqrt{D_{\max} \tau_e}} - \frac{\Delta_d}{T} \rightarrow +\infty.$$

1823 *Consequently, if $N \setminus \{j^*\}$ is covered by eligible competitors satisfying the distance-margin and*
 1824 *compatibility-ratio bounds and wrong-side competitors satisfying either wrong-side bound above,*
 1825 *then $\alpha_{j^*} \rightarrow 1$ under the corresponding temperature limit.*

1826 *Proof.* If $j_{\text{hard}}^* \notin N$, then the soft normalization assigns weight only to candidates in N and the
 1827 omitted hard-route concept has no route to receive weight. This is the candidate-recall caveat.

1828 For the eligible-competitor statement, the denominator is at least $\exp(-d_{j^*}/T)\chi_{j^*}$, so

$$\begin{aligned} \sum_{j \in E} \alpha_j &\leq \sum_{j \in E} \frac{\exp(-d_j/T)\chi_j}{\exp(-d_{j^*}/T)\chi_{j^*}} \\ &\leq C_E \sum_{j \in E} \exp(-(d_j - d_{j^*})/T) \leq C_E |E| \exp(-\delta/T). \end{aligned}$$

1829 For an edge violation, set $a = \delta_e/\tau_e > 0$. Since

$$\text{sigmoid}(-a) = \frac{1}{1 + \exp(a)} \leq \exp(-a),$$

1830 the corresponding log-compatibility contribution is at most $-\beta_e \delta_e / (\sqrt{D_j} \tau_e)$, and therefore $\chi_j \leq$
 1831 $\exp(-\beta_e \delta_e / (\sqrt{D_j} \tau_e))$. If W is the wrong-side set and r is an eligible candidate whose compatibility
 1832 satisfies $\chi_r \geq \chi_{\min}$, the uniform wrong-side assumptions give, for every $j \in W$,

$$\chi_j \leq \exp\left(-\frac{\beta_{\min} \delta_e}{\sqrt{D_{\max}} \tau_e}\right).$$

1833 Combining this with the assumed distance-score ratio bound gives

$$\begin{aligned} \sum_{j \in W} \alpha_j &\leq \sum_{j \in W} \frac{C}{\chi_{\min}} \exp\left(-\frac{\beta_{\min} \delta_e}{\sqrt{D_{\max}} \tau_e}\right) \\ &\leq \frac{C |N|}{\chi_{\min}} \exp\left(-\frac{\beta_{\min} \delta_e}{\sqrt{D_{\max}} \tau_e}\right). \end{aligned}$$

1834 Without a uniform distance-score ratio, use

$$\frac{\exp(-d_j/T)}{\exp(-d_r/T)} = \exp\left(\frac{d_r - d_j}{T}\right) \leq \exp(\Delta_d/T),$$

1835 which gives the joint-limit bound and the stated convergence condition. $\square$

1836 **Theorem Q.5** (Refine-or-repair objective descent, formal version of Theorem J.11). *Let*

$$J_{\mathcal{B}_t^{\text{score}}}(\mathcal{S}) = \hat{R}(\mathcal{S}; \mathcal{B}_t^{\text{score}}) + \lambda_{\text{concept}} N_{\text{concept}}(\mathcal{S}) + \lambda_{\text{edge}} |E(\mathcal{S})|$$

1837 *with nonnegative penalties on a fixed scoring buffer, where λ_{concept} includes the anchored concept*
 1838 *and the anchor-order bookkeeping induced by creating it. If a repair step is committed only after the*
 1839 *guarded test*

$$\hat{R}(\mathcal{S}_{t+1}; \mathcal{B}_t^{\text{score}}) \leq \hat{R}(\mathcal{S}_t; \mathcal{B}_t^{\text{score}}),$$

1840 *and leaves anchors, edges, and concept count unchanged, then $J_{\mathcal{B}_t^{\text{score}}}$ does not increase. If a*
 1841 *refinement creates one concept and actual edge set E_{new} and is accepted only when*

$$\hat{\Delta}_{\text{refine}} > \max\{\hat{\Delta}_{\text{repair}}, 0\} + \tau + 4\epsilon_m,$$

1842 *where $\hat{\Delta}_{\text{repair}} \geq 0$ is the empirical repair gain and*

$$\hat{\Delta}_{\text{refine}} = \hat{R}(\mathcal{S}_t; \mathcal{B}_t^{\text{score}}) - \hat{R}(\mathcal{S}_{t+1}; \mathcal{B}_t^{\text{score}}) - \lambda_{\text{concept}} - \lambda_{\text{edge}} |E_{\text{new}}|,$$

1843 *then*

$$J_{\mathcal{B}_t^{\text{score}}}(\mathcal{S}_{t+1}) < J_{\mathcal{B}_t^{\text{score}}}(\mathcal{S}_t) - \hat{\Delta}_{\text{repair}} - \tau - 4\epsilon_m.$$

1844 *In particular, $J_{\mathcal{B}_t^{\text{score}}}$ decreases by more than $\tau + 4\epsilon_m$.*

1845 *Proof.* For repair, N_{concept} and $|E|$ are unchanged. The empirical-risk term on $\mathcal{B}_t^{\text{score}}$ is no larger by
 1846 the guard, so $J_{\mathcal{B}_t^{\text{score}}}$ does not increase. The identity candidate is always feasible, so the guarded rule
 1847 can decline harmful centroid or memory movement instead of committing it.

1848 For refinement,

$$\begin{aligned} J_{\mathcal{B}_t^{\text{score}}}(\mathcal{S}_t) - J_{\mathcal{B}_t^{\text{score}}}(\mathcal{S}_{t+1}) &= \hat{R}(\mathcal{S}_t; \mathcal{B}_t^{\text{score}}) - \hat{R}(\mathcal{S}_{t+1}; \mathcal{B}_t^{\text{score}}) - \lambda_{\text{concept}} - \lambda_{\text{edge}} |E_{\text{new}}| \\ &= \hat{\Delta}_{\text{refine}} \\ &> \hat{\Delta}_{\text{repair}} + \tau + 4\epsilon_m. \end{aligned}$$

1849 Rearranging gives the claim. $\square$

1850 The theorem is local to the fixed scoring buffer used in its statement. If the scoring buffer changes
 1851 over time, these inequalities alone do not define a common global descent potential.

1852 **Theorem Q.6** (Statistically safe adaptive refinement, formal version of Theorem 4.1). Assume
 1853 losses lie in $[0, 1]$. Let $(\mathfrak{H}_t)$ be the filtration containing the past stream, current CA state, and all
 1854 proposal-buffer information available before scoring at time t . Let $\mathcal{C}_t$ be a finite candidate class
 1855 predictable with respect to $\mathfrak{H}_t$, and let $\mathcal{B}_t^{\text{score}}$ contain m_t fresh held-out samples, or samples not
 1856 previously used to choose proposals or update the state, independent of $\mathfrak{H}_t$ conditional on the current
 1857 scoring distribution. For candidate g , let $\hat{G}_t(g)$ be the empirical raw risk decrease on $\mathcal{B}_t^{\text{score}}$ before
 1858 complexity cost and let $G_t(g)$ be the corresponding population raw gain. Define

$$\epsilon_t = \sqrt{\frac{\log(4t^2(|\mathcal{C}_t| + 1)/\delta)}{2m_t}}.$$

1859 Then, with probability at least $1 - \delta$, every time t and candidate $g \in \mathcal{C}_t$ satisfy

$$\left| \hat{G}_t(g) - G_t(g) \right| \leq 2\epsilon_t.$$

1860 Consequently, if

$$\hat{G}_t(g) > \lambda \text{cost}(g) + \tau + 2\epsilon_t,$$

1861 then on the same event

$$G_t(g) > \lambda \text{cost}(g) + \tau.$$

1862 *Proof.* Condition on $\mathfrak{H}_t$. The candidate class is then fixed, and the scoring buffer is independent
 1863 of the proposal mechanism and prior state-selection decisions by assumption. Apply Hoeffding's
 1864 inequality to the empirical risk of the old state and to the empirical risk of each candidate-refined
 1865 state. For any fixed state in this finite collection,

$$\mathbb{P} \left(\left| \hat{R} - R \right| > \epsilon \right) \leq 2 \exp(-2m\epsilon^2).$$

1866 With $m = m_t$ and $\epsilon = \epsilon_t$, the failure probability at time t is at most $\delta/(2t^2)$ after a union bound over
 1867 the old state and the $|\mathcal{C}_t|$ candidate states. Since $\sum_{t \geq 1} t^{-2} < 2$, the displayed constant $4t^2$ gives total
 1868 failure probability at most δ over all times. On this event, each raw gain is a difference of two risks,
 1869 so empirical and population raw gain differ by at most $2\epsilon_t$. The implication follows by subtracting
 1870 this deviation from the empirical raw-gain lower bound. $\square$

1871 **Corollary Q.7** (Bounded language-model refinement score, formal version of Corollary J.13). For
 1872 language modeling, let

$$\ell_{\text{clip}}(x_{\leq t}, x_{t+1}) = \frac{1}{L_{\max}} \min\{-\log p(x_{t+1} \mid x_{\leq t}), L_{\max}\}$$

1873 for a fixed $L_{\max} > 0$. If the scoring buffer, empirical raw gain, and population raw gain in
 1874 Theorem Q.6 are computed using ℓ_{clip} , then the theorem applies to language-model refinement
 1875 decisions made with this clipped score. In particular, an accepted refinement has positive clipped
 1876 population gain after paying complexity cost and margin τ with probability at least $1 - \delta$.

1877 *Proof.* The clipped score satisfies $0 \leq \ell_{\text{clip}} \leq 1$. Substituting this bounded loss into Theorem Q.6
 1878 gives the claim directly. The statement is only about the clipped score; raw negative log likelihood is
 1879 unbounded and would require a separate concentration assumption. $\square$

1880 **Theorem Q.8** (Anchor-stable CA information structure, formal version of Theorem J.14). Let

$$\mathcal{F}_t^{\text{CA}} = \sigma(g_e : e \in E_t) \vee \sigma \left(\{h : \|h - a_i\|_2^2 \leq \|h - a_j\|_2^2 : 1 \leq i, j \leq N_t\} \right).$$

1881 Suppose every edge generator $g_e(h) = \mathbf{1}\{u_e^\top h > b_e\}$ is computed from frozen endpoint anchors
 1882 a_i, a_j . Any repair step that changes only centroids c_j , local memories M_j , and counts n_j , while
 1883 keeping anchors and edges fixed, leaves $\mathcal{F}_t^{\text{CA}}$ unchanged. If a refinement adds a new anchor and
 1884 new edge generators, then $\mathcal{F}_{t+1}^{\text{CA}}$ is $\mathcal{F}_t^{\text{CA}}$ joined with the new anchor-order events and the new edge
 1885 generators.

1886 *Proof.* For an edge $e = (i, j)$, both

$$u_e = \frac{a_i - a_j}{\|a_i - a_j\|_2}, \quad b_e = \frac{1}{2} u_e^\top (a_i + a_j)$$

1887 are deterministic functions of the frozen anchors. A repair step does not change these anchors, so it
 1888 does not change u_e , b_e , or g_e . It also does not change any anchor-distance comparison event because
 1889 all anchors are fixed. Hence both sigma-algebras in the join defining $\mathcal{F}_t^{\text{CA}}$ are unchanged.

1890 If a refinement adds anchor a_{new} and edge generators E_{new} , the new edge sigma-algebra is $\sigma(g_e : e \in E_t) \vee \sigma(g_e : e \in E_{\text{new}})$, and the new anchor-order sigma-algebra is the old anchor-order
 1891 sigma-algebra joined with the pairwise comparisons involving a_{new} . This is exactly the stated
 1892 join. $\square$

1894 **Corollary Q.9** (Finite number of accepted refinements, formal version of Corollary J.15). *Let Φ be a
 1895 common potential with $\Phi(\mathcal{S}) \geq 0$ for all reachable states. If each accepted refinement decreases Φ
 1896 by at least $\tau > 0$, then the number of accepted refinements is at most $\Phi(\mathcal{S}_0)/\tau$.*

1897 *Proof.* After m accepted refinements,

$$\Phi(\mathcal{S}_m) \leq \Phi(\mathcal{S}_0) - m\tau.$$

1898 Since $\Phi(\mathcal{S}_m) \geq 0$, we have $m \leq \Phi(\mathcal{S}_0)/\tau$. $\square$

1899 **Proposition Q.10** (Trees as generalized hard-route CAs, formal version of Proposition N.1). *Let
 1900 $\mathcal{T}$ be a binary decision tree whose internal nodes compute binary distinctions $g_\nu(h) \in \{0, 1\}$ and
 1901 whose leaves have predictors M_ℓ . Construct a generalized hard-route CA with one concept c_ℓ and
 1902 memory M_ℓ for each leaf ℓ , where the generalized model permits arbitrary finite binary eligibility
 1903 constraints in addition to pairwise distinction edges. For each leaf, add hard eligibility constraints
 1904 equal to the signed distinctions along the root-to-leaf path. If the generalized hard route selects the
 1905 unique eligible leaf concept, then its prediction equals the tree prediction for every h .*

1906 *Proof.* For any input h , the tree follows exactly one root-to-leaf path. The leaf reached by the tree
 1907 is the unique leaf whose path constraints are all satisfied by h . By construction, the corresponding
 1908 generalized hard-route concept has exactly those eligibility constraints, and all other leaf concepts
 1909 violate at least one path constraint at the first node where their path differs from the tree path.
 1910 Therefore the generalized hard route selects the same leaf and applies the same memory M_ℓ . $\square$

1911 **Theorem Q.11** (CA data-structure bound, formal version of Theorem N.2). *Assume CA has N_t
 1912 concepts in $\mathbb{R}^d$, exact centroid and anchor retrieval, candidate set $N(h)$, selected incident edge-gate
 1913 count $Q(h) = \sum_{j \in N(h)} |\text{Inc}_t(j)|$, and local-memory evaluation cost C_{local} . Then one prediction
 1914 costs $O(N_t d + Q(h)d + |N(h)|d + C_{\text{local}})$ for dense linear edge gates. One observation that
 1915 repairs a selected concept costs this plus the guarded local repair scoring cost. One observation that
 1916 creates a new anchor-centroid pair and q guarded distinction edges costs an additional $O(d + qd)$
 1917 plus anchor-order query bookkeeping. If each concept stores an anchor, a centroid, and a sparse
 1918 categorical memory with average support $\bar{s}$, and each edge stores a linear distinction in $\mathbb{R}^d$, then the
 1919 runtime state uses $O(N_t(2d + \bar{s}) + |E_t|d)$ memory.*

1920 *Proof.* A distance computation between $h \in \mathbb{R}^d$ and one concept center costs $O(d)$. Exact centroid
 1921 and anchor retrieval over N_t concepts therefore costs

$$O(N_t d).$$

1922 Graph-neighbor expansion has cost proportional to the incident lists touched and is lower order
 1923 relative to dense edge-gate evaluation. After the candidate set $N(h)$ is selected, evaluating the
 1924 incident linear edge gates costs $O(Q(h)d)$. Computing sparse routing weights, distances within the
 1925 selected set, and evaluating selected local memories costs

$$O(|N(h)|d + C_{\text{local}}).$$

1926 Thus prediction costs

$$O(N_t d + Q(h)d + |N(h)|d + C_{\text{local}}).$$

1927 Observation begins with the same retrieval computation. If it repairs a selected concept, it adds the
 1928 cost of evaluating and scoring the proposed local repair candidates on the scoring buffer. If it creates
 1929 a new anchor-centroid pair, storing the pair costs $O(d)$. For each new edge, computing and storing
 1930 the linear distinction vector costs $O(d)$, so q edges cost

$$O(qd).$$

1931 The total additional vector-update cost for refinement is therefore $O(d + qd)$, plus whatever indexing
 1932 or comparison bookkeeping is used to support anchor-order queries.

1933 For memory, each concept stores $a_j, c_j \in \mathbb{R}^d$ and a sparse categorical memory of average support $\bar{s}$,
 1934 giving

$$O(N_t(2d + \bar{s})).$$

1935 Each edge stores one distinction vector and threshold, giving

$$O(|E_t| d).$$

1936 Adding the two terms proves the claimed bound. $\square$

1937 **Theorem Q.12** (CA layer cost, formal version of Theorem N.3). *Consider a CA adapter extension*
 1938 *with hidden dimension d , query dimension d_q , local rank r , candidate count $|N|$, selected incident*
 1939 *edge-gate count Q_N , and concept-bank query cost $C_{\text{ret}}(N_t, d_q)$. For one token, the read overhead is*

$$O(dd_q + C_{\text{ret}}(N_t, d_q) + Q_N d_q + |N| dr).$$

1940 *If the write path is detached and uses rank-one local repair, the write overhead is*

$$O(dd_q + C_{\text{ret}}(N_t, d_q) + Q_N d_q + |N| (d_q + dr)).$$

1941 *For a block of length L , backpropagation through the slow parameters does not require storing the*
 1942 *full CA state history $\mathcal{S}_1, \dots, \mathcal{S}_L$.*

1943 *Proof.* For a token hidden state $h \in \mathbb{R}^d$, the query computation

$$q = Qh$$

1944 costs $O(dd_q)$. The retrieval cost is $C_{\text{ret}}(N_t, d_q)$ by definition. Evaluating selected incident dense
 1945 edge gates costs $O(Q_N d_q)$.

1946 For a selected concept j , the rank- r adapter read is

$$W_j K h.$$

1947 Multiplying one selected $W_j \in \mathbb{R}^{d \times r}$ by a rank- r key costs $O(dr)$. Therefore $|N|$ selected adapters
 1948 cost

$$O(|N| dr).$$

1949 The routing weights add lower-order top- k normalization cost, absorbed into the same bound. This
 1950 proves the read overhead.

1951 The write path first computes the same query, retrieval, and edge gates. It then updates selected
 1952 concept statistics and selected local memories. Concept bookkeeping costs $O(d_q)$ per selected
 1953 concept. A rank-one local repair of a rank- r or low-rank sufficient-statistic memory costs $O(dr)$ per
 1954 selected memory. Hence the write overhead is

$$O(dd_q + C_{\text{ret}}(N_t, d_q) + Q_N d_q + |N| (d_q + dr)).$$

1955 Finally, the update rule is applied as

$$\mathcal{S}_{t+1} = \text{stopgrad}(\text{CAObserve}(\text{stopgrad}(\mathcal{S}_t), \text{stopgrad}(h_t), x_{t+1})).$$

1956 Thus the computational graph for the loss contains the selected read computation, but not the chain of
 1957 state-update maps from $\mathcal{S}_1$ through $\mathcal{S}_L$ or the write-time hidden states used to mutate the fast state.
 1958 Therefore full CA state history is not required for backpropagation through the slow parameters. $\square$

1959 **Theorem Q.13** (One-hot memory is partition-measurable, formal version of Theorem M.1). *Let*
 1960 $\mathcal{P} = \{B_1, \dots, B_n\}$ *be a finite partition and let* $\phi_{\mathcal{P}}(\omega) = e_i$ *for* $\omega \in B_i$. *For every* $W \in \mathbb{R}^{m \times n}$, *the*
 1961 *map* $f_W(\omega) = W\phi_{\mathcal{P}}(\omega)$ *is* $\mathcal{P}$ -*measurable. Conversely, every* $\mathcal{P}$ -*measurable map* $f : \Omega \rightarrow \mathbb{R}^m$ *is*
 1962 *equal to* f_W *for some* $W \in \mathbb{R}^{m \times n}$.

1963 *Proof.* If $\omega, \omega' \in B_i$, then

$$\phi_{\mathcal{P}}(\omega) = \phi_{\mathcal{P}}(\omega') = e_i.$$

1964 Thus

$$f_W(\omega) = We_i = f_W(\omega'),$$

1965 so f_W is constant on each atom and is $\mathcal{P}$ -measurable.

1966 Conversely, if f is $\mathcal{P}$ -measurable, it is constant on each B_i . Let w_i be this constant value on B_i , and
 1967 define

$$W = [w_1, \dots, w_n].$$

1968 Then for $\omega \in B_i$,

$$W\phi_{\mathcal{P}}(\omega) = We_i = w_i = f(\omega).$$

1969 □

1970 **Theorem Q.14** (CPM conflict equals missing distinction, formal version of Theorem M.2). *Let*
 1971 $\mathcal{P} = \{B_1, \dots, B_n\}$ *and let* $\omega, \omega' \in B_i$. *Suppose* $v(\omega) \neq v(\omega')$. *Let* $k = \phi_{\mathcal{P}}(\omega) = \phi_{\mathcal{P}}(\omega') = e_i$. *If*
 1972 *a memory* W *has already committed* $We_i = v(\omega)$, *then the evidence* $We_i = v(\omega')$ *has innovation*
 1973 $z = 0$ *relative to the old key span and residual* $e \neq 0$.

1974 *Proof.* Here the stored-key matrix K is oriented with stored keys as columns, so $KK^\dagger$ is the
 1975 orthogonal projector onto their column span. Since $\omega, \omega' \in B_i$,

$$\phi_{\mathcal{P}}(\omega) = \phi_{\mathcal{P}}(\omega') = e_i.$$

1976 The old key span contains e_i . Therefore the innovation of the new key $k = e_i$ relative to the old key
 1977 matrix $K = e_i$ is

$$z = (I - KK^\dagger)k = (I - e_i e_i^\top)e_i = 0.$$

1978 The new residual is

$$e = v(\omega') - We_i = v(\omega') - v(\omega).$$

1979 Since $v(\omega') \neq v(\omega)$, we have $e \neq 0$. □

1980 **NeurIPS Paper Checklist**

1981 This file is retained only for the conference-template branch of `main.tex`.

- 1982
- 1983
- 1984
- 1985
- 1986
- 1987
- 1988
- 1989
- 1990
- 1991
- **Claims.** The abstract and introduction state the paper’s scope as a method proposal: CA replaces token-indexed attention memory with a fixed bank of runtime concept slots.
 - **Limitations.** Appendix B states the main limitations, including remaining kernel and sparse-retrieval work, retention tuning, concept-count selection, and the fact that 180M–440M LM results require completed training and latency reporting before any large-scale win can be claimed.
 - **Theory.** Section 5 gives the state definition, read/write equations, exact chunk-scan interface, state-size accounting, and backpropagation boundary for fixed-bank CA.
 - **Experiments.** Section 6 centers on MQAR as the controlled recall benchmark and a PoST-style CoM pretraining/evaluation protocol as the large-scale sequence-modeling test.